
The Python GTK+ 3 Tutorial

Release 3.4

Sebastian Pölsterl

Dec 20, 2022

CONTENTS

1	Installation	3
1.1	Dependencies	3
1.2	Prebuilt Packages	3
1.3	Installing From Source	3
2	Getting Started	5
2.1	Simple Example	5
2.2	Extended Example	6
3	Basics	9
3.1	Main loop and Signals	9
3.2	Properties	10
4	How to Deal With Strings	11
4.1	Definitions	11
4.2	Python 2	11
4.3	Python 3	13
4.4	References	13
5	Widget Gallery	15
6	Layout Containers	35
6.1	Boxes	35
6.2	Grid	36
6.3	ListBox	38
6.4	Stack and StackSwitcher	40
6.5	HeaderBar	42
6.6	FlowBox	43
6.7	Notebook	46
7	Label	49
7.1	Example	50
8	Entry	53
8.1	Example	53
9	Button Widgets	57
9.1	Button	57
9.2	ToggleButton	58
9.3	CheckButton	59
9.4	RadioButton	59

9.5	LinkButton	61
9.6	SpinButton	61
9.7	Switch	63
10	Expander	65
10.1	Example	65
11	ProgressBar	67
11.1	Example	67
12	Spinner	71
12.1	Example	71
12.2	Extended example	72
13	Tree and List Widgets	77
13.1	The Model	77
13.2	The View	79
13.3	The Selection	80
13.4	Sorting	80
13.5	Filtering	82
14	CellRenderers	85
14.1	CellRendererText	85
14.2	CellRendererToggle	87
14.3	CellRendererPixbuf	89
14.4	CellRendererCombo	90
14.5	CellRendererProgress	92
14.6	CellRendererSpin	93
15	ComboBox	97
15.1	Example	97
16	IconView	101
16.1	Example	101
17	Multiline Text Editor	103
17.1	The View	103
17.2	The Model	103
17.3	Tags	104
17.4	Example	105
18	Dialogs	111
18.1	Custom Dialogs	111
18.2	MessageDialog	113
18.3	FileChooserDialog	115
19	Popovers	119
19.1	Custom Popover	119
19.2	Menu Popover	120
19.3	See Also	123
20	Clipboard	125
20.1	Example	125
21	Drag and Drop	127
21.1	Target Entries	127

21.2	Drag Source Signals	128
21.3	Drag Destination Signals	128
21.4	Example	128
22	Glade and Gtk.Builder	131
22.1	Creating and loading the .glade file	131
22.2	Accessing widgets	132
22.3	Connecting Signals	132
22.4	Example	133
22.5	Gtk.Template	134
23	Objects	137
23.1	Inherit from GObject.GObject	137
23.2	Signals	137
23.3	Properties	138
23.4	API	141
24	Application	145
24.1	Actions	145
24.2	Menus	145
24.3	Command Line	146
24.4	Example	146
24.5	See Also	149
25	Menus	151
25.1	Actions	151
25.2	UI Manager	152
25.3	Example	152
26	Table	157
26.1	Example	157
27	Indices and tables	159
	Index	161

Release 3.4

Date Dec 20, 2022

Copyright GNU Free Documentation License 1.3 with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts

This tutorial gives an introduction to writing GTK+ 3 applications in Python.

Prior to working through this tutorial, it is recommended that you have a reasonable grasp of the Python programming language. GUI programming introduces new problems compared to interacting with the standard output (console / terminal). It is necessary for you to know how to create and run Python files, understand basic interpreter errors, and work with strings, integers, floats and Boolean values. For the more advanced widgets in this tutorial, good knowledge of lists and tuples will be needed.

Although this tutorial describes the most important classes and methods within GTK+ 3, it is not supposed to serve as an API reference. Please refer to the [GTK+ 3 Reference Manual](#) for a detailed description of the API. Also there's a [Python-specific reference](#) available.

Contents:

INSTALLATION

The first step before we start with actual coding consists of setting up [PyGObject](#) and its dependencies. PyGObject is a Python module that enables developers to access GObject-based libraries such as GTK+ within Python. It exclusively supports GTK+ version 3 or later.

1.1 Dependencies

- GTK+3
- Python 2 (2.6 or later) or Python 3 (3.1 or later)
- gobject-introspection

1.2 Prebuilt Packages

Recent versions of PyGObject and its dependencies are packaged by nearly all major Linux distributions. So, if you use Linux, you can probably get started by installing the package from the official repository for your distribution.

1.3 Installing From Source

The easiest way to install PyGObject from source is using [JHBuild](#). It is designed to easily build source packages and discover what dependencies need to be build and in what order. To setup JHBuild, please follow the [JHBuild manual](#).

Once you have installed JHBuild successfully, download the latest configuration from¹. Copy files with the suffix *.modules* to JHBuild's module directory and the file *sample-tarball.jhbuildrc* to *~/jhbuildrc*.

If you have not done it before, verify that your build environment is setup correctly by running:

```
$ jhbuild sanitycheck
```

It will print any applications and libraries that are currently missing on your system but required for building. You should install those using your distribution's package repository. A list of [package names](#) for different distributions is maintained on the GNOME wiki. Run the command above again to ensure the required tools are present.

Executing the following command will build PyGObject and all its dependencies:

```
$ jhbuild build pygobject
```

Finally, you might want to install GTK+ from source as well:

¹ <https://download.gnome.org/teams/releng/>

```
$ jhbuild build gtk+-3
```

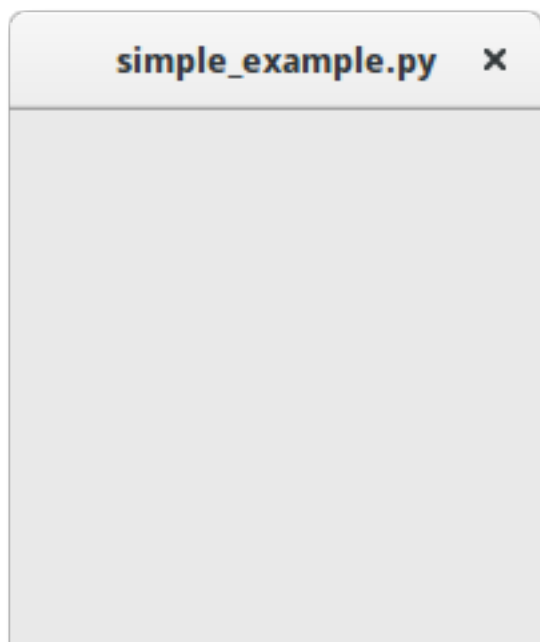
To start a shell with the same environment as used by JHBuild, run:

```
$ jhbuild shell
```

GETTING STARTED

2.1 Simple Example

To start with our tutorial we create the simplest program possible. This program will create an empty 200 x 200 pixel window.



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6 win = Gtk.Window()
7 win.connect("destroy", Gtk.main_quit)
8 win.show_all()
9 Gtk.main()
```

We will now explain each line of the example.

```
import gi
```

(continues on next page)

(continued from previous page)

```
gi.require_version("Gtk", "3.0")
from gi.repository import Gtk
```

In the beginning, we have to import the Gtk module to be able to access GTK+'s classes and functions. Since a user's system can have multiple versions of GTK+ installed at the same, we want to make sure that when we import Gtk that it refers to GTK+ 3 and not any other version of the library, which is the purpose of the statement `gi.require_version('Gtk', '3.0')`.

The next line creates an empty window.

```
win = Gtk.Window()
```

Followed by connecting to the window's delete event to ensure that the application is terminated if we click on the *x* to close the window.

```
win.connect("destroy", Gtk.main_quit)
```

In the next step we display the window.

```
win.show_all()
```

Finally, we start the GTK+ processing loop which we quit when the window is closed (see line 6).

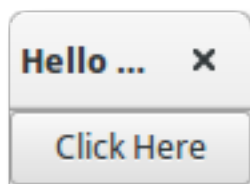
```
Gtk.main()
```

To run the program, open a terminal, change to the directory of the file, and enter:

```
python simple_example.py
```

2.2 Extended Example

For something a little more useful, here's the PyGObject version of the classic "Hello World" program.



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class MyWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="Hello World")
10
11         self.button = Gtk.Button(label="Click Here")
12         self.button.connect("clicked", self.on_button_clicked)
13         self.add(self.button)
```

(continues on next page)

(continued from previous page)

```
14
15     def on_button_clicked(self, widget):
16         print("Hello World")
17
18
19 win = MyWindow()
20 win.connect("destroy", Gtk.main_quit)
21 win.show_all()
22 Gtk.main()
```

This example differs from the simple example as we sub-class `Gtk.Window` to define our own `MyWindow` class.

```
class MyWindow(Gtk.Window):
```

In the class's constructor we have to call the constructor of the super class. In addition, we tell it to set the value of the property *title* to *Hello World*.

```
    super().__init__(title="Hello World")
```

The next three lines are used to create a button widget, connect to its *clicked* signal and add it as child to the top-level window.

```
        self.button = Gtk.Button(label="Click Here")
        self.button.connect("clicked", self.on_button_clicked)
        self.add(self.button)
```

Accordingly, the method `on_button_clicked()` will be called if you click on the button.

```
    def on_button_clicked(self, widget):
        print("Hello World")
```

The last block, outside of the class, is very similar to the simple example above, but instead of creating an instance of the generic `Gtk.Window` class, we create an instance of `MyWindow`.

This section will introduce some of the most important aspects of GTK+.

3.1 Main loop and Signals

Like most GUI toolkits, GTK+ uses an event-driven programming model. When the user is doing nothing, GTK+ sits in the main loop and waits for input. If the user performs some action - say, a mouse click - then the main loop “wakes up” and delivers an event to GTK+.

When widgets receive an event, they frequently emit one or more signals. Signals notify your program that “something interesting happened” by invoking functions you’ve connected to the signal. Such functions are commonly known as *callbacks*. When your callbacks are invoked, you would typically take some action - for example, when an Open button is clicked you might display a file chooser dialog. After a callback finishes, GTK+ will return to the main loop and await more user input.

A generic example is:

```
handler_id = widget.connect("event", callback, data)
```

Firstly, *widget* is an instance of a widget we created earlier. Next, the event we are interested in. Each widget has its own particular events which can occur. For instance, if you have a button you usually want to connect to the “clicked” event. This means that when the button is clicked, the signal is issued. Thirdly, the *callback* argument is the name of the callback function. It contains the code which runs when signals of the specified type are issued. Finally, the *data* argument includes any data which should be passed when the signal is issued. However, this argument is completely optional and can be left out if not required.

The function returns a number that identifies this particular signal-callback pair. It is required to disconnect from a signal such that the callback function will not be called during any future or currently ongoing emissions of the signal it has been connected to.

```
widget.disconnect(handler_id)
```

If you have lost the “handler_id” for some reason (for example the handlers were installed using `Gtk.Builder.connect_signals()`), you can still disconnect a specific callback using the function `disconnect_by_func()`:

```
widget.disconnect_by_func(callback)
```

Applications should connect to the “destroy” signal of the top-level window. It is emitted when an object is destroyed, so when a user requests that a toplevel window is closed, the default handler for this signal destroys the window, but does not terminate the application. Connecting the “destroy” signal of the top-level window to the function `Gtk.main_quit()` will result in the desired behaviour.

```
window.connect("destroy", Gtk.main_quit)
```

Calling `Gtk.main_quit()` makes the main loop inside of `Gtk.main()` return.

3.2 Properties

Properties describe the configuration and state of widgets. As for signals, each widget has its own particular set of properties. For example, a button has the property “label” which contains the text of the label widget inside the button. You can specify the name and value of any number of properties as keyword arguments when creating an instance of a widget. To create a label aligned to the right with the text “Hello World” and an angle of 25 degrees, use:

```
label = Gtk.Label(label="Hello World", angle=25, halign=Gtk.Align.END)
```

which is equivalent to

```
label = Gtk.Label()
label.set_label("Hello World")
label.set_angle(25)
label.set_halign(Gtk.Align.END)
```

Instead of using getters and setters you can also get and set the widget properties through the “props” property such as `widget.props.prop_name = value`. This is equivalent to the more verbose `widget.get_property("prop-name")` and `widget.set_property("prop-name", value)`.

To see which properties are available for a widget in the running version of GTK you can “dir” the “props” property:

```
widget = Gtk.Box()
print(dir(widget.props))
```

This will print in the console the list of properties a `Gtk.Box` has.

HOW TO DEAL WITH STRINGS

This section explains how strings are represented in Python 2.x, Python 3.x and GTK+ and discusses common errors that arise when working with strings.

4.1 Definitions

Conceptually, a string is a list of characters such as ‘A’, ‘B’, ‘C’ or ‘É’. **Characters** are abstract representations and their meaning depends on the language and context they are used in. The Unicode standard describes how characters are represented by **code points**. For example the characters above are represented with the code points U+0041, U+0042, U+0043, and U+00C9, respectively. Basically, code points are numbers in the range from 0 to 0x10FFFF.

As mentioned earlier, the representation of a string as a list of code points is abstract. In order to convert this abstract representation into a sequence of bytes, the Unicode string must be **encoded**. The simplest form of encoding is ASCII and is performed as follows:

1. If the code point is < 128, each byte is the same as the value of the code point.
2. If the code point is 128 or greater, the Unicode string can’t be represented in this encoding. (Python raises a `UnicodeEncodeError` exception in this case.)

Although ASCII encoding is simple to apply it can only encode for 128 different characters which is hardly enough. One of the most commonly used encodings that addresses this problem is UTF-8 (it can handle any Unicode code point). UTF stands for “Unicode Transformation Format”, and the ‘8’ means that 8-bit numbers are used in the encoding.

4.2 Python 2

4.2.1 Python 2.x’s Unicode Support

Python 2 comes with two different kinds of objects that can be used to represent strings, `str` and `unicode`. Instances of the latter are used to express Unicode strings, whereas instances of the `str` type are byte representations (the encoded string). Under the hood, Python represents Unicode strings as either 16- or 32-bit integers, depending on how the Python interpreter was compiled. Unicode strings can be converted to 8-bit strings with `unicode.encode()`:

```
>>> unicode_string = u"Fu\u00dfb\u00e4lle"
>>> print unicode_string
Fußbälle
>>> type(unicode_string)
<type 'unicode'>
>>> unicode_string.encode("utf-8")
'Fu\xc3\x9fb\xc3\xa4lle'
```

Python's 8-bit strings have a `str.decode()` method that interprets the string using the given encoding:

```
>>> utf8_string = unicode_string.encode("utf-8")
>>> type(utf8_string)
<type 'str'>
>>> u2 = utf8_string.decode("utf-8")
>>> unicode_string == u2
True
```

Unfortunately, Python 2.x allows you to mix unicode and `str` if the 8-bit string happened to contain only 7-bit (ASCII) bytes, but would get `UnicodeDecodeError` if it contained non-ASCII values:

```
>>> utf8_string = " sind rund"
>>> unicode_string + utf8_string
u'Fu\xdfb\xe4lle sind rund'
>>> utf8_string = " k\xc3\xb6nnten rund sein"
>>> print utf8_string
können rund sein
>>> unicode_string + utf8_string
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 2:
ordinal not in range(128)
```

4.2.2 Unicode in GTK+

GTK+ uses UTF-8 encoded strings for all text. This means that if you call a method that returns a string you will always obtain an instance of the `str` type. The same applies to methods that expect one or more strings as parameter, they must be UTF-8 encoded. However, for convenience PyGObject will automatically convert any unicode instance to `str` if supplied as argument:

```
>>> from gi.repository import Gtk
>>> label = Gtk.Label()
>>> unicode_string = u"Fu\u00dfb\u00e4lle"
>>> label.set_text(unicode_string)
>>> txt = label.get_text()
>>> type(txt), txt
(<type 'str'>, 'Fu\xc3\x9fb\xc3\xa4lle')
>>> txt == unicode_string
__main__:1: UnicodeWarning: Unicode equal comparison failed to convert
both arguments to Unicode - interpreting them as being unequal
False
```

Note the warning at the end. Although we called `Gtk.Label.set_text()` with a unicode instance as argument, `Gtk.Label.get_text()` will always return a `str` instance. Accordingly, `txt` and `unicode_string` are *not* equal.

This is especially important if you want to internationalize your program using `gettext`. You have to make sure that `gettext` will return UTF-8 encoded 8-bit strings for all languages. In general it is recommended to not use unicode objects in GTK+ applications at all and only use UTF-8 encoded `str` objects since GTK+ does not fully integrate with unicode objects. Otherwise, you would have to decode the return values to Unicode strings each time you call a GTK+ method:

```
>>> txt = label.get_text().decode("utf-8")
>>> txt == unicode_string
True
```

4.3 Python 3

4.3.1 Python 3.x's Unicode support

Since Python 3.0, all strings are stored as Unicode in an instance of the `str` type. *Encoded* strings on the other hand are represented as binary data in the form of instances of the `bytes` type. Conceptually, `str` refers to *text*, whereas `bytes` refers to *data*. Use `str.encode()` to go from `str` to `bytes`, and `bytes.decode()` to go from `bytes` to `str`.

In addition, it is no longer possible to mix Unicode strings with encoded strings, because it will result in a `TypeError`:

```
>>> text = "Fu\u00dfb\u00e4lle"
>>> data = b" sind rund"
>>> text + data
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't convert 'bytes' object to str implicitly
>>> text + data.decode("utf-8")
'Fußbälle sind rund'
>>> text.encode("utf-8") + data
b'Fu\xc3\x9fb\xc3\xa4lle sind rund'
```

4.3.2 Unicode in GTK+

As a consequence, things are much cleaner and consistent with Python 3.x, because `PyGObject` will automatically encode/decode to/from UTF-8 if you pass a string to a method or a method returns a string. Strings, or *text*, will always be represented as instances of `str` only:

```
>>> from gi.repository import Gtk
>>> label = Gtk.Label()
>>> text = "Fu\u00dfb\u00e4lle"
>>> label.set_text(text)
>>> txt = label.get_text()
>>> type(txt), txt
(<class 'str'>, 'Fußbälle')
>>> txt == text
True
```

4.4 References

[What's new in Python 3.0](#) describes the new concepts that clearly distinguish between text and data.

The [Unicode HOWTO](#) discusses Python 2.x's support for Unicode, and explains various problems that people commonly encounter when trying to work with Unicode.

The [Unicode HOWTO for Python 3.x](#) discusses Unicode support in Python 3.x.

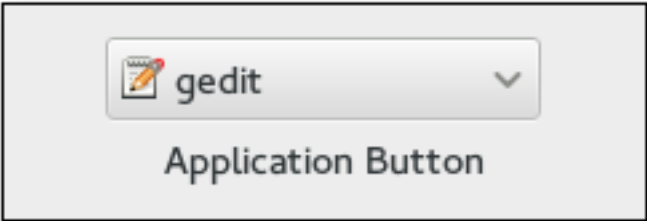
[UTF-8 encoding table and Unicode characters](#) contains a list of Unicode code points and their respective UTF-8 encoding.

WIDGET GALLERY

<code>Gtk.AboutDialog</code>	
<code>Gtk.AccelLabel</code>	

cont

Table 1 – continued from previous page

<code>Gtk.ActionBar</code>	
<code>Gtk.AppChooserButton</code>	

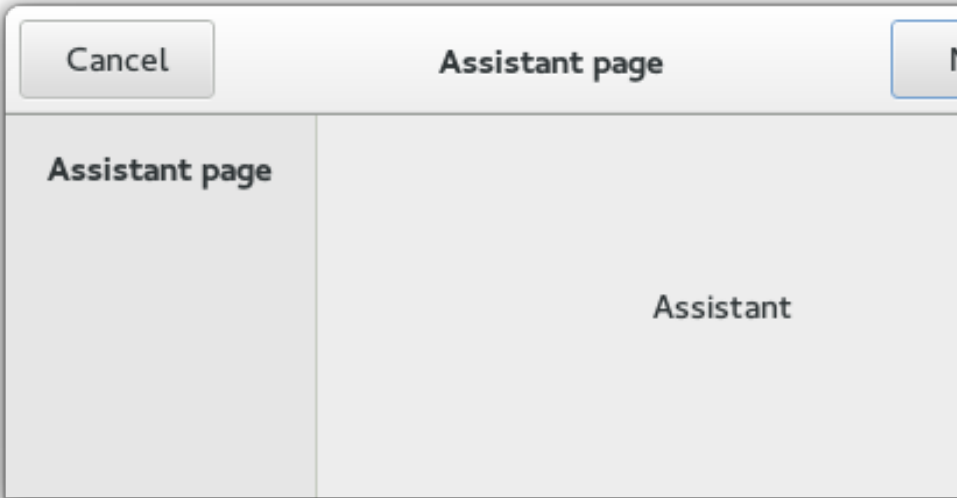
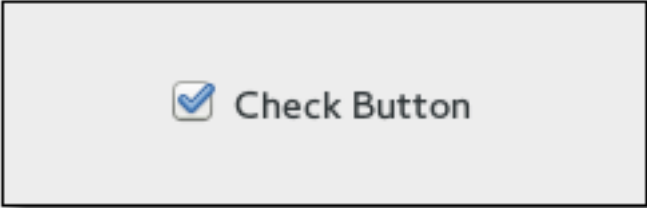
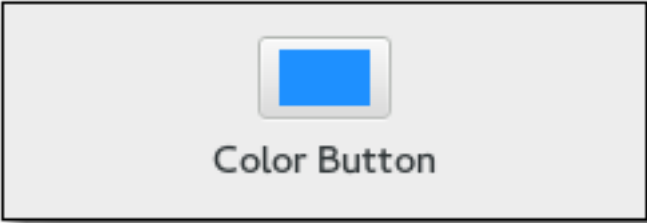
cont

Table 1 – continued from previous page

<code>Gtk.AppChooserDialog</code>	
--	--

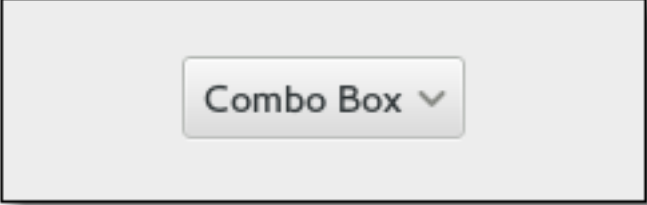
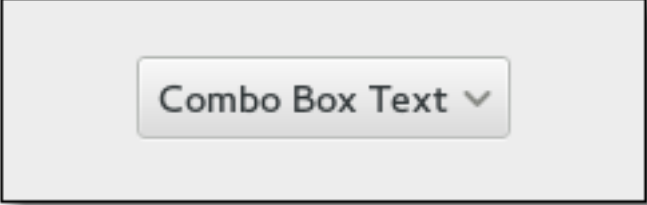
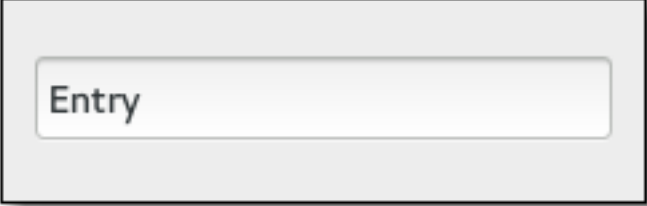
cont

Table 1 – continued from previous page

<code>Gtk.Assistant</code>	
<code>Gtk.Button</code>	
<code>Gtk.CheckButton</code>	
<code>Gtk.ColorButton</code>	

cont

Table 1 – continued from previous page

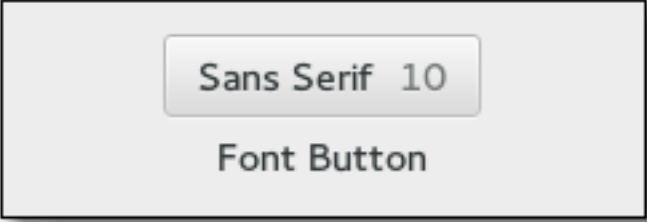
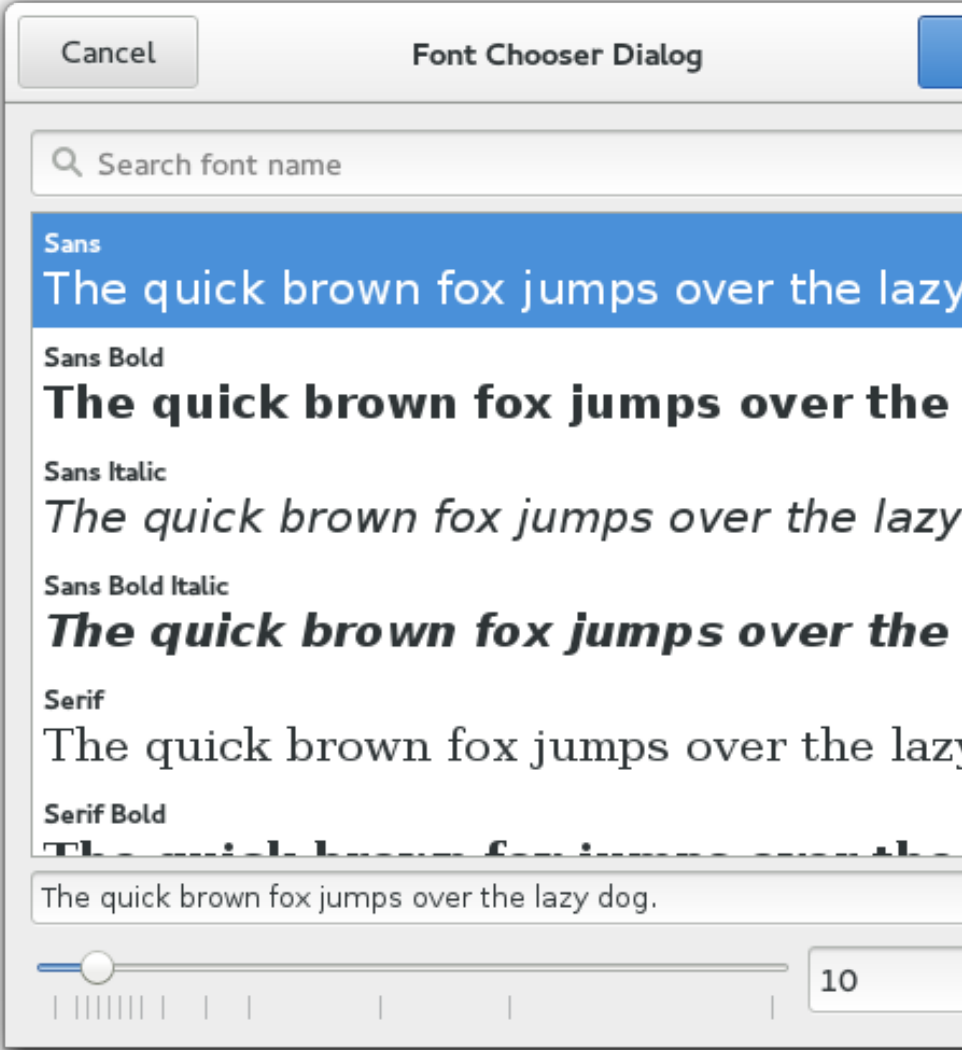
<code>Gtk.ColorChooserDialog</code>	
<code>Gtk.ComboBox</code>	
<code>Gtk.ComboBoxText</code>	
<code>Gtk.Entry</code>	

cont

Table 1 – continued from previous page

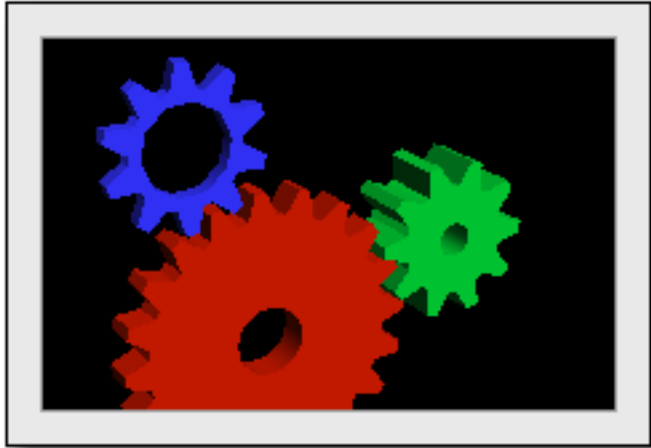
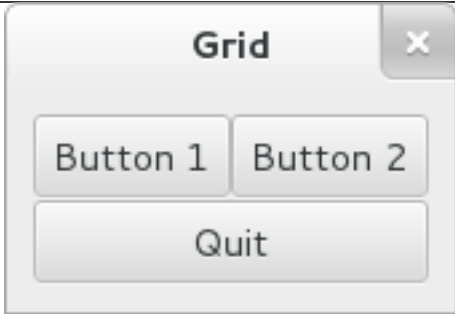
	(None)  File Button (Files)  Documents  File Button (Select Folder)																														
Gtk.FileChooserButton																															
	Cancel File Chooser Dialog  Recently Used Places Recent Home Documents Downloads Music Pictures Videos images <table><thead><tr><th>Name</th><th>Size</th><th>Me</th></tr></thead><tbody><tr><td> printdialog.png</td><td>42.8 kB</td><td>14</td></tr><tr><td> levelbar.png</td><td>2.4 kB</td><td>14</td></tr><tr><td> document-save-as.png</td><td>38.3 kB</td><td>08</td></tr><tr><td> document-save.png</td><td>39.8 kB</td><td>08</td></tr><tr><td> Screenshot from 2014-02-14 15:14:48.png</td><td>59.5 kB</td><td>Fri</td></tr><tr><td> list-and-tree.png</td><td>4.0 kB</td><td>Th</td></tr><tr><td> link-button.png</td><td>1.8 kB</td><td>Th</td></tr><tr><td> notebook.png</td><td>5.5 kB</td><td>Th</td></tr><tr><td> messagedialog.png</td><td>7.1 kB</td><td>Th</td></tr></tbody></table>	Name	Size	Me	 printdialog.png	42.8 kB	14	 levelbar.png	2.4 kB	14	 document-save-as.png	38.3 kB	08	 document-save.png	39.8 kB	08	 Screenshot from 2014-02-14 15:14:48.png	59.5 kB	Fri	 list-and-tree.png	4.0 kB	Th	 link-button.png	1.8 kB	Th	 notebook.png	5.5 kB	Th	 messagedialog.png	7.1 kB	Th
Name	Size	Me																													
 printdialog.png	42.8 kB	14																													
 levelbar.png	2.4 kB	14																													
 document-save-as.png	38.3 kB	08																													
 document-save.png	39.8 kB	08																													
 Screenshot from 2014-02-14 15:14:48.png	59.5 kB	Fri																													
 list-and-tree.png	4.0 kB	Th																													
 link-button.png	1.8 kB	Th																													
 notebook.png	5.5 kB	Th																													
 messagedialog.png	7.1 kB	Th																													
Gtk.FileChooserDialog																															
	Child One Child Two Child Three  Flow Box																														
Gtk.FlowBox																															

Table 1 – continued from previous page

Gtk.FontButton	
Gtk.FontChooserDialog	

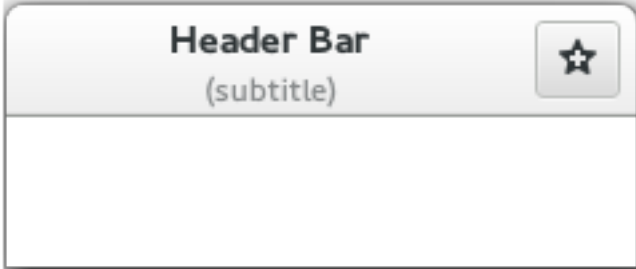
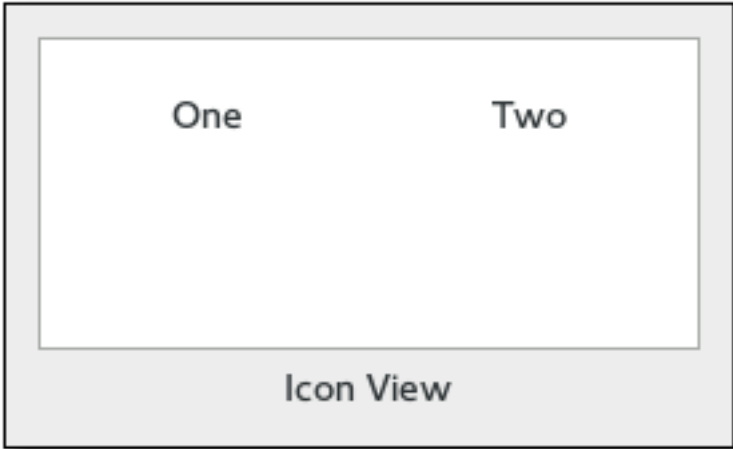
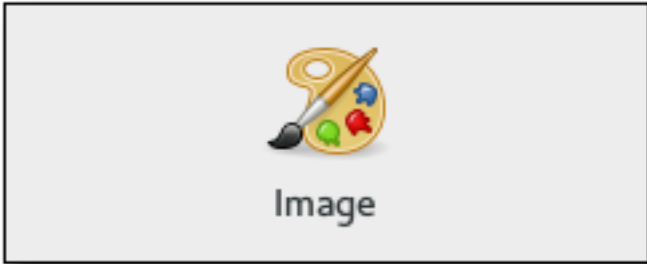
cont

Table 1 – continued from previous page

<code>Gtk.Frame</code>	
<code>Gtk.GLArea</code>	
<code>Gtk.Grid</code>	

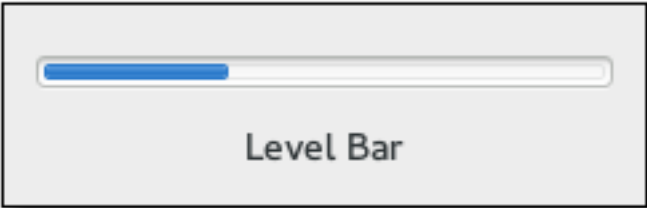
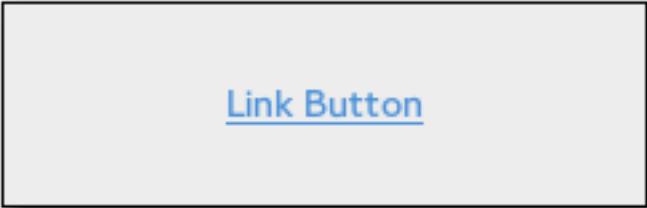
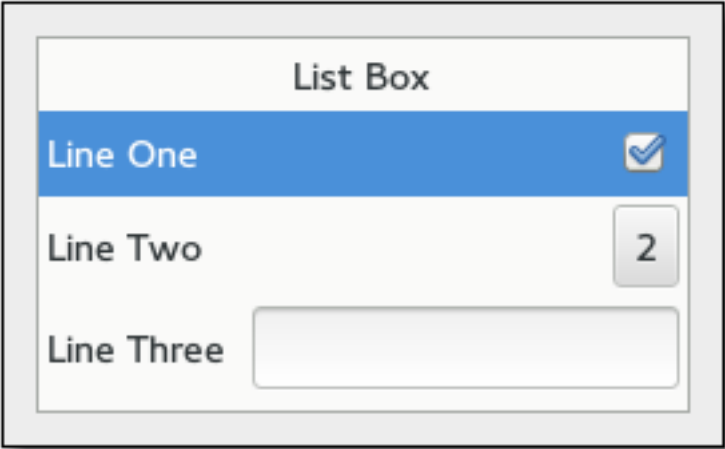
cont

Table 1 – continued from previous page

<code>Gtk.HeaderBar</code>	
<code>Gtk.IconView</code>	
<code>Gtk.Image</code>	
<code>Gtk.InfoBar</code>	

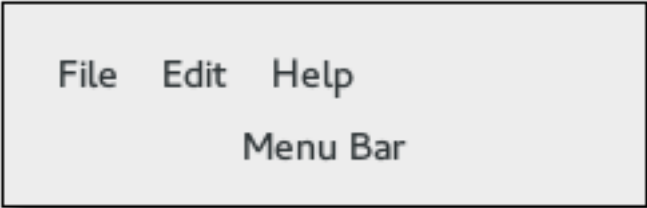
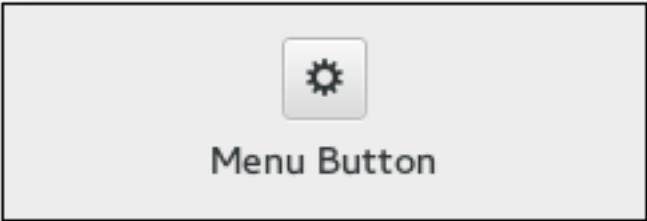
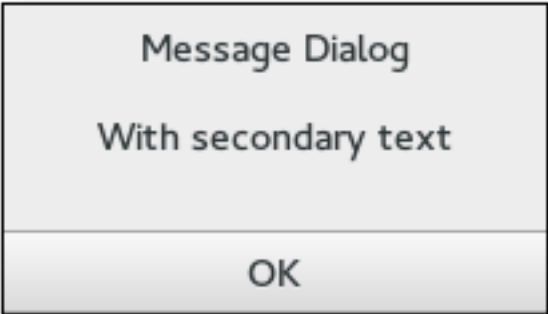
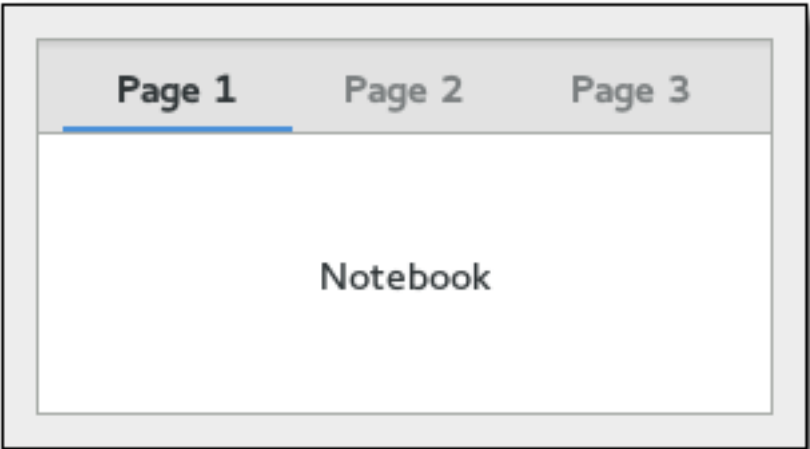
cont

Table 1 – continued from previous page

<code>Gtk.Label</code>	
<code>Gtk.LevelBar</code>	
<code>Gtk.LinkButton</code>	
<code>Gtk.ListBox</code>	
<code>Gtk.LockButton</code>	

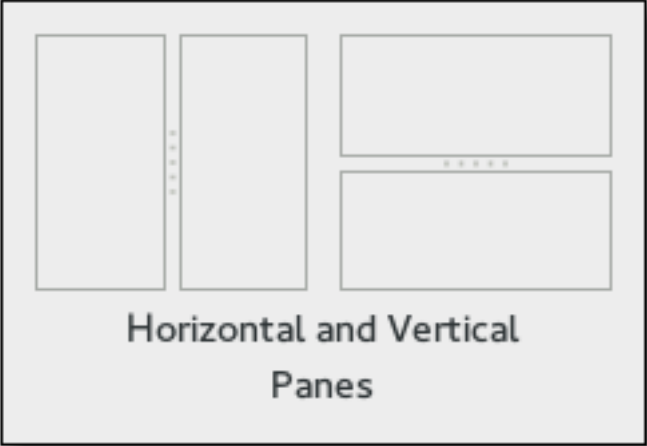
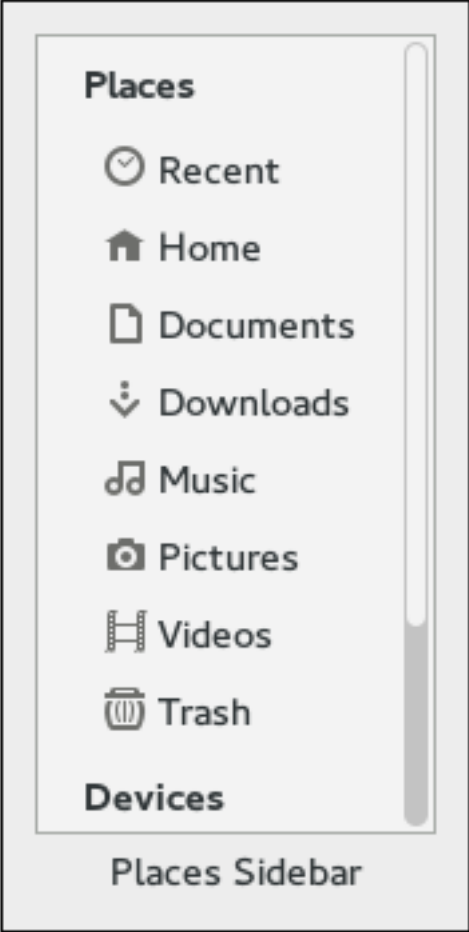
cont

Table 1 – continued from previous page

<code>Gtk.MenuBar</code>	 A horizontal menu bar with three items: "File", "Edit", and "Help". Below the items, the text "Menu Bar" is centered.
<code>Gtk.MenuButton</code>	 A rectangular button with a gear icon in the center. Below the icon, the text "Menu Button" is centered.
<code>Gtk.MessageDialog</code>	 A message dialog box. The title bar says "Message Dialog". The main area contains the text "With secondary text". At the bottom is an "OK" button.
<code>Gtk.Notebook</code>	 A notebook widget with three tabs: "Page 1", "Page 2", and "Page 3". "Page 1" is selected and highlighted with a blue underline. The main area of the notebook is labeled "Notebook".

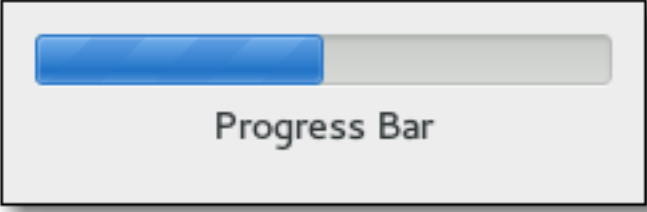
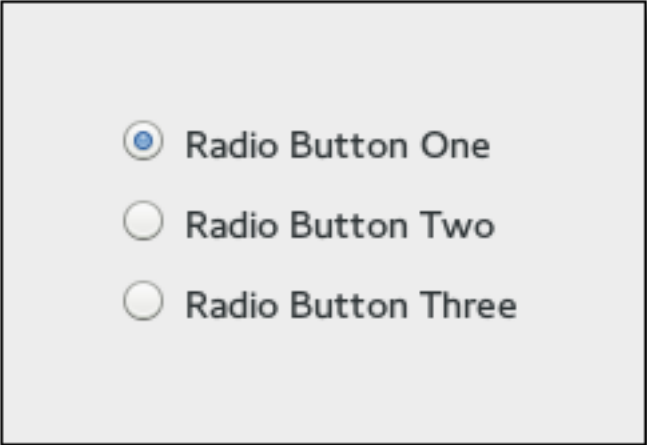
cont

Table 1 – continued from previous page

<code>Gtk.Paned</code>	 The diagram shows a rectangular window titled "Horizontal and Vertical Panes". Inside, there are four rectangular panes. On the left, two panes are stacked vertically, separated by a vertical line with a double-headed arrow. On the right, two panes are stacked horizontally, separated by a horizontal line with a double-headed arrow.
<code>Gtk.PlacesSidebar</code>	 The diagram shows a vertical sidebar titled "Places Sidebar". It contains a list of "Places" with icons: Recent (clock), Home (house), Documents (folder), Downloads (downward arrow), Music (musical notes), Pictures (camera), Videos (film strip), and Trash (trash can). Below these is a section titled "Devices". A vertical scrollbar is on the right side of the list.

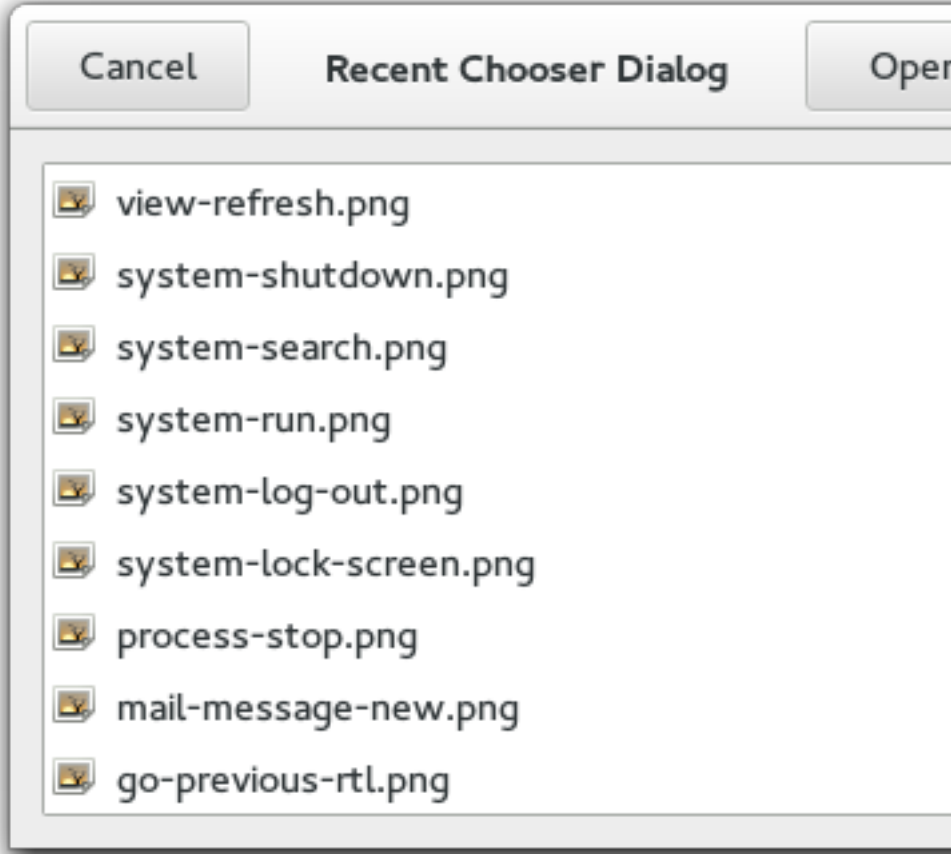
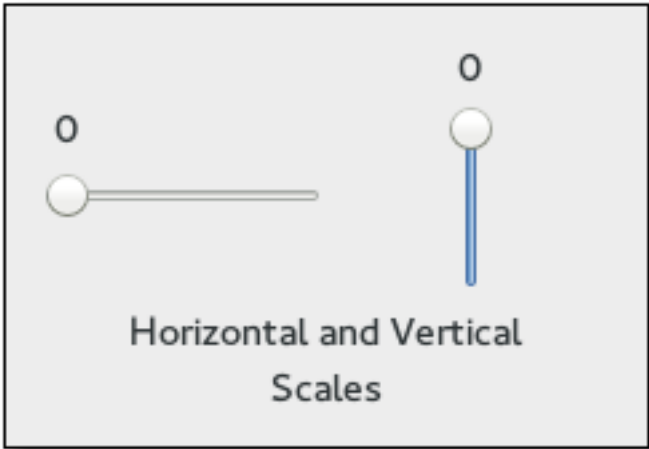
cont

Table 1 – continued from previous page

<code>Gtk.ProgressBar</code>	 A screenshot of a GTK+ 3 progress bar widget. It consists of a horizontal bar with a blue fill and a grey track. Below the bar, the text "Progress Bar" is displayed.
<code>Gtk.RadioButton</code>	 A screenshot of three radio buttons arranged vertically. The first button is selected and labeled "Radio Button One". The second button is labeled "Radio Button Two" and the third is labeled "Radio Button Three".

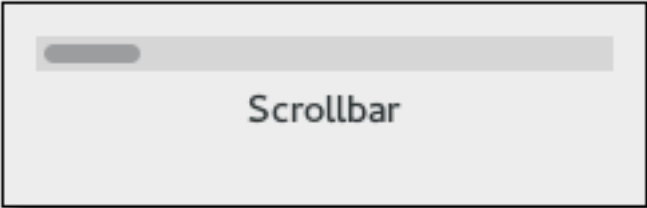
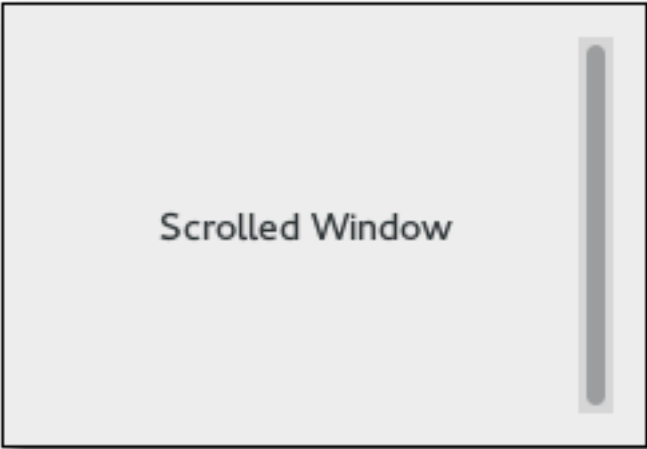
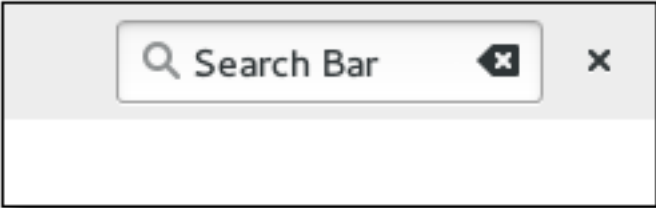
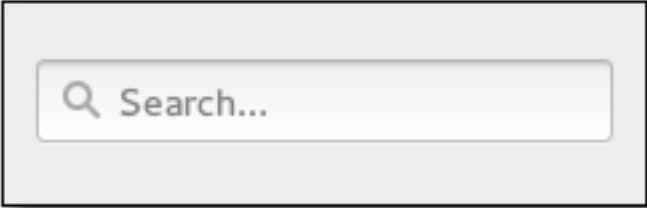
cont

Table 1 – continued from previous page

	 A screenshot of the 'Recent Chooser Dialog' window. It has a title bar with 'Cancel', 'Recent Chooser Dialog', and 'Open' buttons. The main area lists several files with icons: view-refresh.png, system-shutdown.png, system-search.png, system-run.png, system-log-out.png, system-lock-screen.png, process-stop.png, mail-message-new.png, and go-previous-rtl.png.
Gtk.RecentChooserDialog	
	 A screenshot showing two scale widgets. The left one is a horizontal scale with a slider at the left end, labeled '0' at the right. The right one is a vertical scale with a slider at the top, labeled '0' at the bottom. Below them is the text 'Horizontal and Vertical Scales'.
Gtk.Scale	

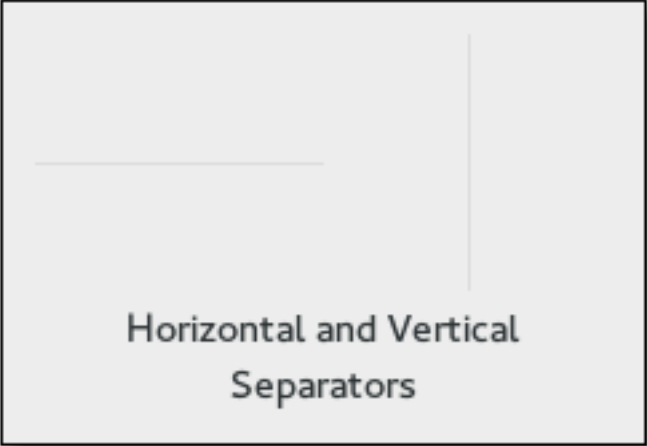
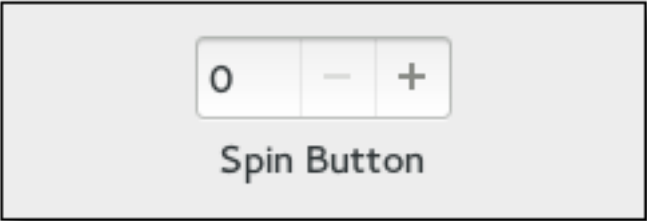
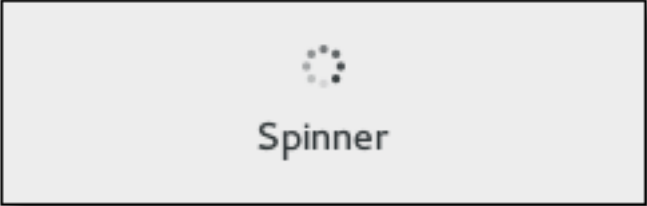
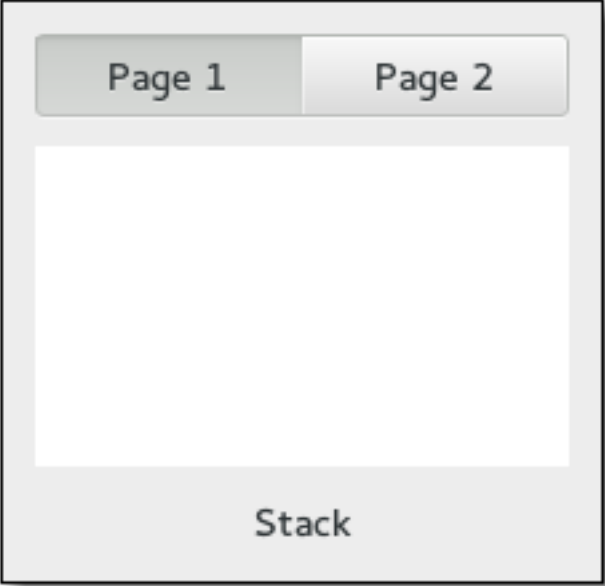
cont

Table 1 – continued from previous page

<code>Gtk.Scrollbar</code>	
<code>Gtk.ScrolledWindow</code>	
<code>Gtk.SearchBar</code>	
<code>Gtk.SearchEntry</code>	

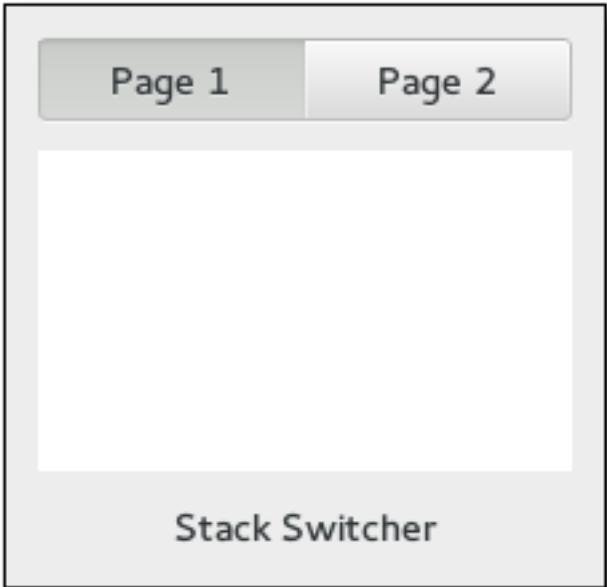
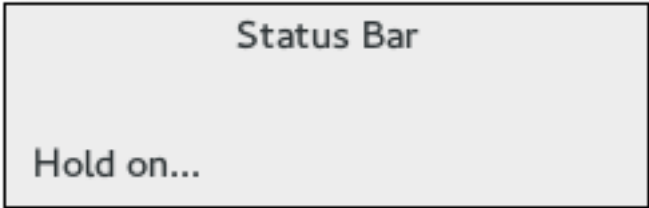
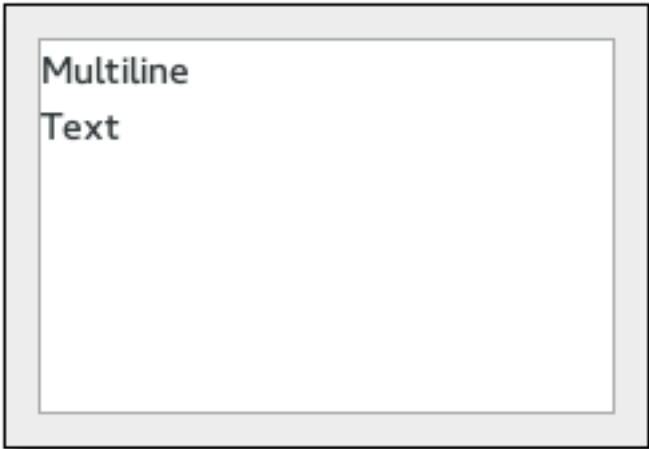
cont

Table 1 – continued from previous page

<code>Gtk.Separator</code>	 Horizontal and Vertical Separators
<code>Gtk.SpinButton</code>	 Spin Button
<code>Gtk.Spinner</code>	 Spinner
<code>Gtk.Stack</code>	 Stack

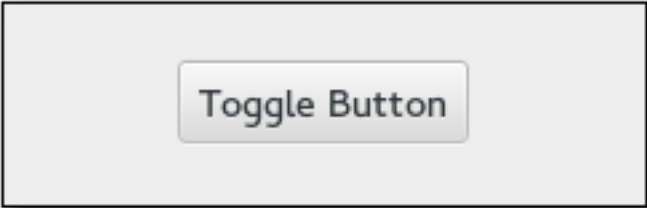
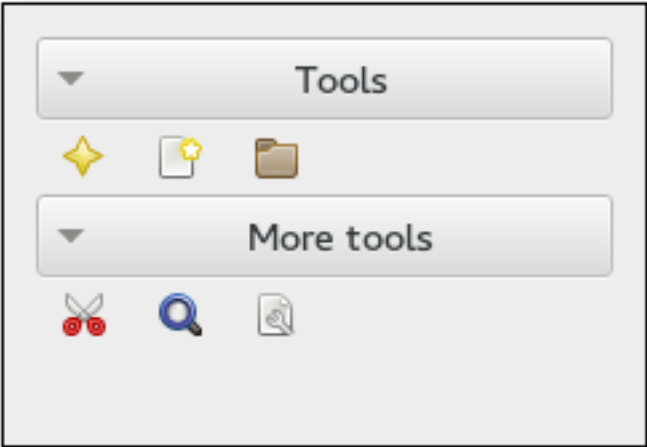
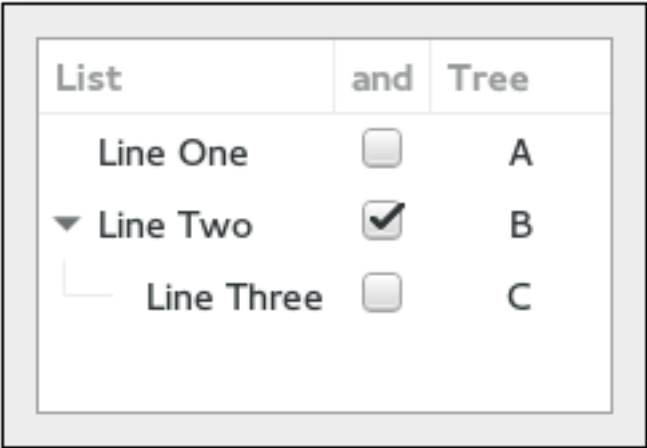
cont

Table 1 – continued from previous page

<code>Gtk.StackSwitcher</code>	
<code>Gtk.Statusbar</code>	
<code>Gtk.Switch</code>	
<code>Gtk.TextView</code>	

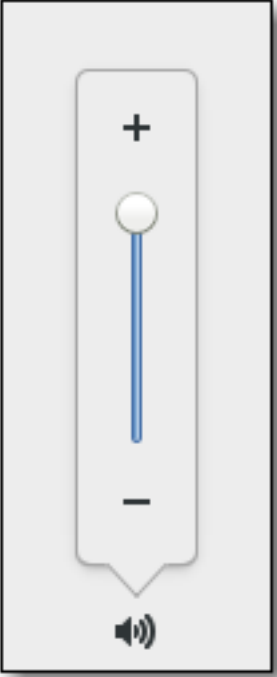
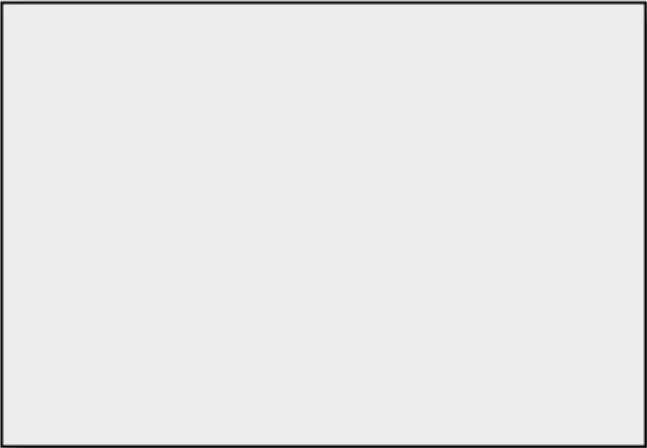
cont

Table 1 – continued from previous page

<code>Gtk.ToggleButton</code>	
<code>Gtk.ToolPalette</code>	
<code>Gtk.Toolbar</code>	
<code>Gtk.TreeView</code>	

cont

Table 1 – continued from previous page

<code>Gtk.VolumeButton</code>	
<code>Gtk.Window</code>	

LAYOUT CONTAINERS

While many GUI toolkits require you to precisely place widgets in a window, using absolute positioning, GTK+ uses a different approach. Rather than specifying the position and size of each widget in the window, you can arrange your widgets in rows, columns, and/or tables. The size of your window can be determined automatically, based on the sizes of the widgets it contains. And the sizes of the widgets are, in turn, determined by the amount of text they contain, or the minimum and maximum sizes that you specify, and/or how you have requested that the available space should be shared between sets of widgets. You can perfect your layout by specifying padding distance and centering values for each of your widgets. GTK+ then uses all this information to resize and reposition everything sensibly and smoothly when the user manipulates the window.

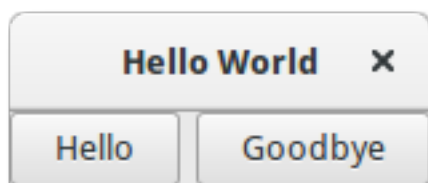
GTK+ arranges widgets hierarchically, using *containers*. They are invisible to the end user and are inserted into a window, or placed within each other to layout components. There are two flavours of containers: single-child containers, which are all descendants of `Gtk.Bin`, and multiple-child containers, which are descendants of `Gtk.Container`. The most commonly used are vertical or horizontal boxes (`Gtk.Box`) and grids (`Gtk.Grid`).

6.1 Boxes

Boxes are invisible containers into which we can pack our widgets. When packing widgets into a horizontal box, the objects are inserted horizontally from left to right or right to left depending on whether `Gtk.Box.pack_start()` or `Gtk.Box.pack_end()` is used. In a vertical box, widgets are packed from top to bottom or vice versa. You may use any combination of boxes inside or beside other boxes to create the desired effect.

6.1.1 Example

Let's take a look at a slightly modified version of the extended example with two buttons.



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class MyWindow(Gtk.Window):
```

(continues on next page)

(continued from previous page)

```

8  def __init__(self):
9      super().__init__(title="Hello World")
10
11     self.box = Gtk.Box(spacing=6)
12     self.add(self.box)
13
14     self.button1 = Gtk.Button(label="Hello")
15     self.button1.connect("clicked", self.on_button1_clicked)
16     self.box.pack_start(self.button1, True, True, 0)
17
18     self.button2 = Gtk.Button(label="Goodbye")
19     self.button2.connect("clicked", self.on_button2_clicked)
20     self.box.pack_start(self.button2, True, True, 0)
21
22     def on_button1_clicked(self, widget):
23         print("Hello")
24
25     def on_button2_clicked(self, widget):
26         print("Goodbye")
27
28
29 win = MyWindow()
30 win.connect("destroy", Gtk.main_quit)
31 win.show_all()
32 Gtk.main()

```

First, we create a horizontally orientated box container where 6 pixels are placed between children. This box becomes the child of the top-level window.

```

self.box = Gtk.Box(spacing=6)
self.add(self.box)

```

Subsequently, we add two different buttons to the box container.

```

self.button1 = Gtk.Button(label="Hello")
self.button1.connect("clicked", self.on_button1_clicked)
self.box.pack_start(self.button1, True, True, 0)

self.button2 = Gtk.Button(label="Goodbye")
self.button2.connect("clicked", self.on_button2_clicked)
self.box.pack_start(self.button2, True, True, 0)

```

While with `Gtk.Box.pack_start()` widgets are positioned from left to right, `Gtk.Box.pack_end()` positions them from right to left.

6.2 Grid

`Gtk.Grid` is a container which arranges its child widgets in rows and columns, but you do not need to specify the dimensions in the constructor. Children are added using `Gtk.Grid.attach()`. They can span multiple rows or columns. The `Gtk.Grid.attach()` method takes five parameters:

1. The child parameter is the `Gtk.Widget` to add.
2. `left` is the column number to attach the left side of child to.
3. `top` indicates the row number to attach the top side of child to.

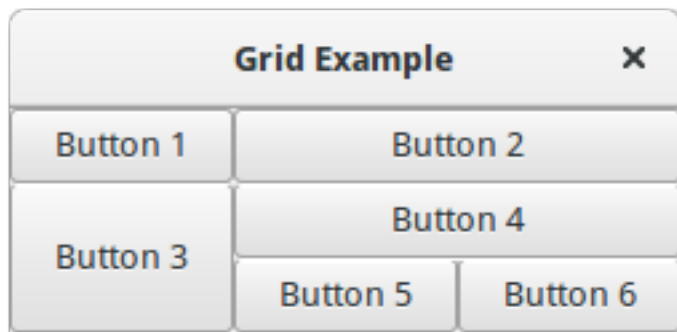
4. `width` and `height` indicate the number of columns that the child will span, and the number of rows that the child will span, respectively.

It is also possible to add a child next to an existing child, using `Gtk.Grid.attach_next_to()`, which also takes five parameters:

1. `child` is the `Gtk.Widget` to add, as above.
2. `sibling` is an existing child widget of `self` (a `Gtk.Grid` instance) or `None`. The child widget will be placed next to `sibling`, or if `sibling` is `None`, at the beginning or end of the grid.
3. `side` is a `Gtk.PositionType` indicating the side of `sibling` that `child` is positioned next to.
4. `width` and `height` indicate the number of columns and rows the child widget will span, respectively.

Finally, `Gtk.Grid` can be used like a `Gtk.Box` by just using `Gtk.Grid.add()`, which will place children next to each other in the direction determined by the “orientation” property (defaults to `Gtk.Orientation.HORIZONTAL`).

6.2.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class GridWindow(Gtk.Window):
8      def __init__(self):
9
10         super().__init__(title="Grid Example")
11
12         button1 = Gtk.Button(label="Button 1")
13         button2 = Gtk.Button(label="Button 2")
14         button3 = Gtk.Button(label="Button 3")
15         button4 = Gtk.Button(label="Button 4")
16         button5 = Gtk.Button(label="Button 5")
17         button6 = Gtk.Button(label="Button 6")
18
19         grid = Gtk.Grid()
20         grid.add(button1)
21         grid.attach(button2, 1, 0, 2, 1)
22         grid.attach_next_to(button3, button1, Gtk.PositionType.BOTTOM, 1, 2)
23         grid.attach_next_to(button4, button3, Gtk.PositionType.RIGHT, 2, 1)
24         grid.attach(button5, 1, 2, 1, 1)

```

(continues on next page)

(continued from previous page)

```

25         grid.attach_next_to(button6, button5, Gtk.PositionType.RIGHT, 1, 1)
26
27         self.add(grid)
28
29
30 win = GridWindow()
31 win.connect("destroy", Gtk.main_quit)
32 win.show_all()
33 Gtk.main()

```

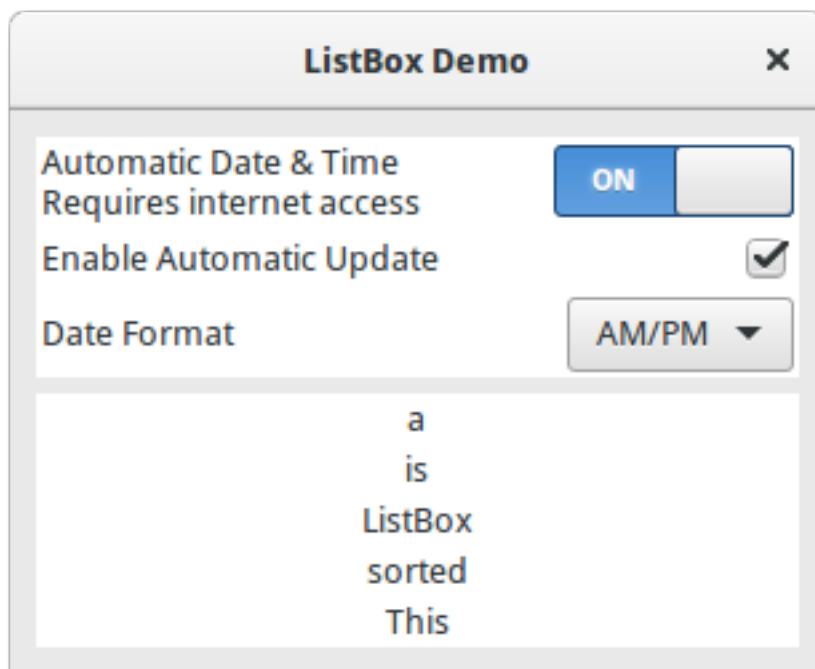
6.3 ListBox

A `Gtk.ListBox` is a vertical container that contains `Gtk.ListBoxRow` children. These rows can be dynamically sorted and filtered, and headers can be added dynamically depending on the row content. It also allows keyboard and mouse navigation and selection like a typical list.

Using `Gtk.ListBox` is often an alternative to `Gtk.TreeView`, especially when the list content has a more complicated layout than what is allowed by a `Gtk.CellRenderer`, or when the content is interactive (e.g. has a button in it).

Although a `Gtk.ListBox` must have only `Gtk.ListBoxRow` children, you can add any kind of widget to it via `Gtk.Container.add()` and a `Gtk.ListBoxRow` widget will automatically be inserted between the list and the widget.

6.3.1 Example



```

1 import gi
2

```

(continues on next page)

(continued from previous page)

```

3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class ListBoxRowWithData(Gtk.ListBoxRow):
8     def __init__(self, data):
9         super().__init__()
10        self.data = data
11        self.add(Gtk.Label(label=data))
12
13
14 class ListBoxWindow(Gtk.Window):
15     def __init__(self):
16         super().__init__(title="ListBox Demo")
17         self.set_border_width(10)
18
19         box_outer = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
20         self.add(box_outer)
21
22         listbox = Gtk.ListBox()
23         listbox.set_selection_mode(Gtk.SelectionMode.NONE)
24         box_outer.pack_start(listbox, True, True, 0)
25
26         row = Gtk.ListBoxRow()
27         hbox = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=50)
28         row.add(hbox)
29         vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL)
30         hbox.pack_start(vbox, True, True, 0)
31
32         label1 = Gtk.Label(label="Automatic Date & Time", xalign=0)
33         label2 = Gtk.Label(label="Requires internet access", xalign=0)
34         vbox.pack_start(label1, True, True, 0)
35         vbox.pack_start(label2, True, True, 0)
36
37         switch = Gtk.Switch()
38         switch.props.valign = Gtk.Align.CENTER
39         hbox.pack_start(switch, False, True, 0)
40
41         listbox.add(row)
42
43         row = Gtk.ListBoxRow()
44         hbox = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=50)
45         row.add(hbox)
46         label = Gtk.Label(label="Enable Automatic Update", xalign=0)
47         check = Gtk.CheckButton()
48         hbox.pack_start(label, True, True, 0)
49         hbox.pack_start(check, False, True, 0)
50
51         listbox.add(row)
52
53         row = Gtk.ListBoxRow()
54         hbox = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL, spacing=50)
55         row.add(hbox)
56         label = Gtk.Label(label="Date Format", xalign=0)
57         combo = Gtk.ComboBoxText()
58         combo.insert(0, "0", "24-hour")
59         combo.insert(1, "1", "AM/PM")

```

(continues on next page)

(continued from previous page)

```

60     hbox.pack_start(label, True, True, 0)
61     hbox.pack_start(combo, False, True, 0)
62
63     listbox.add(row)
64
65     listbox_2 = Gtk.ListBox()
66     items = "This is a sorted ListBox Fail".split()
67
68     for item in items:
69         listbox_2.add(ListBoxRowWithData(item))
70
71     def sort_func(row_1, row_2, data, notify_destroy):
72         return row_1.data.lower() > row_2.data.lower()
73
74     def filter_func(row, data, notify_destroy):
75         return False if row.data == "Fail" else True
76
77     listbox_2.set_sort_func(sort_func, None, False)
78     listbox_2.set_filter_func(filter_func, None, False)
79
80     def on_row_activated(listbox_widget, row):
81         print(row.data)
82
83     listbox_2.connect("row-activated", on_row_activated)
84
85     box_outer.pack_start(listbox_2, True, True, 0)
86     listbox_2.show_all()
87
88
89 win = ListBoxWindow()
90 win.connect("destroy", Gtk.main_quit)
91 win.show_all()
92 Gtk.main()

```

6.4 Stack and StackSwitcher

A `Gtk.Stack` is a container which only shows one of its children at a time. In contrast to `Gtk.Notebook`, `Gtk.Stack` does not provide a means for users to change the visible child. Instead, the `Gtk.StackSwitcher` widget can be used with `Gtk.Stack` to provide this functionality.

Transitions between pages can be animated as slides or fades. This can be controlled with `Gtk.Stack.set_transition_type()`. These animations respect the “gtk-enable-animations” setting.

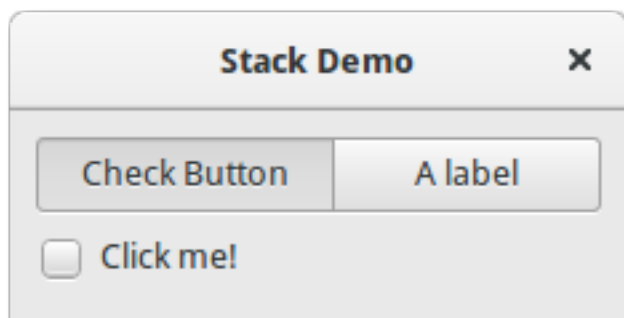
Transition speed can be adjusted with `Gtk.Stack.set_transition_duration()`

The `Gtk.StackSwitcher` widget acts as a controller for a `Gtk.Stack`; it shows a row of buttons to switch between the various pages of the associated stack widget.

All the content for the buttons comes from the child properties of the `Gtk.Stack`.

It is possible to associate multiple `Gtk.StackSwitcher` widgets with the same `Gtk.Stack` widget.

6.4.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class StackWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="Stack Demo")
10         self.set_border_width(10)
11
12         vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
13         self.add(vbox)
14
15         stack = Gtk.Stack()
16         stack.set_transition_type(Gtk.StackTransitionType.SLIDE_LEFT_RIGHT)
17         stack.set_transition_duration(1000)
18
19         checkbutton = Gtk.CheckButton(label="Click me!")
20         stack.add_titled(checkbutton, "check", "Check Button")
21
22         label = Gtk.Label()
23         label.set_markup("<big>A fancy label</big>")
24         stack.add_titled(label, "label", "A label")
25
26         stack_switcher = Gtk.StackSwitcher()
27         stack_switcher.set_stack(stack)
28         vbox.pack_start(stack_switcher, True, True, 0)
29         vbox.pack_start(stack, True, True, 0)
30
31
32 win = StackWindow()
33 win.connect("destroy", Gtk.main_quit)
34 win.show_all()
35 Gtk.main()

```

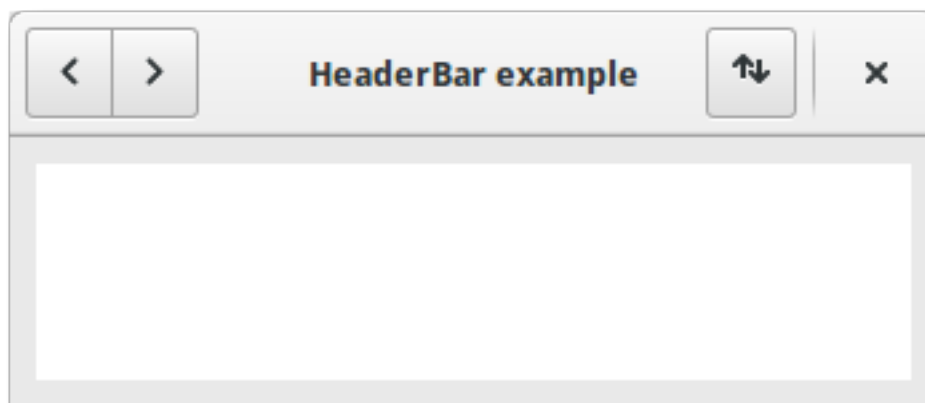
6.5 HeaderBar

A `Gtk.HeaderBar` is similar to a horizontal `Gtk.Box`, it allows to place children at the start or the end. In addition, it allows a title to be displayed. The title will be centered with respect to the width of the box, even if the children at either side take up different amounts of space.

Since GTK+ now supports Client Side Decoration, a `Gtk.HeaderBar` can be used in place of the title bar (which is rendered by the Window Manager).

A `Gtk.HeaderBar` is usually located across the top of a window and should contain commonly used controls which affect the content below. They also provide access to window controls, including the close window button and window menu.

6.5.1 Example



```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk, Gio
5
6
7 class HeaderBarWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="HeaderBar Demo")
10        self.set_border_width(10)
11        self.set_default_size(400, 200)
12
13        hb = Gtk.HeaderBar()
14        hb.set_show_close_button(True)
15        hb.props.title = "HeaderBar example"
16        self.set_titlebar(hb)
17
18        button = Gtk.Button()
19        icon = Gio.ThemedIcon(name="mail-send-receive-symbolic")
20        image = Gtk.Image.new_from_gicon(icon, Gtk.IconSize.BUTTON)
21        button.add(image)
22        hb.pack_end(button)
23
24        box = Gtk.Box(orientation=Gtk.Orientation.HORIZONTAL)
25        Gtk.StyleContext.add_class(box.get_style_context(), "linked")

```

(continues on next page)

(continued from previous page)

```
26
27     button = Gtk.Button()
28     button.add(
29         Gtk.Arrow(arrow_type=Gtk.ArrowType.LEFT, shadow_type=Gtk.ShadowType.NONE)
30     )
31     box.add(button)
32
33     button = Gtk.Button.new_from_icon_name("pan-end-symbolic", Gtk.IconSize.MENU)
34     box.add(button)
35
36     hb.pack_start(box)
37
38     self.add(Gtk.TextView())
39
40
41 win = HeaderBarWindow()
42 win.connect("destroy", Gtk.main_quit)
43 win.show_all()
44 Gtk.main()
```

6.6 FlowBox

Note: This example requires at least GTK+ 3.12.

A `Gtk.FlowBox` is a container that positions child widgets in sequence according to its orientation.

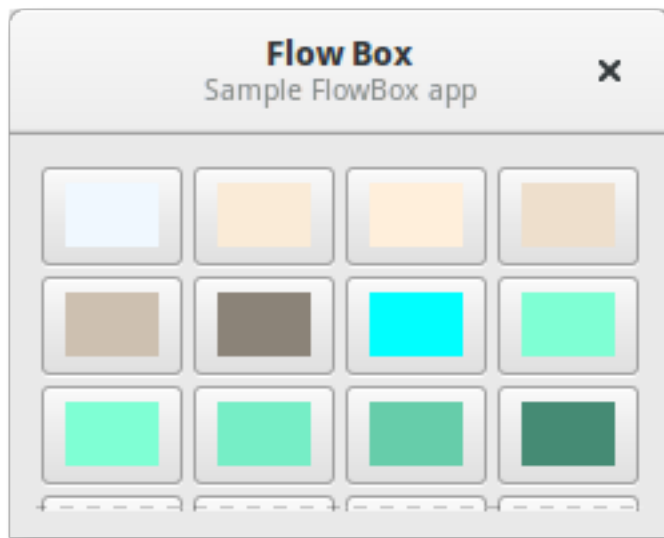
For instance, with the horizontal orientation, the widgets will be arranged from left to right, starting a new row under the previous row when necessary. Reducing the width in this case will require more rows, so a larger height will be requested.

Likewise, with the vertical orientation, the widgets will be arranged from top to bottom, starting a new column to the right when necessary. Reducing the height will require more columns, so a larger width will be requested.

The children of a `Gtk.FlowBox` can be dynamically sorted and filtered.

Although a `Gtk.FlowBox` must have only `Gtk.FlowBoxChild` children, you can add any kind of widget to it via `Gtk.Container.add()`, and a `Gtk.FlowBoxChild` widget will automatically be inserted between the box and the widget.

6.6.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk, Gdk
5
6
7  class FlowBoxWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="FlowBox Demo")
10         self.set_border_width(10)
11         self.set_default_size(300, 250)
12
13         header = Gtk.HeaderBar(title="Flow Box")
14         header.set_subtitle("Sample FlowBox app")
15         header.props.show_close_button = True
16
17         self.set_titlebar(header)
18
19         scrolled = Gtk.ScrolledWindow()
20         scrolled.set_policy(Gtk.PolicyType.NEVER, Gtk.PolicyType.AUTOMATIC)
21
22         flowbox = Gtk.FlowBox()
23         flowbox.set_valign(Gtk.Align.START)
24         flowbox.set_max_children_per_line(30)
25         flowbox.set_selection_mode(Gtk.SelectionMode.NONE)
26
27         self.create_flowbox(flowbox)
28
29         scrolled.add(flowbox)
30
31         self.add(scrolled)
32         self.show_all()
33
34     def on_draw(self, widget, cr, data):
35         context = widget.get_style_context()

```

(continues on next page)

(continued from previous page)

```

36     width = widget.get_allocated_width()
37     height = widget.get_allocated_height()
38     Gtk.render_background(context, cr, 0, 0, width, height)
39
40
41     r, g, b, a = data["color"]
42     cr.set_source_rgba(r, g, b, a)
43     cr.rectangle(0, 0, width, height)
44     cr.fill()
45
46     def color_swatch_new(self, str_color):
47         rgba = Gdk.RGBA()
48         rgba.parse(str_color)
49
50         button = Gtk.Button()
51
52         area = Gtk.DrawingArea()
53         area.set_size_request(24, 24)
54         area.connect("draw", self.on_draw, {"color": rgba})
55
56         button.add(area)
57
58         return button
59
60     def create_flowbox(self, flowbox):
61         colors = [
62             "AliceBlue",
63             "AntiqueWhite",
64             "AntiqueWhite1",
65             "AntiqueWhite2",
66             "AntiqueWhite3",
67             "AntiqueWhite4",
68             "aqua",
69             "aquamarine",
70             "aquamarine1",
71             "aquamarine2",
72             "aquamarine3",
73             "aquamarine4",
74             "azure",
75             "azure1",
76             "azure2",
77             "azure3",
78             "azure4",
79             "beige",
80             "bisque",
81             "bisque1",
82             "bisque2",
83             "bisque3",
84             "bisque4",
85             "black",
86             "BlanchedAlmond",
87             "blue",
88             "blue1",
89             "blue2",
90             "blue3",
91             "blue4",
92             "BlueViolet",

```

(continues on next page)

(continued from previous page)

```

93         "brown",
94         "brown1",
95         "brown2",
96         "brown3",
97         "brown4",
98         "burlywood",
99         "burlywood1",
100        "burlywood2",
101        "burlywood3",
102        "burlywood4",
103        "CadetBlue",
104        "CadetBlue1",
105        "CadetBlue2",
106        "CadetBlue3",
107        "CadetBlue4",
108        "chartreuse",
109        "chartreuse1",
110        "chartreuse2",
111        "chartreuse3",
112        "chartreuse4",
113        "chocolate",
114        "chocolate1",
115        "chocolate2",
116        "chocolate3",
117        "chocolate4",
118        "coral",
119        "coral1",
120        "coral2",
121        "coral3",
122        "coral4",
123    ]
124
125    for color in colors:
126        button = self.color_swatch_new(color)
127        flowbox.add(button)
128
129
130    win = FlowBoxWindow()
131    win.connect("destroy", Gtk.main_quit)
132    win.show_all()
133    Gtk.main()

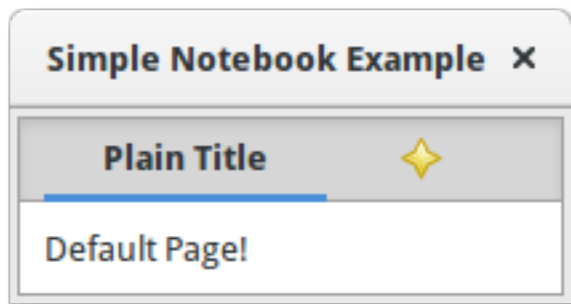
```

6.7 Notebook

The `Gtk.Notebook` widget is a `Gtk.Container` whose children are pages that can be switched between using tab labels along one edge.

There are many configuration options for `GtkNotebook`. Among other things, you can choose on which edge the tabs appear (see `Gtk.Notebook.set_tab_pos()`), whether, if there are too many tabs to fit the notebook should be made bigger or scrolling arrows added (see `Gtk.Notebook.set_scrollable()`), and whether there will be a popup menu allowing the users to switch pages (see `Gtk.Notebook.popup_enable()`, `Gtk.Notebook.popup_disable()`).

6.7.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class MyWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="Simple Notebook Example")
10         self.set_border_width(3)
11
12         self.notebook = Gtk.Notebook()
13         self.add(self.notebook)
14
15         self.page1 = Gtk.Box()
16         self.page1.set_border_width(10)
17         self.page1.add(Gtk.Label(label="Default Page!"))
18         self.notebook.append_page(self.page1, Gtk.Label(label="Plain Title"))
19
20         self.page2 = Gtk.Box()
21         self.page2.set_border_width(10)
22         self.page2.add(Gtk.Label(label="A page with an image for a Title."))
23         self.notebook.append_page(
24             self.page2, Gtk.Image.new_from_icon_name("help-about", Gtk.IconSize.MENU)
25         )
26
27
28  win = MyWindow()
29  win.connect("destroy", Gtk.main_quit)
30  win.show_all()
31  Gtk.main()

```


LABEL

Labels are the main method of placing non-editable text in windows, for instance to place a title next to a `Gtk.Entry` widget. You can specify the text in the constructor, or later with the `Gtk.Label.set_text()` or `Gtk.Label.set_markup()` methods.

The width of the label will be adjusted automatically. You can produce multi-line labels by putting line breaks (“\n”) in the label string.

Labels can be made selectable with `Gtk.Label.set_selectable()`. Selectable labels allow the user to copy the label contents to the clipboard. Only labels that contain useful-to-copy information — such as error messages — should be made selectable.

The label text can be justified using the `Gtk.Label.set_justify()` method. The widget is also capable of word-wrapping, which can be activated with `Gtk.Label.set_line_wrap()`.

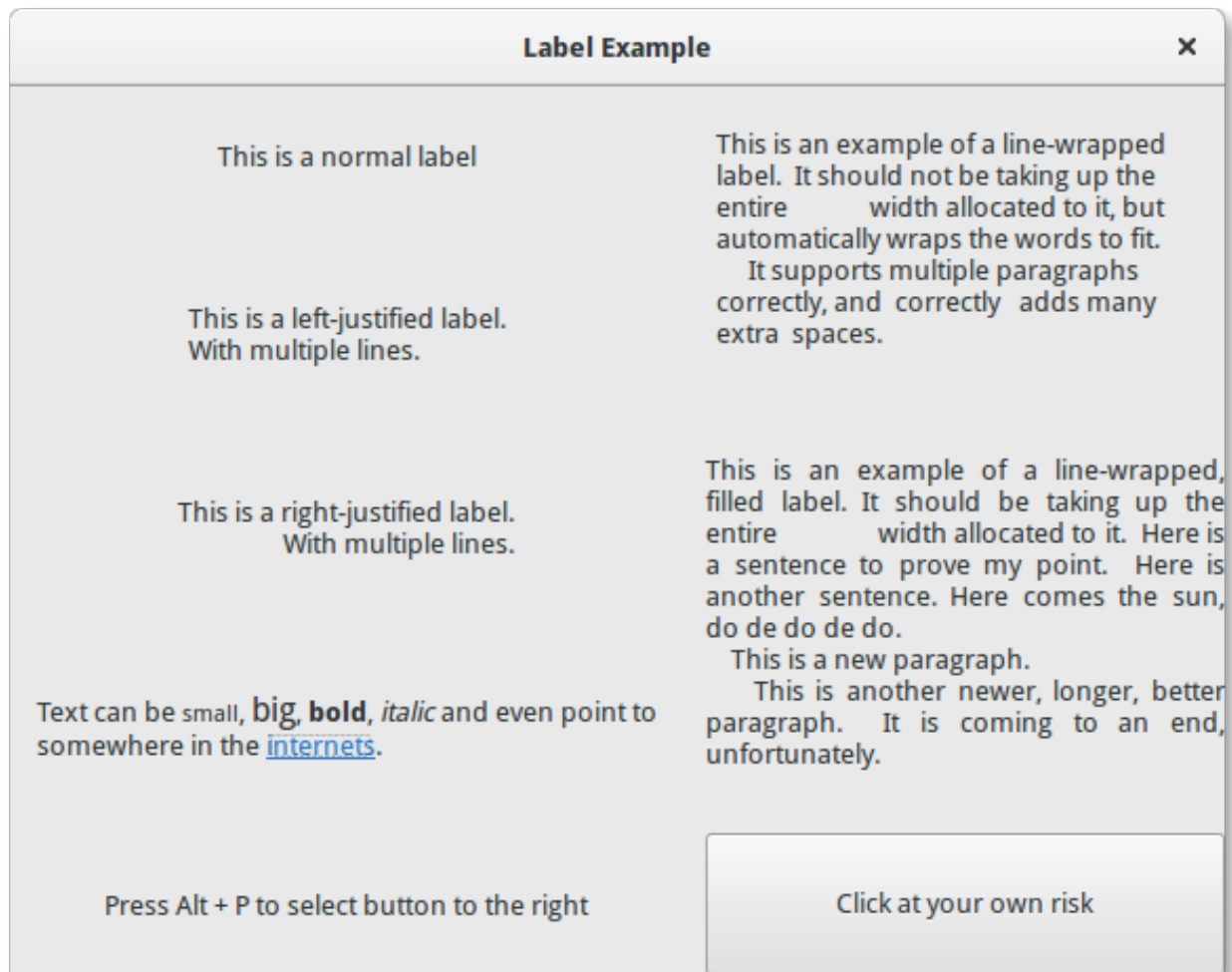
`Gtk.Label` support some simple formatting, for instance allowing you to make some text bold, colored, or larger. You can do this by providing a string to `Gtk.Label.set_markup()`, using the Pango Markup syntax¹. For instance, `<b>bold text</b>` and `<s>strikethrough text</s>`. In addition, `Gtk.Label` supports clickable hyperlinks. The markup for links is borrowed from HTML, using the `a` with `href` and `title` attributes. GTK+ renders links similar to the way they appear in web browsers, with colored, underlined text. The title attribute is displayed as a tooltip on the link.

```
label.set_markup("Go to <a href=\"https://www.gtk.org\" \"  
                \"title=\"Our website\">GTK+ website</a> for more")
```

Labels may contain *mnemonics*. Mnemonics are underlined characters in the label, used for keyboard navigation. Mnemonics are created by providing a string with an underscore before the mnemonic character, such as “_File”, to the functions `Gtk.Label.new_with_mnemonic()` or `Gtk.Label.set_text_with_mnemonic()`. Mnemonics automatically activate any activatable widget the label is inside, such as a `Gtk.Button`; if the label is not inside the mnemonic’s target widget, you have to tell the label about the target using `Gtk.Label.set_mnemonic_widget()`.

¹ Pango Markup Syntax, https://docs.gtk.org/Pango/pango_markup.html

7.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class LabelWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="Label Example")
10
11         hbox = Gtk.Box(spacing=10)
12         hbox.set_homogeneous(False)
13         vbox_left = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=10)
14         vbox_left.set_homogeneous(False)
15         vbox_right = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=10)
16         vbox_right.set_homogeneous(False)
17
18         hbox.pack_start(vbox_left, True, True, 0)
19         hbox.pack_start(vbox_right, True, True, 0)
20

```

(continues on next page)

(continued from previous page)

```

21     label = Gtk.Label(label="This is a normal label")
22     vbox_left.pack_start(label, True, True, 0)
23
24     label = Gtk.Label()
25     label.set_text("This is a left-justified label.\nWith multiple lines.")
26     label.set_justify(Gtk.Justification.LEFT)
27     vbox_left.pack_start(label, True, True, 0)
28
29     label = Gtk.Label(
30         label="This is a right-justified label.\nWith multiple lines."
31     )
32     label.set_justify(Gtk.Justification.RIGHT)
33     vbox_left.pack_start(label, True, True, 0)
34
35     label = Gtk.Label(
36         label="This is an example of a line-wrapped label. It "
37         "should not be taking up the entire "
38         "width allocated to it, but automatically "
39         "wraps the words to fit.\n"
40         "    It supports multiple paragraphs correctly, "
41         "and correctly adds "
42         "many          extra spaces. "
43     )
44     label.set_line_wrap(True)
45     label.set_max_width_chars(32)
46     vbox_right.pack_start(label, True, True, 0)
47
48     label = Gtk.Label(
49         label="This is an example of a line-wrapped, filled label. "
50         "It should be taking "
51         "up the entire          width allocated to it. "
52         "Here is a sentence to prove "
53         "my point. Here is another sentence. "
54         "Here comes the sun, do de do de do.\n"
55         "    This is a new paragraph.\n"
56         "    This is another newer, longer, better "
57         "paragraph. It is coming to an end, "
58         "unfortunately."
59     )
60     label.set_line_wrap(True)
61     label.set_justify(Gtk.Justification.FILL)
62     label.set_max_width_chars(32)
63     vbox_right.pack_start(label, True, True, 0)
64
65     label = Gtk.Label()
66     label.set_markup(
67         "Text can be <small>small</small>, <big>big</big>, "
68         "<b>bold</b>, <i>italic</i> and even point to "
69         "'somewhere in the <a href='\"https://www.gtk.org\" ' "
70         "'title='\"Click to find out more\">internets</a>.'"
71     )
72     label.set_line_wrap(True)
73     label.set_max_width_chars(48)
74     vbox_left.pack_start(label, True, True, 0)
75
76     label = Gtk.Label.new_with_mnemonic(
77         "_Press Alt + P to select button to the right"

```

(continues on next page)

(continued from previous page)

```
78         )
79         vbox_left.pack_start(label, True, True, 0)
80         label.set_selectable(True)
81
82         button = Gtk.Button(label="Click at your own risk")
83         label.set_mnemonic_widget(button)
84         vbox_right.pack_start(button, True, True, 0)
85
86         self.add(hbox)
87
88
89 window = LabelWindow()
90 window.connect("destroy", Gtk.main_quit)
91 window.show_all()
92 Gtk.main()
```

ENTRY

Entry widgets allow the user to enter text. You can change the contents with the `Gtk.Entry.set_text()` method, and read the current contents with the `Gtk.Entry.get_text()` method. You can also limit the number of characters the Entry can take by calling `Gtk.Entry.set_max_length()`.

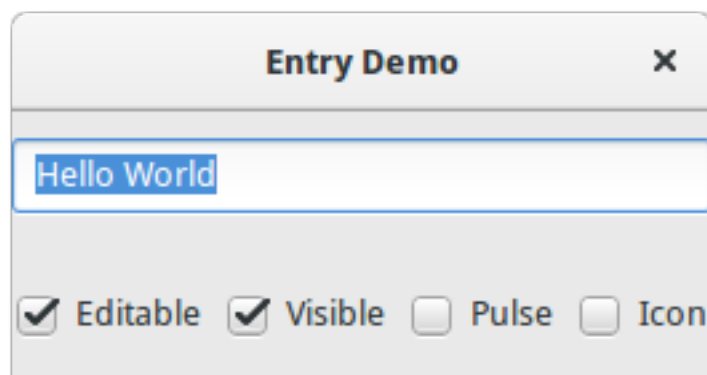
Occasionally you might want to make an Entry widget read-only. This can be done by passing `False` to the `Gtk.Entry.set_editable()` method.

Entry widgets can also be used to retrieve passwords from the user. It is common practice to hide the characters typed into the entry to prevent revealing the password to a third party. Calling `Gtk.Entry.set_visibility()` with `False` will cause the text to be hidden.

`Gtk.Entry` has the ability to display progress or activity information behind the text. This is similar to `Gtk.ProgressBar` widget and is commonly found in web browsers to indicate how much of a page download has been completed. To make an entry display such information, use `Gtk.Entry.set_progress_fraction()`, `Gtk.Entry.set_progress_pulse_step()`, or `Gtk.Entry.progress_pulse()`.

Additionally, an Entry can show icons at either side of the entry. These icons can be activatable by clicking, can be set up as drag source and can have tooltips. To add an icon, use `Gtk.Entry.set_icon_from_icon_name()` or one of the various other functions that set an icon from an icon name, a pixbuf, or icon theme. To set a tooltip on an icon, use `Gtk.Entry.set_icon_tooltip_text()` or the corresponding function for markup.

8.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk, GLib
```

(continues on next page)

(continued from previous page)

```

5
6
7 class EntryWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="Entry Demo")
10        self.set_size_request(200, 100)
11
12        self.timeout_id = None
13
14        vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
15        self.add(vbox)
16
17        self.entry = Gtk.Entry()
18        self.entry.set_text("Hello World")
19        vbox.pack_start(self.entry, True, True, 0)
20
21        hbox = Gtk.Box(spacing=6)
22        vbox.pack_start(hbox, True, True, 0)
23
24        self.check_editable = Gtk.CheckButton(label="Editable")
25        self.check_editable.connect("toggled", self.on_editable_toggled)
26        self.check_editable.set_active(True)
27        hbox.pack_start(self.check_editable, True, True, 0)
28
29        self.check_visible = Gtk.CheckButton(label="Visible")
30        self.check_visible.connect("toggled", self.on_visible_toggled)
31        self.check_visible.set_active(True)
32        hbox.pack_start(self.check_visible, True, True, 0)
33
34        self.pulse = Gtk.CheckButton(label="Pulse")
35        self.pulse.connect("toggled", self.on_pulse_toggled)
36        self.pulse.set_active(False)
37        hbox.pack_start(self.pulse, True, True, 0)
38
39        self.icon = Gtk.CheckButton(label="Icon")
40        self.icon.connect("toggled", self.on_icon_toggled)
41        self.icon.set_active(False)
42        hbox.pack_start(self.icon, True, True, 0)
43
44        def on_editable_toggled(self, button):
45            value = button.get_active()
46            self.entry.set_editable(value)
47
48        def on_visible_toggled(self, button):
49            value = button.get_active()
50            self.entry.set_visibility(value)
51
52        def on_pulse_toggled(self, button):
53            if button.get_active():
54                self.entry.set_progress_pulse_step(0.2)
55                # Call self.do_pulse every 100 ms
56                self.timeout_id = GLib.timeout_add(100, self.do_pulse, None)
57            else:
58                # Don't call self.do_pulse anymore
59                GLib.source_remove(self.timeout_id)
60                self.timeout_id = None
61                self.entry.set_progress_pulse_step(0)

```

(continues on next page)

(continued from previous page)

```
62
63     def do_pulse(self, user_data):
64         self.entry.progress_pulse()
65         return True
66
67     def on_icon_toggled(self, button):
68         if button.get_active():
69             icon_name = "system-search-symbolic"
70         else:
71             icon_name = None
72         self.entry.set_icon_from_icon_name(Gtk.EntryIconPosition.PRIMARY, icon_name)
73
74
75 win = EntryWindow()
76 win.connect("destroy", Gtk.main_quit)
77 win.show_all()
78 Gtk.main()
```


BUTTON WIDGETS

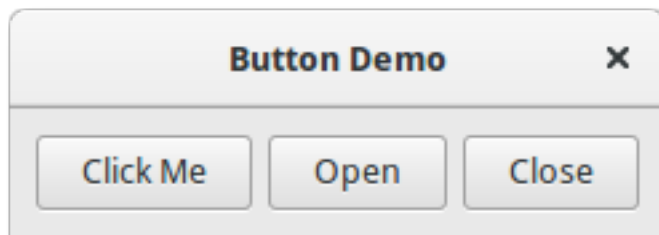
9.1 Button

The Button widget is another commonly used widget. It is generally used to attach a function that is called when the button is pressed.

The `Gtk.Button` widget can hold any valid child widget. That is it can hold most any other standard `Gtk.Widget`. The most commonly used child is the `Gtk.Label`.

Usually, you want to connect to the button's "clicked" signal which is emitted when the button has been pressed and released.

9.1.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class ButtonWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="Button Demo")
10        self.set_border_width(10)
11
12        hbox = Gtk.Box(spacing=6)
13        self.add(hbox)
14
15        button = Gtk.Button.new_with_label("Click Me")
16        button.connect("clicked", self.on_click_me_clicked)
17        hbox.pack_start(button, True, True, 0)
18
```

(continues on next page)

(continued from previous page)

```

19     button = Gtk.Button.new_with_mnemonic("_Open")
20     button.connect("clicked", self.on_open_clicked)
21     hbox.pack_start(button, True, True, 0)
22
23     button = Gtk.Button.new_with_mnemonic("_Close")
24     button.connect("clicked", self.on_close_clicked)
25     hbox.pack_start(button, True, True, 0)
26
27     def on_click_me_clicked(self, button):
28         print('Click me button was clicked')
29
30     def on_open_clicked(self, button):
31         print('Open button was clicked')
32
33     def on_close_clicked(self, button):
34         print("Closing application")
35         Gtk.main_quit()
36
37
38 win = ButtonWindow()
39 win.connect("destroy", Gtk.main_quit)
40 win.show_all()
41 Gtk.main()

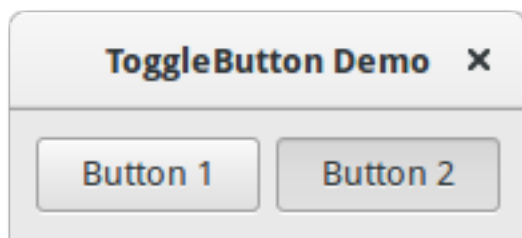
```

9.2 ToggleButton

A `Gtk.ToggleButton` is very similar to a normal `Gtk.Button`, but when clicked they remain activated, or pressed, until clicked again. When the state of the button is changed, the “toggled” signal is emitted.

To retrieve the state of the `Gtk.ToggleButton`, you can use the `Gtk.ToggleButton.get_active()` method. This returns `True` if the button is “down”. You can also set the toggle button’s state, with `Gtk.ToggleButton.set_active()`. Note that, if you do this, and the state actually changes, it causes the “toggled” signal to be emitted.

9.2.1 Example



```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6

```

(continues on next page)

(continued from previous page)

```

7 class ToggleButtonWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="ToggleButton Demo")
10        self.set_border_width(10)
11
12        hbox = Gtk.Box(spacing=6)
13        self.add(hbox)
14
15        button = Gtk.ToggleButton(label="Button 1")
16        button.connect("toggled", self.on_button_toggled, "1")
17        hbox.pack_start(button, True, True, 0)
18
19        button = Gtk.ToggleButton(label="B_utton 2", use_underline=True)
20        button.set_active(True)
21        button.connect("toggled", self.on_button_toggled, "2")
22        hbox.pack_start(button, True, True, 0)
23
24    def on_button_toggled(self, button, name):
25        if button.get_active():
26            state = "on"
27        else:
28            state = "off"
29        print("Button", name, "was turned", state)
30
31
32 win = ToggleButtonWindow()
33 win.connect("destroy", Gtk.main_quit)
34 win.show_all()
35 Gtk.main()

```

9.3 CheckButton

`Gtk.CheckButton` inherits from `Gtk.ToggleButton`. The only real difference between the two is `Gtk.CheckButton`'s appearance. A `Gtk.CheckButton` places a discrete `Gtk.ToggleButton` next to a widget, (usually a `Gtk.Label`). The “toggled” signal, `Gtk.ToggleButton.set_active()` and `Gtk.ToggleButton.get_active()` are inherited.

9.4 RadioButton

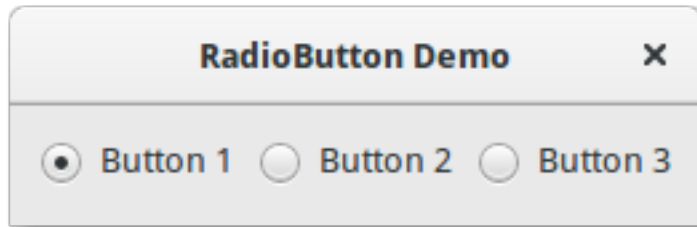
Like checkboxes, radio buttons also inherit from `Gtk.ToggleButton`, but these work in groups, and only one `Gtk.RadioButton` in a group can be selected at any one time. Therefore, a `Gtk.RadioButton` is one way of giving the user a choice from many options.

Radio buttons can be created with one of the static methods `Gtk.RadioButton.new_from_widget()`, `Gtk.RadioButton.new_with_label_from_widget()` or `Gtk.RadioButton.new_with_mnemonic_from_widget()`. The first radio button in a group will be created passing `None` as the *group* argument. In subsequent calls, the group you wish to add this button to should be passed as an argument.

When first run, the first radio button in the group will be active. This can be changed by calling `Gtk.ToggleButton.set_active()` with `True` as first argument.

Changing a `Gtk.RadioButton`'s widget group after its creation can be achieved by calling `Gtk.RadioButton.join_group()`.

9.4.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class RadioButtonWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="RadioButton Demo")
10         self.set_border_width(10)
11
12         hbox = Gtk.Box(spacing=6)
13         self.add(hbox)
14
15         button1 = Gtk.RadioButton.new_with_label_from_widget(None, "Button 1")
16         button1.connect("toggled", self.on_button_toggled, "1")
17         hbox.pack_start(button1, False, False, 0)
18
19         button2 = Gtk.RadioButton.new_from_widget(button1)
20         button2.set_label("Button 2")
21         button2.connect("toggled", self.on_button_toggled, "2")
22         hbox.pack_start(button2, False, False, 0)
23
24         button3 = Gtk.RadioButton.new_with_mnemonic_from_widget(button1, "B_utton 3")
25         button3.connect("toggled", self.on_button_toggled, "3")
26         hbox.pack_start(button3, False, False, 0)
27
28         def on_button_toggled(self, button, name):
29             if button.get_active():
30                 state = "on"
31             else:
32                 state = "off"
33             print("Button", name, "was turned", state)
34
35
36 win = RadioButtonWindow()
37 win.connect("destroy", Gtk.main_quit)
38 win.show_all()
39 Gtk.main()

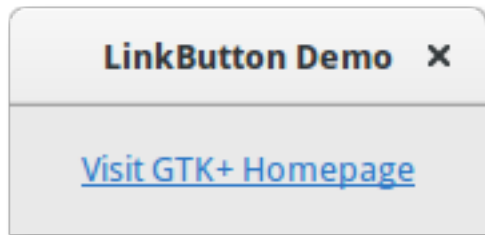
```

9.5 LinkButton

A `Gtk.LinkButton` is a `Gtk.Button` with a hyperlink, similar to the one used by web browsers, which triggers an action when clicked. It is useful to show quick links to resources.

The URI bound to a `Gtk.LinkButton` can be set specifically using `Gtk.LinkButton.set_uri()`, and retrieved using `Gtk.LinkButton.get_uri()`.

9.5.1 Example



```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class LinkButtonWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="LinkButton Demo")
10        self.set_border_width(10)
11
12        button = Gtk.LinkButton.new_with_label(
13            uri="https://www.gtk.org",
14            label="Visit GTK+ Homepage"
15        )
16        self.add(button)
17
18
19 win = LinkButtonWindow()
20 win.connect("destroy", Gtk.main_quit)
21 win.show_all()
22 Gtk.main()

```

9.6 SpinButton

A `Gtk.SpinButton` is an ideal way to allow the user to set the value of some attribute. Rather than having to directly type a number into a `Gtk.Entry`, `Gtk.SpinButton` allows the user to click on one of two arrows to increment or decrement the displayed value. A value can still be typed in, with the bonus that it can be checked to ensure it is in a given range. The main properties of a `Gtk.SpinButton` are set through `Gtk.Adjustment`.

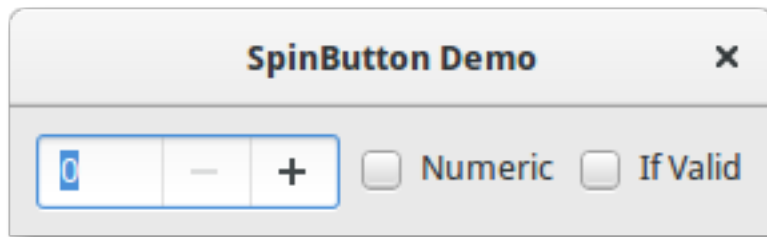
To change the value that `Gtk.SpinButton` is showing, use `Gtk.SpinButton.set_value()`. The value entered can either be an integer or float, depending on your requirements, use `Gtk.SpinButton.get_value_as_int()` or `Gtk.SpinButton.get_value()`, respectively.

When you allow the displaying of float values in the spin button, you may wish to adjust the number of decimal spaces displayed by calling `Gtk.SpinButton.set_digits()`.

By default, `Gtk.SpinButton` accepts textual data. If you wish to limit this to numerical values only, call `Gtk.SpinButton.set_numeric()` with `True` as argument.

We can also adjust the update policy of `Gtk.SpinButton`. There are two options here; by default the spin button updates the value even if the data entered is invalid. Alternatively, we can set the policy to only update when the value entered is valid by calling `Gtk.SpinButton.set_update_policy()`.

9.6.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class SpinButtonWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="SpinButton Demo")
10         self.set_border_width(10)
11
12         hbox = Gtk.Box(spacing=6)
13         self.add(hbox)
14
15         adjustment = Gtk.Adjustment(upper=100, step_increment=1, page_increment=10)
16         self.spinbutton = Gtk.SpinButton()
17         self.spinbutton.set_adjustment(adjustment)
18         self.spinbutton.connect("value-changed", self.on_value_changed)
19         hbox.pack_start(self.spinbutton, False, False, 0)
20
21         check_numeric = Gtk.CheckButton(label="Numeric")
22         check_numeric.connect("toggled", self.on_numeric_toggled)
23         hbox.pack_start(check_numeric, False, False, 0)
24
25         check_ifvalid = Gtk.CheckButton(label="If Valid")
26         check_ifvalid.connect("toggled", self.on_ifvalid_toggled)
27         hbox.pack_start(check_ifvalid, False, False, 0)
28
29         def on_value_changed(self, scroll):
30             print(self.spinbutton.get_value_as_int())
31
32         def on_numeric_toggled(self, button):
33             self.spinbutton.set_numeric(button.get_active())
34
35         def on_ifvalid_toggled(self, button):

```

(continues on next page)

(continued from previous page)

```

36         if button.get_active():
37             policy = Gtk.SpinButtonUpdatePolicy.IF_VALID
38         else:
39             policy = Gtk.SpinButtonUpdatePolicy.ALWAYS
40         self.spinbutton.set_update_policy(policy)
41
42
43 win = SpinButtonWindow()
44 win.connect("destroy", Gtk.main_quit)
45 win.show_all()
46 Gtk.main()

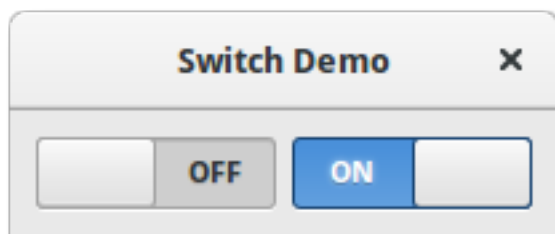
```

9.7 Switch

A `Gtk.Switch` is a widget that has two states: on or off. The user can control which state should be active by clicking the empty area, or by dragging the handle.

You shouldn't use the "activate" signal on the `Gtk.Switch` which is an action signal and emitting it causes the switch to animate. Applications should never connect to this signal, but use the "notify::active" signal, see the example here below.

9.7.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class SwitcherWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="Switch Demo")
10         self.set_border_width(10)
11
12         hbox = Gtk.Box(spacing=6)
13         self.add(hbox)
14
15         switch = Gtk.Switch()
16         switch.connect("notify::active", self.on_switch_activated)
17         switch.set_active(False)
18         hbox.pack_start(switch, True, True, 0)
19

```

(continues on next page)

(continued from previous page)

```
20     switch = Gtk.Switch()
21     switch.connect("notify::active", self.on_switch_activated)
22     switch.set_active(True)
23     hbox.pack_start(switch, True, True, 0)
24
25     def on_switch_activated(self, switch, gparam):
26         if switch.get_active():
27             state = "on"
28         else:
29             state = "off"
30         print("Switch was turned", state)
31
32
33 win = SwitcherWindow()
34 win.connect("destroy", Gtk.main_quit)
35 win.show_all()
36 Gtk.main()
```

EXPANDER

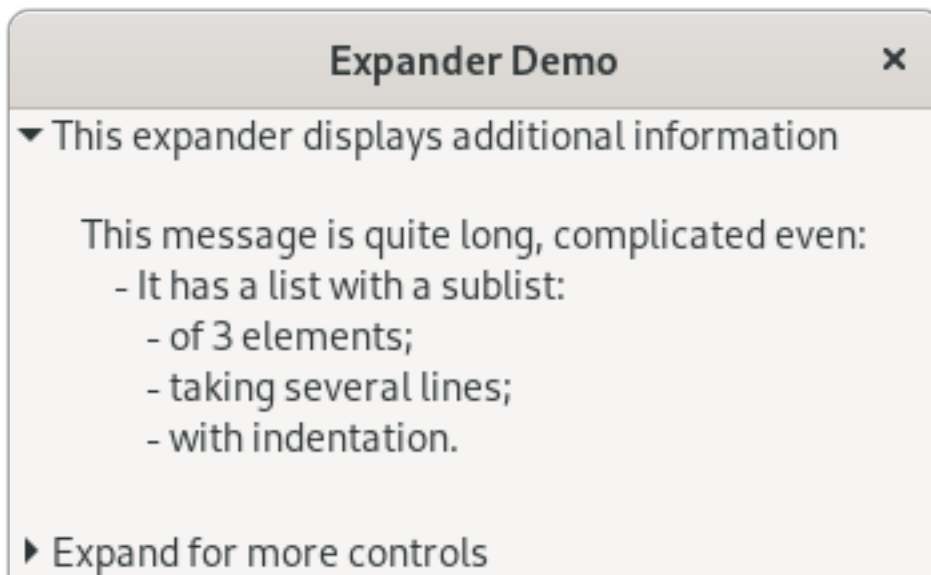
Expanders allow to dynamically hide or show information within a window or dialog. An expander can take a single widget that will be displayed when expanded.

Expanders remain expanded until clicked again. When the state of an expander is changed, the “activate” signal is emitted.

An expander can be programmatically expanded or collapsed by passing *True* or *False* to `Gtk.Expander.set_expanded()`. Note that doing so causes the “activate” signal to be emitted.

More than one widget, such as a `Gtk.Label` and `Gtk.Button`, can be added by appending them to a `Gtk.Box`.

10.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
```

(continues on next page)

(continued from previous page)

```
7 class ExpanderExample(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="Expander Demo")
10
11         self.set_size_request(350, 100)
12
13         vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
14         self.add(vbox)
15
16         text_expander = Gtk.Expander(
17             label="This expander displays additional information"
18         )
19         text_expander.set_expanded(True)
20         vbox.add(text_expander)
21
22         msg = """
23 This message is quite long, complicated even:
24 - It has a list with a sublist:
25     - of 3 elements;
26     - taking several lines;
27     - with indentation.
28 """
29         details = Gtk.Label(label=msg)
30         text_expander.add(details)
31
32         widget_expander = Gtk.Expander(label="Expand for more controls")
33         vbox.add(widget_expander)
34
35         expander_hbox = Gtk.HBox()
36         widget_expander.add(expander_hbox)
37
38         expander_hbox.add(Gtk.Label(label="Text message"))
39         expander_hbox.add(Gtk.Button(label="Click me"))
40
41         self.show_all()
42
43
44 win = ExpanderExample()
45 win.connect("destroy", Gtk.main_quit)
46 win.show_all()
47 Gtk.main()
```


PROGRESSBAR

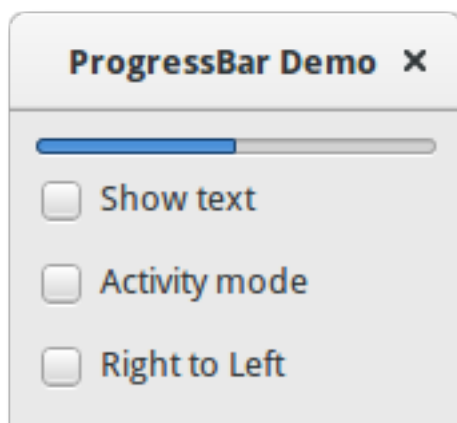
The `Gtk.ProgressBar` is typically used to display the progress of a long running operation. It provides a visual clue that processing is underway. The `Gtk.ProgressBar` can be used in two different modes: *percentage mode* and *activity mode*.

When an application can determine how much work needs to take place (e.g. read a fixed number of bytes from a file) and can monitor its progress, it can use the `Gtk.ProgressBar` in *percentage mode* and the user sees a growing bar indicating the percentage of the work that has been completed. In this mode, the application is required to call `Gtk.ProgressBar.set_fraction()` periodically to update the progress bar, passing a float between 0 and 1 to provide the new percentage value.

When an application has no accurate way of knowing the amount of work to do, it can use *activity mode*, which shows activity by a block moving back and forth within the progress area. In this mode, the application is required to call `Gtk.ProgressBar.pulse()` periodically to update the progress bar. You can also choose the step size, with the `Gtk.ProgressBar.set_pulse_step()` method.

By default, `Gtk.ProgressBar` is horizontal and left-to-right, but you can change it to a vertical progress bar by using the `Gtk.ProgressBar.set_orientation()` method. Changing the direction the progress bar grows can be done using `Gtk.ProgressBar.set_inverted()`. `Gtk.ProgressBar` can also contain text which can be set by calling `Gtk.ProgressBar.set_text()` and `Gtk.ProgressBar.set_show_text()`.

11.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
```

(continues on next page)

(continued from previous page)

```

4  from gi.repository import Gtk, GLib
5
6
7  class ProgressBarWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="ProgressBar Demo")
10         self.set_border_width(10)
11
12         vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
13         self.add(vbox)
14
15         self.progressbar = Gtk.ProgressBar()
16         vbox.pack_start(self.progressbar, True, True, 0)
17
18         button = Gtk.CheckButton(label="Show text")
19         button.connect("toggled", self.on_show_text_toggled)
20         vbox.pack_start(button, True, True, 0)
21
22         button = Gtk.CheckButton(label="Activity mode")
23         button.connect("toggled", self.on_activity_mode_toggled)
24         vbox.pack_start(button, True, True, 0)
25
26         button = Gtk.CheckButton(label="Right to Left")
27         button.connect("toggled", self.on_right_to_left_toggled)
28         vbox.pack_start(button, True, True, 0)
29
30         self.timeout_id = GLib.timeout_add(50, self.on_timeout, None)
31         self.activity_mode = False
32
33     def on_show_text_toggled(self, button):
34         show_text = button.get_active()
35         if show_text:
36             text = "some text"
37         else:
38             text = None
39         self.progressbar.set_text(text)
40         self.progressbar.set_show_text(show_text)
41
42     def on_activity_mode_toggled(self, button):
43         self.activity_mode = button.get_active()
44         if self.activity_mode:
45             self.progressbar.pulse()
46         else:
47             self.progressbar.set_fraction(0.0)
48
49     def on_right_to_left_toggled(self, button):
50         value = button.get_active()
51         self.progressbar.set_inverted(value)
52
53     def on_timeout(self, user_data):
54         """
55         Update value on the progress bar
56         """
57         if self.activity_mode:
58             self.progressbar.pulse()
59         else:
60             new_value = self.progressbar.get_fraction() + 0.01

```

(continues on next page)

(continued from previous page)

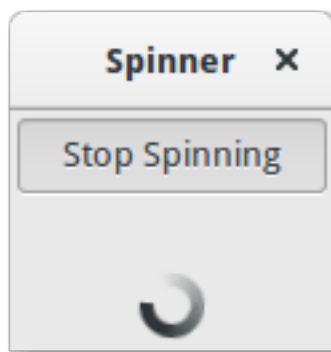
```
61         if new_value > 1:
62             new_value = 0
63
64         self.progressbar.set_fraction(new_value)
65
66         # As this is a timeout function, return True so that it
67         # continues to get called
68         return True
69
70
71 win = ProgressBarWindow()
72 win.connect("destroy", Gtk.main_quit)
73 win.show_all()
74 Gtk.main()
```


SPINNER

The `Gtk.Spinner` displays an icon-size spinning animation. It is often used as an alternative to a `GtkProgressBar` for displaying indefinite activity, instead of actual progress.

To start the animation, use `Gtk.Spinner.start()`, to stop it use `Gtk.Spinner.stop()`.

12.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class SpinnerAnimation(Gtk.Window):
8     def __init__(self):
9
10         super().__init__(title="Spinner")
11         self.set_border_width(3)
12         self.connect("destroy", Gtk.main_quit)
13
14         self.button = Gtk.ToggleButton(label="Start Spinning")
15         self.button.connect("toggled", self.on_button_toggled)
16         self.button.set_active(False)
17
18         self.spinner = Gtk.Spinner()
19
20         self.grid = Gtk.Grid()
21         self.grid.add(self.button)
```

(continues on next page)

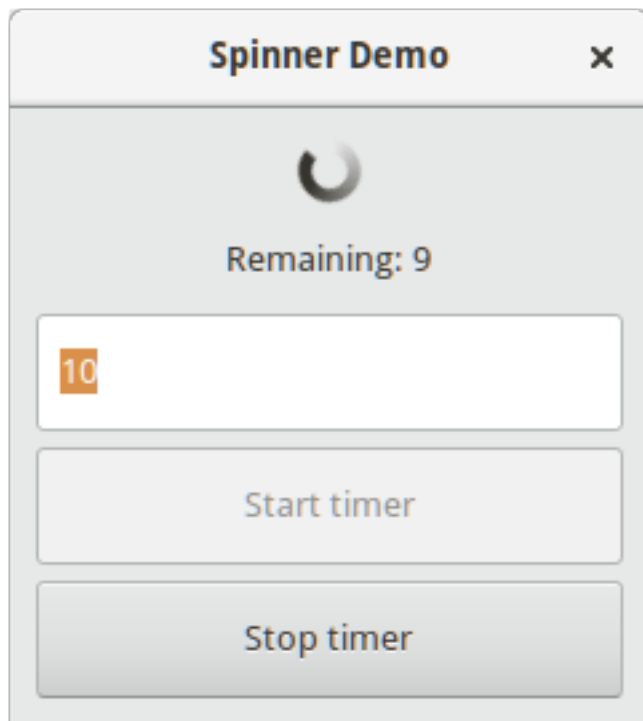
(continued from previous page)

```
22     self.grid.attach_next_to(
23         self.spinner, self.button, Gtk.PositionType.BOTTOM, 1, 2
24     )
25     self.grid.set_row_homogeneous(True)
26
27     self.add(self.grid)
28     self.show_all()
29
30     def on_button_toggled(self, button):
31
32         if button.get_active():
33             self.spinner.start()
34             self.button.set_label("Stop Spinning")
35
36         else:
37             self.spinner.stop()
38             self.button.set_label("Start Spinning")
39
40
41 myspinner = SpinnerAnimation()
42
43 Gtk.main()
```

12.2 Extended example

An extended example that uses a timeout function to start and stop the spinning animation. The `on_timeout()` function is called at regular intervals until it returns `False`, at which point the timeout is automatically destroyed and the function will not be called again.

12.2.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk, GLib
5
6
7  class SpinnerWindow(Gtk.Window):
8      def __init__(self, *args, **kwargs):
9          super().__init__(title="Spinner Demo")
10         self.set_border_width(10)
11
12         mainBox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
13         self.add(mainBox)
14
15         self.spinner = Gtk.Spinner()
16         mainBox.pack_start(self.spinner, True, True, 0)
17
18         self.label = Gtk.Label()
19         mainBox.pack_start(self.label, True, True, 0)
20
21         self.entry = Gtk.Entry()
22         self.entry.set_text("10")
23         mainBox.pack_start(self.entry, True, True, 0)
24
25         self.buttonStart = Gtk.Button(label="Start timer")
26         self.buttonStart.connect("clicked", self.on_buttonStart_clicked)
27         mainBox.pack_start(self.buttonStart, True, True, 0)
28
29         self.buttonStop = Gtk.Button(label="Stop timer")

```

(continues on next page)

(continued from previous page)

```

30     self.buttonStop.set_sensitive(False)
31     self.buttonStop.connect("clicked", self.on_buttonStop_clicked)
32     mainBox.pack_start(self.buttonStop, True, True, 0)
33
34     self.timeout_id = None
35     self.connect("destroy", self.on_SpinnerWindow_destroy)
36
37     def on_buttonStart_clicked(self, widget, *args):
38         """ Handles "clicked" event of buttonStart. """
39         self.start_timer()
40
41     def on_buttonStop_clicked(self, widget, *args):
42         """ Handles "clicked" event of buttonStop. """
43         self.stop_timer("Stopped from button")
44
45     def on_SpinnerWindow_destroy(self, widget, *args):
46         """ Handles destroy event of main window. """
47         # ensure the timeout function is stopped
48         if self.timeout_id:
49             GLib.source_remove(self.timeout_id)
50             self.timeout_id = None
51         Gtk.main_quit()
52
53     def on_timeout(self, *args, **kwargs):
54         """ A timeout function.
55
56         Return True to stop it.
57         This is not a precise timer since next timeout
58         is recalculated based on the current time. """
59         self.counter -= 1
60         if self.counter <= 0:
61             self.stop_timer("Reached time out")
62             return False
63         self.label.set_label("Remaining: " + str(int(self.counter / 4)))
64         return True
65
66     def start_timer(self):
67         """ Start the timer. """
68         self.buttonStart.set_sensitive(False)
69         self.buttonStop.set_sensitive(True)
70         # time out will check every 250 milliseconds (1/4 of a second)
71         self.counter = 4 * int(self.entry.get_text())
72         self.label.set_label("Remaining: " + str(int(self.counter / 4)))
73         self.spinner.start()
74         self.timeout_id = GLib.timeout_add(250, self.on_timeout, None)
75
76     def stop_timer(self, alabeltext):
77         """ Stop the timer. """
78         if self.timeout_id:
79             GLib.source_remove(self.timeout_id)
80             self.timeout_id = None
81         self.spinner.stop()
82         self.buttonStart.set_sensitive(True)
83         self.buttonStop.set_sensitive(False)
84         self.label.set_label(alabeltext)
85
86

```

(continues on next page)

(continued from previous page)

```
87 win = SpinnerWindow()  
88 win.show_all()  
89 Gtk.main()
```


TREE AND LIST WIDGETS

A `Gtk.TreeView` and its associated widgets are an extremely powerful way of displaying data. They are used in conjunction with a `Gtk.ListStore` or `Gtk.TreeStore` and provide a way of displaying and manipulating data in many ways, including:

- Automatic updates when data is added, removed or edited
- Drag and drop support
- Sorting data
- Embedding widgets such as check boxes, progress bars, etc.
- Reorderable and resizable columns
- Filtering data

With the power and flexibility of a `Gtk.TreeView` comes complexity. It is often difficult for beginner developers to be able to utilize it correctly due to the number of methods which are required.

13.1 The Model

Each `Gtk.TreeView` has an associated `Gtk.TreeModel`, which contains the data displayed by the `TreeView`. Each `Gtk.TreeModel` can be used by more than one `Gtk.TreeView`. For instance, this allows the same underlying data to be displayed and edited in 2 different ways at the same time. Or the 2 Views might display different columns from the same Model data, in the same way that 2 SQL queries (or “views”) might show different fields from the same database table.

Although you can theoretically implement your own Model, you will normally use either the `Gtk.ListStore` or `Gtk.TreeStore` model classes. `Gtk.ListStore` contains simple rows of data, and each row has no children, whereas `Gtk.TreeStore` contains rows of data, and each row may have child rows.

When constructing a model you have to specify the data types for each column the model holds.

```
store = Gtk.ListStore(str, str, float)
```

This creates a list store with three columns, two string columns, and a float column.

Adding data to the model is done using `Gtk.ListStore.append()` or `Gtk.TreeStore.append()`, depending upon which sort of model was created.

For a `Gtk.ListStore`:

```
treeiter = store.append(["The Art of Computer Programming",  
                        "Donald E. Knuth", 25.46])
```

For a `Gtk.TreeStore` you must specify an existing row to append the new row to, using a `Gtk.TreeIter`, or `None` for the top level of the tree:

```
treeiter = store.append(None, ["The Art of Computer Programming",  
                                "Donald E. Knuth", 25.46])
```

Both methods return a `Gtk.TreeIter` instance, which points to the location of the newly inserted row. You can retrieve a `Gtk.TreeIter` by calling `Gtk.TreeModel.get_iter()`.

Once data has been inserted, you can retrieve or modify data using the tree iter and column index.

```
print(store[treeiter][2]) # Prints value of third column  
store[treeiter][2] = 42.15
```

As with Python's built-in `list` object you can use `len()` to get the number of rows and use slices to retrieve or set values.

```
# Print number of rows  
print(len(store))  
# Print all but first column  
print(store[treeiter][1:])  
# Print last column  
print(store[treeiter][-1])  
# Set last two columns  
store[treeiter][1:] = ["Donald Ervin Knuth", 41.99]
```

Iterating over all rows of a tree model is very simple as well.

```
for row in store:  
    # Print values of all columns  
    print(row[:])
```

Keep in mind, that if you use `Gtk.TreeStore`, the above code will only iterate over the rows of the top level, but not the children of the nodes. To iterate over all rows, use `Gtk.TreeModel.foreach()`.

```
def print_row(store, treepath, treeiter):  
    print("\t" * (treepath.get_depth() - 1), store[treeiter][:], sep="")  
  
store.foreach(print_row)
```

Apart from accessing values stored in a `Gtk.TreeModel` with the list-like method mentioned above, it is also possible to either use `Gtk.TreeIter` or `Gtk.TreePath` instances. Both reference a particular row in a tree model. One can convert a path to an iterator by calling `Gtk.TreeModel.get_iter()`. As `Gtk.ListStore` contains only one level, i.e. nodes do not have any child nodes, a path is essentially the index of the row you want to access.

```
# Get path pointing to 6th row in list store  
path = Gtk.TreePath(5)  
treeiter = liststore.get_iter(path)  
# Get value at 2nd column  
value = liststore.get_value(treeiter, 1)
```

In the case of `Gtk.TreeStore`, a path is a list of indexes or a string. The string form is a list of numbers separated by a colon. Each number refers to the offset at that level. Thus, the path "0" refers to the root node and the path "2:4" refers to the fifth child of the third node.

```
# Get path pointing to 5th child of 3rd row in tree store
path = Gtk.TreePath([2, 4])
treeiter = treestore.get_iter(path)
# Get value at 2nd column
value = treestore.get_value(treeiter, 1)
```

Instances of `Gtk.TreePath` can be accessed like lists, i.e. `len(treepath)` returns the depth of the item `treepath` is pointing to, and `treepath[i]` returns the child's index on the *i*-th level.

13.2 The View

While there are several different models to choose from, there is only one view widget to deal with. It works with either the list or the tree store. Setting up a `Gtk.TreeView` is not a difficult matter. It needs a `Gtk.TreeModel` to know where to retrieve its data from, either by passing it to the `Gtk.TreeView` constructor, or by calling `Gtk.TreeView.set_model()`.

```
tree = Gtk.TreeView(model=store)
```

Once the `Gtk.TreeView` widget has a model, it will need to know how to display the model. It does this with columns and cell renderers. `headers_visible` controls whether it displays column headers.

Cell renderers are used to draw the data in the tree model in a specific way. There are a number of cell renderers that come with GTK+, for instance `Gtk.CellRendererText`, `Gtk.CellRendererPixbuf` and `Gtk.CellRendererToggle`. In addition, it is relatively easy to write a custom renderer yourself by subclassing a `Gtk.CellRenderer`, and adding properties with `GObject.Property()`.

A `Gtk.TreeViewColumn` is the object that `Gtk.TreeView` uses to organize the vertical columns in the tree view and hold one or more cell renderers. Each column may have a `title` that will be visible if the `Gtk.TreeView` is showing column headers. The model is mapped to the column by using keyword arguments with properties of the renderer as the identifiers and indexes of the model columns as the arguments.

```
renderer = Gtk.CellRendererPixbuf()
column = Gtk.TreeViewColumn(cell_renderer=renderer, icon_name=3)
tree.append_column(column)
```

Positional arguments can be used for the column title and renderer.

```
renderer = Gtk.CellRendererText()
column = Gtk.TreeViewColumn("Title", renderer, text=0, weight=1)
tree.append_column(column)
```

To render more than one model column in a view column, you need to create a `Gtk.TreeViewColumn` instance and use `Gtk.TreeViewColumn.pack_start()` to add the model columns to it.

```
column = Gtk.TreeViewColumn("Title and Author")

title = Gtk.CellRendererText()
author = Gtk.CellRendererText()

column.pack_start(title, True)
column.pack_start(author, True)

column.add_attribute(title, "text", 0)
column.add_attribute(author, "text", 1)
```

(continues on next page)

(continued from previous page)

```
tree.append_column(column)
```

13.3 The Selection

Most applications will need to not only deal with displaying data, but also receiving input events from users. To do this, simply get a reference to a selection object and connect to the “changed” signal.

```
select = tree.get_selection()
select.connect("changed", on_tree_selection_changed)
```

Then to retrieve data for the row selected:

```
def on_tree_selection_changed(selection):
    model, treeiter = selection.get_selected()
    if treeiter is not None:
        print("You selected", model[treeiter][0])
```

You can control what selections are allowed by calling `Gtk.TreeSelection.set_mode()`. `Gtk.TreeSelection.get_selected()` does not work if the selection mode is set to `Gtk.SelectionMode.MULTIPLE`, use `Gtk.TreeSelection.get_selected_rows()` instead.

13.4 Sorting

Sorting is an important feature for tree views and is supported by the standard tree models (`Gtk.TreeStore` and `Gtk.ListStore`), which implement the `Gtk.TreeSortable` interface.

13.4.1 Sorting by clicking on columns

A column of a `Gtk.TreeView` can easily be made sortable with a call to `Gtk.TreeViewColumn.set_sort_column_id()`. Afterwards the column can be sorted by clicking on its header.

First we need a simple `Gtk.TreeView` and a `Gtk.ListStore` as a model.

```
model = Gtk.ListStore(str)
model.append(["Benjamin"])
model.append(["Charles"])
model.append(["alfred"])
model.append(["Alfred"])
model.append(["David"])
model.append(["charles"])
model.append(["david"])
model.append(["benjamin"])

treeView = Gtk.TreeView(model=model)

cellRenderer = Gtk.CellRendererText()
column = Gtk.TreeViewColumn("Title", renderer, text=0)
```

The next step is to enable sorting. Note that the `column_id` (0 in the example) refers to the column of the model and **not** to the `TreeView`’s column.

```
column.set_sort_column_id(0)
```

13.4.2 Setting a custom sort function

It is also possible to set a custom comparison function in order to change the sorting behaviour. As an example we will create a comparison function that sorts case-sensitive. In the example above the sorted list looked like:

```
alfred
Alfred
benjamin
Benjamin
charles
Charles
david
David
```

The case-sensitive sorted list will look like:

```
Alfred
Benjamin
Charles
David
alfred
benjamin
charles
david
```

First of all a comparison function is needed. This function gets two rows and has to return a negative integer if the first one should come before the second one, zero if they are equal and a positive integer if the second one should come before the first one.

```
def compare(model, row1, row2, user_data):
    sort_column, _ = model.get_sort_column_id()
    value1 = model.get_value(row1, sort_column)
    value2 = model.get_value(row2, sort_column)
    if value1 < value2:
        return -1
    elif value1 == value2:
        return 0
    else:
        return 1
```

Then the sort function has to be set by `Gtk.TreeSortable.set_sort_func()`.

```
model.set_sort_func(0, compare, None)
```

13.5 Filtering

Unlike sorting, filtering is not handled by the two models we previously saw, but by the `Gtk.TreeModelFilter` class. This class, like `Gtk.TreeStore` and `Gtk.ListStore`, is a `Gtk.TreeModel`. It acts as a layer between the “real” model (a `Gtk.TreeStore` or a `Gtk.ListStore`), hiding some elements to the view. In practice, it supplies the `Gtk.TreeView` with a subset of the underlying model. Instances of `Gtk.TreeModelFilter` can be stacked one onto another, to use multiple filters on the same model (in the same way you’d use “AND” clauses in a SQL request). They can also be chained with `Gtk.TreeModelSort` instances.

You can create a new instance of a `Gtk.TreeModelFilter` and give it a model to filter, but the easiest way is to spawn it directly from the filtered model, using the `Gtk.TreeModel.filter_new()` method.

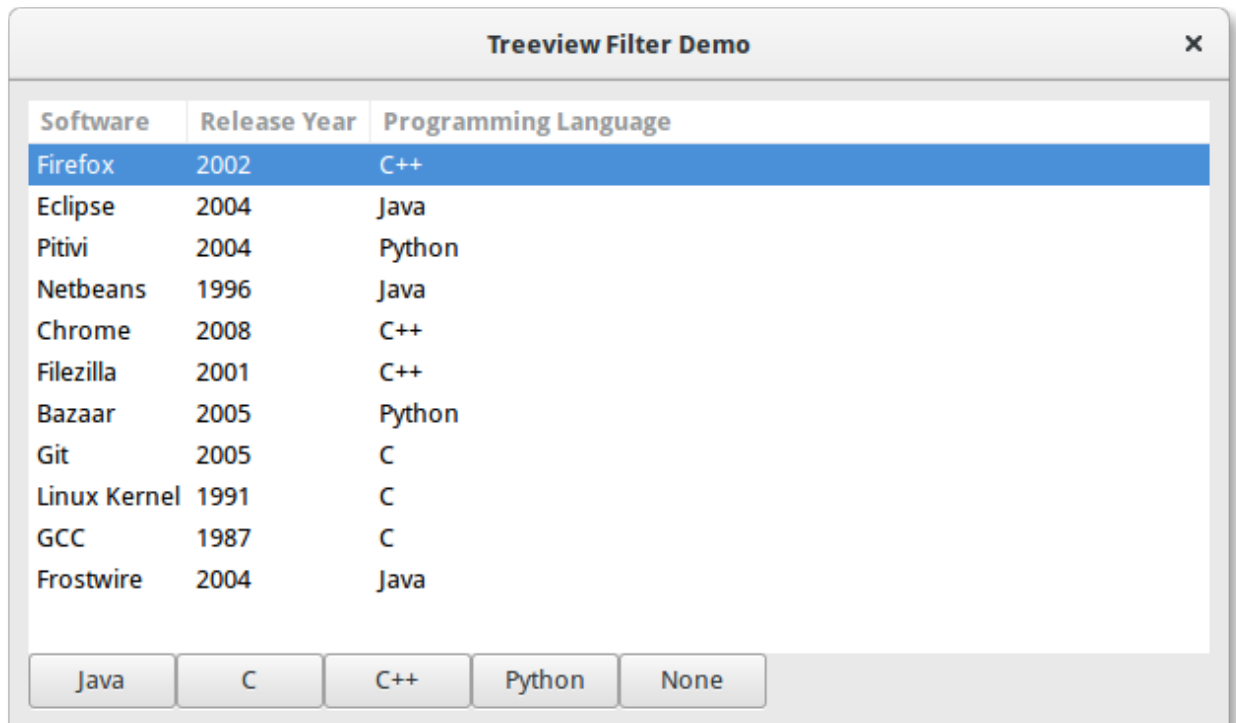
```
filter = model.filter_new()
```

In the same way the sorting function works, the `Gtk.TreeModelFilter` uses a “visibility” function, which, given a row from the underlying model, will return a boolean indicating if this row should be filtered out or not. It’s set by `Gtk.TreeModelFilter.set_visible_func()`:

```
filter.set_visible_func(filter_func, data=None)
```

The alternative to a “visibility” function is to use a boolean column in the model to specify which rows to filter. Choose which column with `Gtk.TreeModelFilter.set_visible_column()`.

Let’s look at a full example which uses the whole `Gtk.ListStore - Gtk.TreeModelFilter - Gtk.TreeModelFilter - Gtk.TreeView` stack.



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
```

(continues on next page)

(continued from previous page)

```

6  # list of tuples for each software, containing the software name, initial release,
   ↪and main programming languages used
7  software_list = [
8      ("Firefox", 2002, "C++"),
9      ("Eclipse", 2004, "Java"),
10     ("Pitivi", 2004, "Python"),
11     ("Netbeans", 1996, "Java"),
12     ("Chrome", 2008, "C++"),
13     ("Filezilla", 2001, "C++"),
14     ("Bazaar", 2005, "Python"),
15     ("Git", 2005, "C"),
16     ("Linux Kernel", 1991, "C"),
17     ("GCC", 1987, "C"),
18     ("Frostwire", 2004, "Java"),
19 ]
20
21
22 class TreeViewFilterWindow(Gtk.Window):
23     def __init__(self):
24         super().__init__(title="Treeview Filter Demo")
25         self.set_border_width(10)
26
27         # Setting up the self.grid in which the elements are to be positioned
28         self.grid = Gtk.Grid()
29         self.grid.set_column_homogeneous(True)
30         self.grid.set_row_homogeneous(True)
31         self.add(self.grid)
32
33         # Creating the ListStore model
34         self.software_liststore = Gtk.ListStore(str, int, str)
35         for software_ref in software_list:
36             self.software_liststore.append(list(software_ref))
37         self.current_filter_language = None
38
39         # Creating the filter, feeding it with the liststore model
40         self.language_filter = self.software_liststore.filter_new()
41         # setting the filter function, note that we're not using the
42         self.language_filter.set_visible_func(self.language_filter_func)
43
44         # creating the treeview, making it use the filter as a model, and adding the
   ↪columns
45         self.treeview = Gtk.TreeView(model=self.language_filter)
46         for i, column_title in enumerate(
47             ["Software", "Release Year", "Programming Language"]
48         ):
49             renderer = Gtk.CellRendererText()
50             column = Gtk.TreeViewColumn(column_title, renderer, text=i)
51             self.treeview.append_column(column)
52
53         # creating buttons to filter by programming language, and setting up their
   ↪events
54         self.buttons = list()
55         for prog_language in ["Java", "C", "C++", "Python", "None"]:
56             button = Gtk.Button(label=prog_language)
57             self.buttons.append(button)
58             button.connect("clicked", self.on_selection_button_clicked)
59

```

(continues on next page)

(continued from previous page)

```

60     # setting up the layout, putting the treeview in a scrollwindow, and the_
    ↪ buttons in a row
61     self.scrollable_treelist = Gtk.ScrolledWindow()
62     self.scrollable_treelist.set_vexpand(True)
63     self.grid.attach(self.scrollable_treelist, 0, 0, 8, 10)
64     self.grid.attach_next_to(
65         self.buttons[0], self.scrollable_treelist, Gtk.PositionType.BOTTOM, 1, 1
66     )
67     for i, button in enumerate(self.buttons[1:]):
68         self.grid.attach_next_to(
69             button, self.buttons[i], Gtk.PositionType.RIGHT, 1, 1
70         )
71     self.scrollable_treelist.add(self.treeview)
72
73     self.show_all()
74
75     def language_filter_func(self, model, iter, data):
76         """Tests if the language in the row is the one in the filter"""
77         if (
78             self.current_filter_language is None
79             or self.current_filter_language == "None"
80         ):
81             return True
82         else:
83             return model[iter][2] == self.current_filter_language
84
85     def on_selection_button_clicked(self, widget):
86         """Called on any of the button clicks"""
87         # we set the current language filter to the button's label
88         self.current_filter_language = widget.get_label()
89         print("%s language selected!" % self.current_filter_language)
90         # we update the filter, which updates in turn the view
91         self.language_filter.refilter()
92
93
94 win = TreeViewFilterWindow()
95 win.connect("destroy", Gtk.main_quit)
96 win.show_all()
97 Gtk.main()

```

CELLRENDERERS

`Gtk.CellRenderer` widgets are used to display information within widgets such as the `Gtk.TreeView` or `Gtk.ComboBox`. They work closely with the associated widgets and are very powerful, with lots of configuration options for displaying a large amount of data in different ways. There are seven `Gtk.CellRenderer` widgets which can be used for different purposes:

- `Gtk.CellRendererText`
- `Gtk.CellRendererToggle`
- `Gtk.CellRendererPixbuf`
- `Gtk.CellRendererCombo`
- `Gtk.CellRendererProgress`
- `Gtk.CellRendererSpinner`
- `Gtk.CellRendererSpin`
- `Gtk.CellRendererAccel`

14.1 CellRendererText

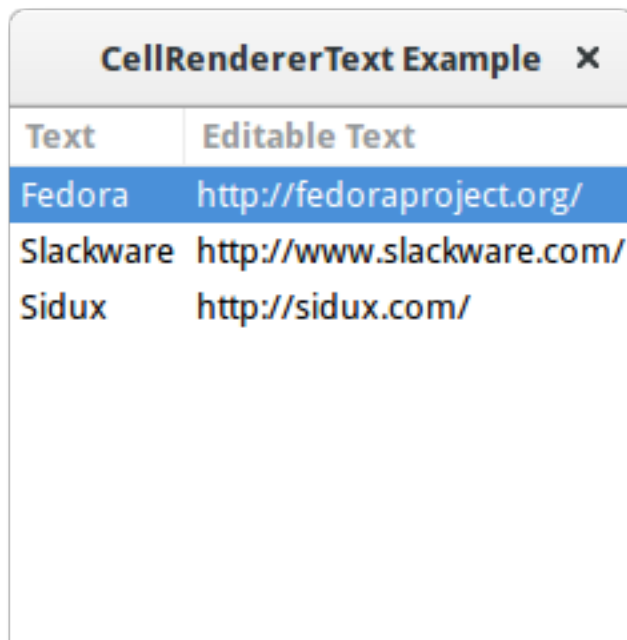
A `Gtk.CellRendererText` renders a given text in its cell, using the font, color and style information provided by its properties. The text will be ellipsized if it is too long and the “ellipsize” property allows it.

By default, text in `Gtk.CellRendererText` widgets is not editable. This can be changed by setting the value of the “editable” property to `True`:

```
cell.set_property("editable", True)
```

You can then connect to the “edited” signal and update your `Gtk.TreeModel` accordingly.

14.1.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class CellRendererTextWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="CellRendererText Example")
10
11         self.set_default_size(200, 200)
12
13         self.liststore = Gtk.ListStore(str, str)
14         self.liststore.append(["Fedora", "https://fedoraproject.org/"])
15         self.liststore.append(["Slackware", "http://www.slackware.com/"])
16         self.liststore.append(["Sidux", "http://sidux.com/"])
17
18         treeview = Gtk.TreeView(model=self.liststore)
19
20         renderer_text = Gtk.CellRendererText()
21         column_text = Gtk.TreeViewColumn("Text", renderer_text, text=0)
22         treeview.append_column(column_text)
23
24         renderer_editabletext = Gtk.CellRendererText()
25         renderer_editabletext.set_property("editable", True)
26
27         column_editabletext = Gtk.TreeViewColumn(
28             "Editable Text", renderer_editabletext, text=1
29         )
30         treeview.append_column(column_editabletext)
31

```

(continues on next page)

(continued from previous page)

```

32     renderer_editabletext.connect("edited", self.text_edited)
33
34     self.add(treeview)
35
36     def text_edited(self, widget, path, text):
37         self.liststore[path][1] = text
38
39
40 win = CellRendererTextWindow()
41 win.connect("destroy", Gtk.main_quit)
42 win.show_all()
43 Gtk.main()

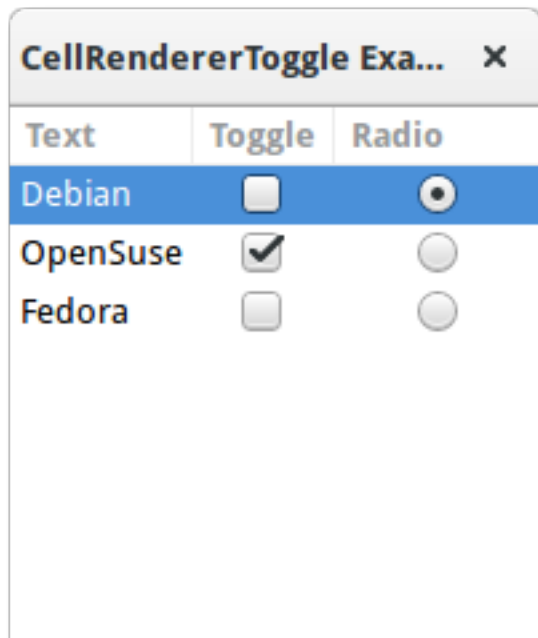
```

14.2 CellRendererToggle

`Gtk.CellRendererToggle` renders a toggle button in a cell. The button is drawn as a radio- or checkbutton, depending on the “radio” property. When activated, it emits the “toggled” signal.

As a `Gtk.CellRendererToggle` can have two states, active and not active, you most likely want to bind the “active” property on the cell renderer to a boolean value in the model, thus causing the check button to reflect the state of the model.

14.2.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5

```

(continues on next page)

(continued from previous page)

```

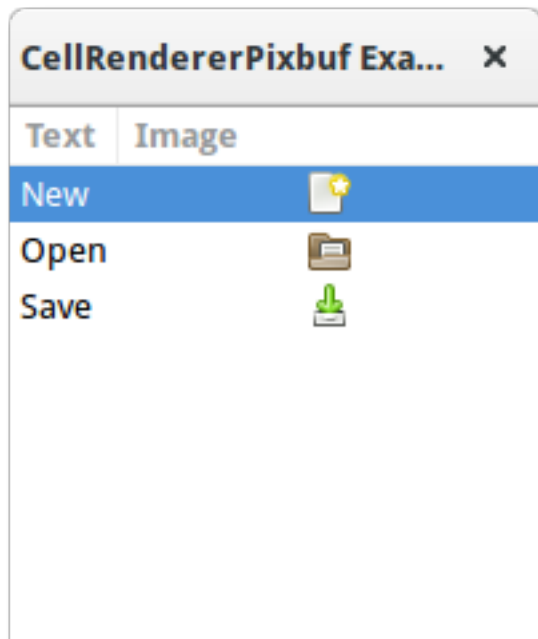
6
7 class CellRendererToggleWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="CellRendererToggle Example")
10
11         self.set_default_size(200, 200)
12
13         self.liststore = Gtk.ListStore(str, bool, bool)
14         self.liststore.append(["Debian", False, True])
15         self.liststore.append(["OpenSuse", True, False])
16         self.liststore.append(["Fedora", False, False])
17
18         treeview = Gtk.TreeView(model=self.liststore)
19
20         renderer_text = Gtk.CellRendererText()
21         column_text = Gtk.TreeViewColumn("Text", renderer_text, text=0)
22         treeview.append_column(column_text)
23
24         renderer_toggle = Gtk.CellRendererToggle()
25         renderer_toggle.connect("toggled", self.on_cell_toggled)
26
27         column_toggle = Gtk.TreeViewColumn("Toggle", renderer_toggle, active=1)
28         treeview.append_column(column_toggle)
29
30         renderer_radio = Gtk.CellRendererToggle()
31         renderer_radio.set_radio(True)
32         renderer_radio.connect("toggled", self.on_cell_radio_toggled)
33
34         column_radio = Gtk.TreeViewColumn("Radio", renderer_radio, active=2)
35         treeview.append_column(column_radio)
36
37         self.add(treeview)
38
39     def on_cell_toggled(self, widget, path):
40         self.liststore[path][1] = not self.liststore[path][1]
41
42     def on_cell_radio_toggled(self, widget, path):
43         selected_path = Gtk.TreePath(path)
44         for row in self.liststore:
45             row[2] = row.path == selected_path
46
47
48 win = CellRendererToggleWindow()
49 win.connect("destroy", Gtk.main_quit)
50 win.show_all()
51 Gtk.main()

```

14.3 CellRendererPixbuf

A `Gtk.CellRendererPixbuf` can be used to render an image in a cell. It allows to render either a given `Gdk.Pixbuf` (set via the “`pixbuf`” property) or a named icon (set via the “`icon-name`” property).

14.3.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class CellRendererPixbufWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="CellRendererPixbuf Example")
10
11          self.set_default_size(200, 200)
12
13          self.liststore = Gtk.ListStore(str, str)
14          self.liststore.append(["New", "document-new"])
15          self.liststore.append(["Open", "document-open"])
16          self.liststore.append(["Save", "document-save"])
17
18          treeview = Gtk.TreeView(model=self.liststore)
19
20          renderer_text = Gtk.CellRendererText()
21          column_text = Gtk.TreeViewColumn("Text", renderer_text, text=0)
22          treeview.append_column(column_text)
23
24          renderer_pixbuf = Gtk.CellRendererPixbuf()
```

(continues on next page)

(continued from previous page)

```

25
26     column_pixbuf = Gtk.TreeViewColumn("Image", renderer_pixbuf, icon_name=1)
27     treeview.append_column(column_pixbuf)
28
29     self.add(treeview)
30
31
32 win = CellRendererPixbufWindow()
33 win.connect("destroy", Gtk.main_quit)
34 win.show_all()
35 Gtk.main()

```

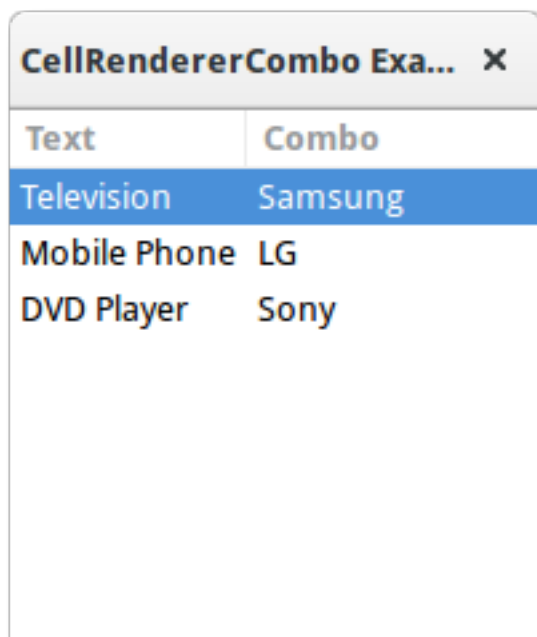
14.4 CellRendererCombo

`Gtk.CellRendererCombo` renders text in a cell like `Gtk.CellRendererText` from which it is derived. But while the latter offers a simple entry to edit the text, `Gtk.CellRendererCombo` offers a `Gtk.ComboBox` widget to edit the text. The values to display in the combo box are taken from the `Gtk.TreeModel` specified in the “model” property.

The combo cell renderer takes care of adding a text cell renderer to the combo box and sets it to display the column specified by its “text-column” property.

A `Gtk.CellRendererCombo` can operate in two modes. It can be used with and without an associated `Gtk.Entry` widget, depending on the value of the “has-entry” property.

14.4.1 Example



```

1 import gi
2

```

(continues on next page)

(continued from previous page)

```

3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class CellRendererComboWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="CellRendererCombo Example")
10
11         self.set_default_size(200, 200)
12
13         liststore_manufacturers = Gtk.ListStore(str)
14         manufacturers = ["Sony", "LG", "Panasonic", "Toshiba", "Nokia", "Samsung"]
15         for item in manufacturers:
16             liststore_manufacturers.append([item])
17
18         self.liststore_hardware = Gtk.ListStore(str, str)
19         self.liststore_hardware.append(["Television", "Samsung"])
20         self.liststore_hardware.append(["Mobile Phone", "LG"])
21         self.liststore_hardware.append(["DVD Player", "Sony"])
22
23         treeview = Gtk.TreeView(model=self.liststore_hardware)
24
25         renderer_text = Gtk.CellRendererText()
26         column_text = Gtk.TreeViewColumn("Text", renderer_text, text=0)
27         treeview.append_column(column_text)
28
29         renderer_combo = Gtk.CellRendererCombo()
30         renderer_combo.set_property("editable", True)
31         renderer_combo.set_property("model", liststore_manufacturers)
32         renderer_combo.set_property("text-column", 0)
33         renderer_combo.set_property("has-entry", False)
34         renderer_combo.connect("edited", self.on_combo_changed)
35
36         column_combo = Gtk.TreeViewColumn("Combo", renderer_combo, text=1)
37         treeview.append_column(column_combo)
38
39         self.add(treeview)
40
41     def on_combo_changed(self, widget, path, text):
42         self.liststore_hardware[path][1] = text
43
44
45 win = CellRendererComboWindow()
46 win.connect("destroy", Gtk.main_quit)
47 win.show_all()
48 Gtk.main()

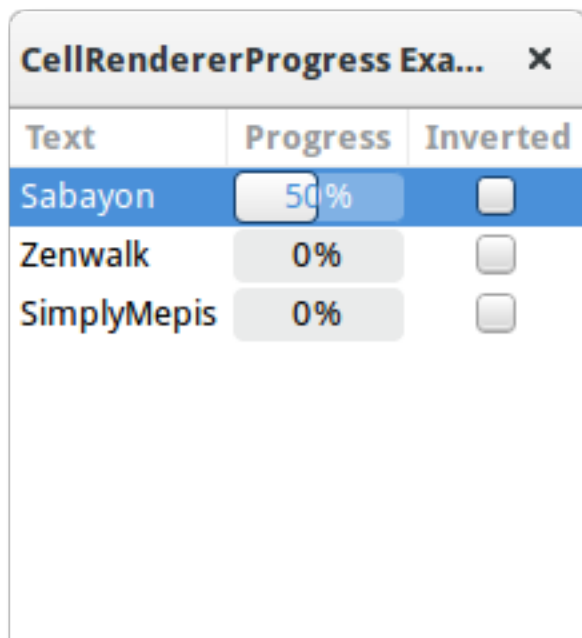
```

14.5 CellRendererProgress

`Gtk.CellRendererProgress` renders a numeric value as a progress bar in a cell. Additionally, it can display a text on top of the progress bar.

The percentage value of the progress bar can be modified by changing the “value” property. Similar to `Gtk.ProgressBar`, you can enable the *activity mode* by incrementing the “pulse” property instead of the “value” property.

14.5.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk, GLib
5
6
7  class CellRendererProgressWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="CellRendererProgress Example")
10
11         self.set_default_size(200, 200)
12
13         self.liststore = Gtk.ListStore(str, int, bool)
14         self.current_iter = self.liststore.append(["Sabayon", 0, False])
15         self.liststore.append(["Zenwalk", 0, False])
16         self.liststore.append(["SimplyMepis", 0, False])
17
18         treeview = Gtk.TreeView(model=self.liststore)
19
20         renderer_text = Gtk.CellRendererText()
21         column_text = Gtk.TreeViewColumn("Text", renderer_text, text=0)

```

(continues on next page)

(continued from previous page)

```

22     treeview.append_column(column_text)
23
24     renderer_progress = Gtk.CellRendererProgress()
25     column_progress = Gtk.TreeViewColumn(
26         "Progress", renderer_progress, value=1, inverted=2
27     )
28     treeview.append_column(column_progress)
29
30     renderer_toggle = Gtk.CellRendererToggle()
31     renderer_toggle.connect("toggled", self.on_inverted_toggled)
32     column_toggle = Gtk.TreeViewColumn("Inverted", renderer_toggle, active=2)
33     treeview.append_column(column_toggle)
34
35     self.add(treeview)
36
37     self.timeout_id = GLib.timeout_add(100, self.on_timeout, None)
38
39     def on_inverted_toggled(self, widget, path):
40         self.liststore[path][2] = not self.liststore[path][2]
41
42     def on_timeout(self, user_data):
43         new_value = self.liststore[self.current_iter][1] + 1
44         if new_value > 100:
45             self.current_iter = self.liststore.iter_next(self.current_iter)
46             if self.current_iter is None:
47                 self.reset_model()
48             new_value = self.liststore[self.current_iter][1] + 1
49
50         self.liststore[self.current_iter][1] = new_value
51         return True
52
53     def reset_model(self):
54         for row in self.liststore:
55             row[1] = 0
56         self.current_iter = self.liststore.get_iter_first()
57
58
59 win = CellRendererProgressWindow()
60 win.connect("destroy", Gtk.main_quit)
61 win.show_all()
62 Gtk.main()

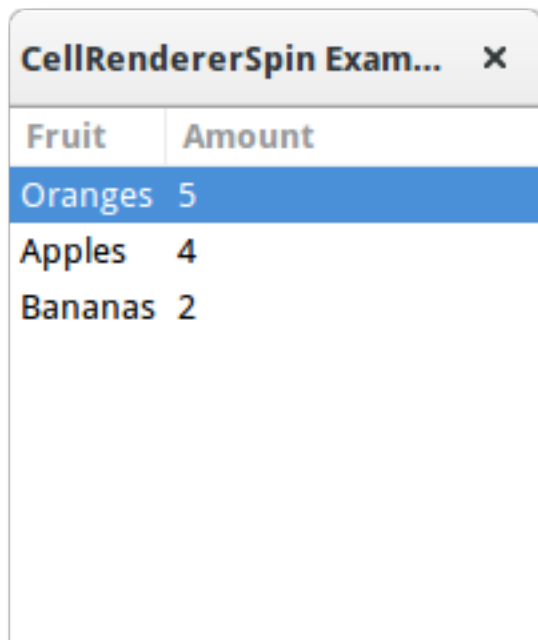
```

14.6 CellRendererSpin

`Gtk.CellRendererSpin` renders text in a cell like `Gtk.CellRendererText` from which it is derived. But while the latter offers a simple entry to edit the text, `Gtk.CellRendererSpin` offers a `Gtk.SpinButton` widget. Of course, that means that the text has to be parseable as a floating point number.

The range of the spinbutton is taken from the adjustment property of the cell renderer, which can be set explicitly or mapped to a column in the tree model, like all properties of cell renders. `Gtk.CellRendererSpin` also has properties for the climb rate and the number of digits to display.

14.6.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class CellRendererSpinWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="CellRendererSpin Example")
10
11         self.set_default_size(200, 200)
12
13         self.liststore = Gtk.ListStore(str, int)
14         self.liststore.append(["Oranges", 5])
15         self.liststore.append(["Apples", 4])
16         self.liststore.append(["Bananas", 2])
17
18         treeview = Gtk.TreeView(model=self.liststore)
19
20         renderer_text = Gtk.CellRendererText()
21         column_text = Gtk.TreeViewColumn("Fruit", renderer_text, text=0)
22         treeview.append_column(column_text)
23
24         renderer_spin = Gtk.CellRendererSpin()
25         renderer_spin.connect("edited", self.on_amount_edited)
26         renderer_spin.set_property("editable", True)
27
28         adjustment = Gtk.Adjustment(
29             value=0,
30             lower=0,
31             upper=100,

```

(continues on next page)

(continued from previous page)

```
32         step_increment=1,  
33         page_increment=10,  
34         page_size=0,  
35     )  
36     renderer_spin.set_property("adjustment", adjustment)  
37  
38     column_spin = Gtk.TreeViewColumn("Amount", renderer_spin, text=1)  
39     treeview.append_column(column_spin)  
40  
41     self.add(treeview)  
42  
43     def on_amount_edited(self, widget, path, value):  
44         self.liststore[path][1] = int(value)  
45  
46  
47 win = CellRendererSpinWindow()  
48 win.connect("destroy", Gtk.main_quit)  
49 win.show_all()  
50 Gtk.main()
```


COMBOBOX

A `Gtk.ComboBox` allows for the selection of an item from a dropdown menu. They are preferable to having many radio buttons on screen as they take up less room. If appropriate, it can show extra information about each item, such as text, a picture, a checkbox, or a progress bar.

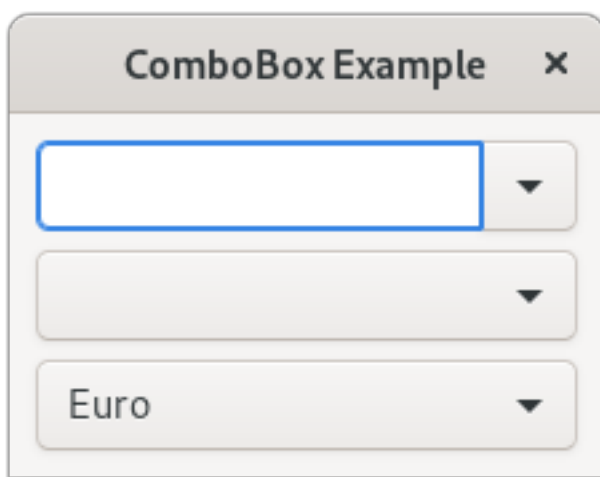
`Gtk.ComboBox` is very similar to `Gtk.TreeView`, as both use the model-view pattern; the list of valid choices is specified in the form of a tree model, and the display of the choices can be adapted to the data in the model by using *cell renderers*. If the combo box contains a large number of items, it may be better to display them in a grid rather than a list. This can be done by calling `Gtk.ComboBox.set_wrap_width()`.

A default value can be set by calling `Gtk.ComboBox.set_active()` with the index of the desired value.

The `Gtk.ComboBox` widget usually restricts the user to the available choices, but it can optionally have an `Gtk.Entry`, allowing the user to enter arbitrary text if none of the available choices are suitable. To do this, use one of the static methods `Gtk.ComboBox.new_with_entry()` or `Gtk.ComboBox.new_with_model_and_entry()` to create an `Gtk.ComboBox` instance.

For a simple list of textual choices, the model-view API of `Gtk.ComboBox` can be a bit overwhelming. In this case, `Gtk.ComboBoxText` offers a simple alternative. Both `Gtk.ComboBox` and `Gtk.ComboBoxText` can contain an entry.

15.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class ComboBoxWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="ComboBox Example")
10
11         self.set_border_width(10)
12
13         name_store = Gtk.ListStore(int, str)
14         name_store.append([1, "Billy Bob"])
15         name_store.append([11, "Billy Bob Junior"])
16         name_store.append([12, "Sue Bob"])
17         name_store.append([2, "Joey Jojo"])
18         name_store.append([3, "Rob McRoberts"])
19         name_store.append([31, "Xavier McRoberts"])
20
21         vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
22
23         name_combo = Gtk.ComboBox.new_with_model_and_entry(name_store)
24         name_combo.connect("changed", self.on_name_combo_changed)
25         name_combo.set_entry_text_column(1)
26         vbox.pack_start(name_combo, False, False, 0)
27
28         country_store = Gtk.ListStore(str)
29         countries = [
30             "Austria",
31             "Brazil",
32             "Belgium",
33             "France",
34             "Germany",
35             "Switzerland",
36             "United Kingdom",
37             "United States of America",
38             "Uruguay",
39         ]
40         for country in countries:
41             country_store.append([country])
42
43         country_combo = Gtk.ComboBox.new_with_model(country_store)
44         country_combo.connect("changed", self.on_country_combo_changed)
45         renderer_text = Gtk.CellRendererText()
46         country_combo.pack_start(renderer_text, True)
47         country_combo.add_attribute(renderer_text, "text", 0)
48         vbox.pack_start(country_combo, False, False, True)
49
50         currencies = [
51             "Euro",
52             "US Dollars",
53             "British Pound",
54             "Japanese Yen",
55             "Russian Ruble",
56             "Mexican peso",
57             "Swiss franc",

```

(continues on next page)

(continued from previous page)

```

58     ]
59     currency_combo = Gtk.ComboBoxText()
60     currency_combo.set_entry_text_column(0)
61     currency_combo.connect("changed", self.on_currency_combo_changed)
62     for currency in currencies:
63         currency_combo.append_text(currency)
64
65     currency_combo.set_active(0)
66     vbox.pack_start(currency_combo, False, False, 0)
67
68     self.add(vbox)
69
70     def on_name_combo_changed(self, combo):
71         tree_iter = combo.get_active_iter()
72         if tree_iter is not None:
73             model = combo.get_model()
74             row_id, name = model[tree_iter][:2]
75             print("Selected: ID=%d, name=%s" % (row_id, name))
76         else:
77             entry = combo.get_child()
78             print("Entered: %s" % entry.get_text())
79
80     def on_country_combo_changed(self, combo):
81         tree_iter = combo.get_active_iter()
82         if tree_iter is not None:
83             model = combo.get_model()
84             country = model[tree_iter][0]
85             print("Selected: country=%s" % country)
86
87     def on_currency_combo_changed(self, combo):
88         text = combo.get_active_text()
89         if text is not None:
90             print("Selected: currency=%s" % text)
91
92
93 win = ComboBoxWindow()
94 win.connect("destroy", Gtk.main_quit)
95 win.show_all()
96 Gtk.main()

```

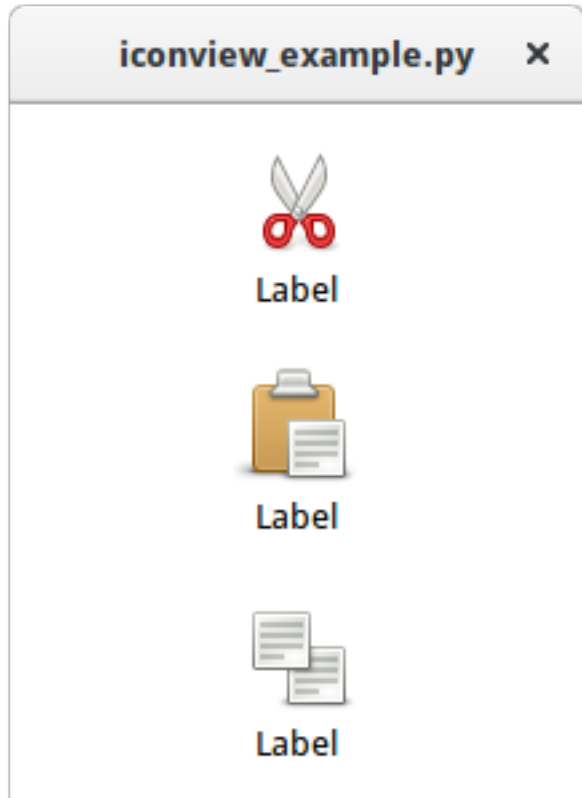

ICONVIEW

A `Gtk.IconView` is a widget that displays a collection of icons in a grid view. It supports features such as drag and drop, multiple selections and item reordering.

Similarly to `Gtk.TreeView`, `Gtk.IconView` uses a `Gtk.ListStore` for its model. Instead of using *cell renderers*, `Gtk.IconView` requires that one of the columns in its `Gtk.ListStore` contains `GdkPixbuf.Pixbuf` objects.

`Gtk.IconView` supports numerous selection modes to allow for either selecting multiple icons at a time, restricting selections to just one item or disallowing selecting items completely. To specify a selection mode, the `Gtk.IconView.set_selection_mode()` method is used with one of the `Gtk.SelectionMode` selection modes.

16.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5 from gi.repository.GdkPixbuf import Pixbuf
6
7 icons = ["edit-cut", "edit-paste", "edit-copy"]
8
9
10 class IconViewWindow(Gtk.Window):
11     def __init__(self):
12         super().__init__()
13         self.set_default_size(200, 200)
14
15         liststore = Gtk.ListStore(Pixbuf, str)
16         iconview = Gtk.IconView.new()
17         iconview.set_model(liststore)
18         iconview.set_pixbuf_column(0)
19         iconview.set_text_column(1)
20
21         for icon in icons:
22             pixbuf = Gtk.IconTheme.get_default().load_icon(icon, 64, 0)
23             liststore.append([pixbuf, "Label"])
24
25         self.add(iconview)
26
27
28 win = IconViewWindow()
29 win.connect("destroy", Gtk.main_quit)
30 win.show_all()
31 Gtk.main()
```

MULTILINE TEXT EDITOR

The `Gtk.TextView` widget can be used to display and edit large amounts of formatted text. Like the `Gtk.TreeView`, it has a model/view design. In this case the `Gtk.TextBuffer` is the model which represents the text being edited. This allows two or more `Gtk.TextView` widgets to share the same `Gtk.TextBuffer`, and allows those text buffers to be displayed slightly differently. Or you could maintain several text buffers and choose to display each one at different times in the same `Gtk.TextView` widget.

17.1 The View

The `Gtk.TextView` is the frontend with which the user can add, edit and delete textual data. They are commonly used to edit multiple lines of text. When creating a `Gtk.TextView` it contains its own default `Gtk.TextBuffer`, which you can access via the `Gtk.TextView.get_buffer()` method.

By default, text can be added, edited and removed from the `Gtk.TextView`. You can disable this by calling `Gtk.TextView.set_editable()`. If the text is not editable, you usually want to hide the text cursor with `Gtk.TextView.set_cursor_visible()` as well. In some cases it may be useful to set the justification of the text with `Gtk.TextView.set_justification()`. The text can be displayed at the left edge (`Gtk.Justification.LEFT`), at the right edge (`Gtk.Justification.RIGHT`), centered (`Gtk.Justification.CENTER`), or distributed across the complete width (`Gtk.Justification.FILL`).

Another default setting of the `Gtk.TextView` widget is long lines of text will continue horizontally until a break is entered. To wrap the text and prevent it going off the edges of the screen call `Gtk.TextView.set_wrap_mode()`.

17.2 The Model

The `Gtk.TextBuffer` is the core of the `Gtk.TextView` widget, and is used to hold whatever text is being displayed in the `Gtk.TextView`. Setting and retrieving the contents is possible with `Gtk.TextBuffer.set_text()` and `Gtk.TextBuffer.get_text()`. However, most text manipulation is accomplished with *iterators*, represented by a `Gtk.TextIter`. An iterator represents a position between two characters in the text buffer. Iterators are not valid indefinitely; whenever the buffer is modified in a way that affects the contents of the buffer, all outstanding iterators become invalid.

Because of this, iterators can't be used to preserve positions across buffer modifications. To preserve a position, use `Gtk.TextMark`. A text buffer contains two built-in marks; an "insert" mark (which is the position of the cursor) and the "selection_bound" mark. Both of them can be retrieved using `Gtk.TextBuffer.get_insert()` and `Gtk.TextBuffer.get_selection_bound()`, respectively. By default, the location of a `Gtk.TextMark` is not shown. This can be changed by calling `Gtk.TextMark.set_visible()`.

Many methods exist to retrieve a `Gtk.TextIter`. For instance, `Gtk.TextBuffer.get_start_iter()` returns an iterator pointing to the first position in the text buffer, whereas `Gtk.TextBuffer.get_end_iter()`

returns an iterator pointing past the last valid character. Retrieving the bounds of the selected text can be achieved by calling `Gtk.TextBuffer.get_selection_bounds()`.

To insert text at a specific position use `Gtk.TextBuffer.insert()`. Another useful method is `Gtk.TextBuffer.insert_at_cursor()` which inserts text wherever the cursor may be currently positioned. To remove portions of the text buffer use `Gtk.TextBuffer.delete()`.

In addition, `Gtk.TextIter` can be used to locate textual matches in the buffer using `Gtk.TextIter.forward_search()` and `Gtk.TextIter.backward_search()`. The start and end iterators are used as the starting point of the search and move forwards/backwards depending on requirements.

17.3 Tags

Text in a buffer can be marked with tags. A tag is an attribute that can be applied to some range of text. For example, a tag might be called “bold” and make the text inside the tag bold. However, the tag concept is more general than that; tags don’t have to affect appearance. They can instead affect the behaviour of mouse and key presses, “lock” a range of text so the user can’t edit it, or countless other things. A tag is represented by a `Gtk.TextTag` object. One `Gtk.TextTag` can be applied to any number of text ranges in any number of buffers.

Each tag is stored in a `Gtk.TextTagTable`. A tag table defines a set of tags that can be used together. Each buffer has one tag table associated with it; only tags from that tag table can be used with the buffer. A single tag table can be shared between multiple buffers, however.

To specify that some text in the buffer should have specific formatting, you must define a tag to hold that formatting information, and then apply that tag to the region of text using `Gtk.TextBuffer.create_tag()` and `Gtk.TextBuffer.apply_tag()`:

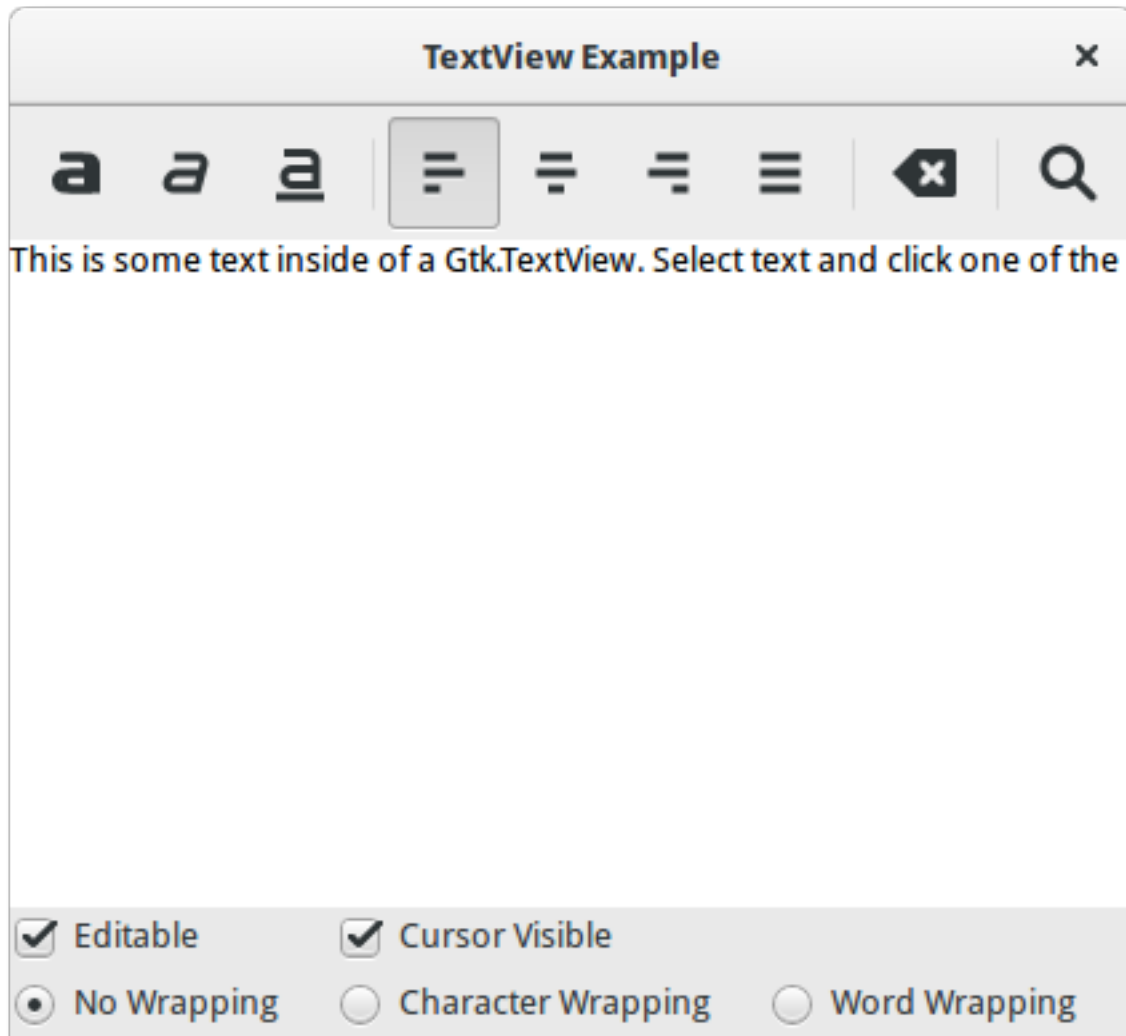
```
tag = textbuffer.create_tag("orange_bg", background="orange")
textbuffer.apply_tag(tag, start_iter, end_iter)
```

The following are some of the common styles applied to text:

- Background colour (“background” property)
- Foreground colour (“foreground” property)
- Underline (“underline” property)
- Bold (“weight” property)
- Italics (“style” property)
- Strikethrough (“strikethrough” property)
- Justification (“justification” property)
- Size (“size” and “size-points” properties)
- Text wrapping (“wrap-mode” property)

You can also delete particular tags later using `Gtk.TextBuffer.remove_tag()` or delete all tags in a given region by calling `Gtk.TextBuffer.remove_all_tags()`.

17.4 Example



```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk, Pango
5
6
7 class SearchDialog(Gtk.Dialog):
8     def __init__(self, parent):
9         super().__init__(title="Search", transient_for=parent, modal=True)
10        self.add_buttons(
11            Gtk.STOCK_FIND,
12            Gtk.ResponseType.OK,
13            Gtk.STOCK_CANCEL,
14            Gtk.ResponseType.CANCEL,
15        )
16
17        box = self.get_content_area()

```

(continues on next page)

(continued from previous page)

```

18         label = Gtk.Label(label="Insert text you want to search for:")
19         box.add(label)
20
21
22         self.entry = Gtk.Entry()
23         box.add(self.entry)
24
25         self.show_all()
26
27
28 class TextViewWindow(Gtk.Window):
29     def __init__(self):
30         Gtk.Window.__init__(self, title="TextView Example")
31
32         self.set_default_size(-1, 350)
33
34         self.grid = Gtk.Grid()
35         self.add(self.grid)
36
37         self.create_textview()
38         self.create_toolbar()
39         self.create_buttons()
40
41     def create_toolbar(self):
42         toolbar = Gtk.Toolbar()
43         self.grid.attach(toolbar, 0, 0, 3, 1)
44
45         button_bold = Gtk.ToolButton()
46         button_bold.set_icon_name("format-text-bold-symbolic")
47         toolbar.insert(button_bold, 0)
48
49         button_italic = Gtk.ToolButton()
50         button_italic.set_icon_name("format-text-italic-symbolic")
51         toolbar.insert(button_italic, 1)
52
53         button_underline = Gtk.ToolButton()
54         button_underline.set_icon_name("format-text-underline-symbolic")
55         toolbar.insert(button_underline, 2)
56
57         button_bold.connect("clicked", self.on_button_clicked, self.tag_bold)
58         button_italic.connect("clicked", self.on_button_clicked, self.tag_italic)
59         button_underline.connect("clicked", self.on_button_clicked, self.tag_
↳underline)
60
61         toolbar.insert(Gtk.SeparatorToolItem(), 3)
62
63         radio_justifyleft = Gtk.RadioToolButton()
64         radio_justifyleft.set_icon_name("format-justify-left-symbolic")
65         toolbar.insert(radio_justifyleft, 4)
66
67         radio_justifycenter = Gtk.RadioToolButton.new_from_widget(radio_justifyleft)
68         radio_justifycenter.set_icon_name("format-justify-center-symbolic")
69         toolbar.insert(radio_justifycenter, 5)
70
71         radio_justifyright = Gtk.RadioToolButton.new_from_widget(radio_justifyleft)
72         radio_justifyright.set_icon_name("format-justify-right-symbolic")
73         toolbar.insert(radio_justifyright, 6)

```

(continues on next page)

(continued from previous page)

```

74
75     radio_justifyfill = Gtk.RadioToolButton.new_from_widget(radio_justifyleft)
76     radio_justifyfill.set_icon_name("format-justify-fill-symbolic")
77     toolbar.insert(radio_justifyfill, 7)
78
79     radio_justifyleft.connect(
80         "toggled", self.on_justify_toggled, Gtk.Justification.LEFT
81     )
82     radio_justifycenter.connect(
83         "toggled", self.on_justify_toggled, Gtk.Justification.CENTER
84     )
85     radio_justifyright.connect(
86         "toggled", self.on_justify_toggled, Gtk.Justification.RIGHT
87     )
88     radio_justifyfill.connect(
89         "toggled", self.on_justify_toggled, Gtk.Justification.FILL
90     )
91
92     toolbar.insert(Gtk.SeparatorToolItem(), 8)
93
94     button_clear = Gtk.ToolButton()
95     button_clear.set_icon_name("edit-clear-symbolic")
96     button_clear.connect("clicked", self.on_clear_clicked)
97     toolbar.insert(button_clear, 9)
98
99     toolbar.insert(Gtk.SeparatorToolItem(), 10)
100
101     button_search = Gtk.ToolButton()
102     button_search.set_icon_name("system-search-symbolic")
103     button_search.connect("clicked", self.on_search_clicked)
104     toolbar.insert(button_search, 11)
105
106     def create_textview(self):
107         scrolledwindow = Gtk.ScrolledWindow()
108         scrolledwindow.set_hexpand(True)
109         scrolledwindow.set_vexpand(True)
110         self.grid.attach(scrolledwindow, 0, 1, 3, 1)
111
112         self.textview = Gtk.TextView()
113         self.textbuffer = self.textview.get_buffer()
114         self.textbuffer.set_text(
115             "This is some text inside of a Gtk.TextView. "
116             + "Select text and click one of the buttons 'bold', 'italic', "
117             + "or 'underline' to modify the text accordingly."
118         )
119         scrolledwindow.add(self.textview)
120
121         self.tag_bold = self.textbuffer.create_tag("bold", weight=Pango.Weight.BOLD)
122         self.tag_italic = self.textbuffer.create_tag("italic", style=Pango.Style.
↵ITALIC)
123         self.tag_underline = self.textbuffer.create_tag(
124             "underline", underline=Pango.Underline.SINGLE
125         )
126         self.tag_found = self.textbuffer.create_tag("found", background="yellow")
127
128     def create_buttons(self):
129         check_editable = Gtk.CheckButton(label="Editable")

```

(continues on next page)

(continued from previous page)

```

130     check_editable.set_active(True)
131     check_editable.connect("toggled", self.on_editable_toggled)
132     self.grid.attach(check_editable, 0, 2, 1, 1)
133
134     check_cursor = Gtk.CheckButton(label="Cursor Visible")
135     check_cursor.set_active(True)
136     check_editable.connect("toggled", self.on_cursor_toggled)
137     self.grid.attach_next_to(
138         check_cursor, check_editable, Gtk.PositionType.RIGHT, 1, 1
139     )
140
141     radio_wrapnone = Gtk.RadioButton.new_with_label_from_widget(None, "No Wrapping
↪")
142     self.grid.attach(radio_wrapnone, 0, 3, 1, 1)
143
144     radio_wrapchar = Gtk.RadioButton.new_with_label_from_widget(
145         radio_wrapnone, "Character Wrapping"
146     )
147     self.grid.attach_next_to(
148         radio_wrapchar, radio_wrapnone, Gtk.PositionType.RIGHT, 1, 1
149     )
150
151     radio_wrapword = Gtk.RadioButton.new_with_label_from_widget(
152         radio_wrapnone, "Word Wrapping"
153     )
154     self.grid.attach_next_to(
155         radio_wrapword, radio_wrapchar, Gtk.PositionType.RIGHT, 1, 1
156     )
157
158     radio_wrapnone.connect("toggled", self.on_wrap_toggled, Gtk.WrapMode.NONE)
159     radio_wrapchar.connect("toggled", self.on_wrap_toggled, Gtk.WrapMode.CHAR)
160     radio_wrapword.connect("toggled", self.on_wrap_toggled, Gtk.WrapMode.WORD)
161
162     def on_button_clicked(self, widget, tag):
163         bounds = self.textbuffer.get_selection_bounds()
164         if len(bounds) != 0:
165             start, end = bounds
166             self.textbuffer.apply_tag(tag, start, end)
167
168     def on_clear_clicked(self, widget):
169         start = self.textbuffer.get_start_iter()
170         end = self.textbuffer.get_end_iter()
171         self.textbuffer.remove_all_tags(start, end)
172
173     def on_editable_toggled(self, widget):
174         self.textview.set_editable(widget.get_active())
175
176     def on_cursor_toggled(self, widget):
177         self.textview.set_cursor_visible(widget.get_active())
178
179     def on_wrap_toggled(self, widget, mode):
180         self.textview.set_wrap_mode(mode)
181
182     def on_justify_toggled(self, widget, justification):
183         self.textview.set_justification(justification)
184
185     def on_search_clicked(self, widget):

```

(continues on next page)

(continued from previous page)

```
186     dialog = SearchDialog(self)
187     response = dialog.run()
188     if response == Gtk.ResponseType.OK:
189         cursor_mark = self.textbuffer.get_insert()
190         start = self.textbuffer.get_iter_at_mark(cursor_mark)
191         if start.get_offset() == self.textbuffer.get_char_count():
192             start = self.textbuffer.get_start_iter()
193
194         self.search_and_mark(dialog.entry.get_text(), start)
195
196     dialog.destroy()
197
198     def search_and_mark(self, text, start):
199         end = self.textbuffer.get_end_iter()
200         match = start.forward_search(text, 0, end)
201
202         if match is not None:
203             match_start, match_end = match
204             self.textbuffer.apply_tag(self.tag_found, match_start, match_end)
205             self.search_and_mark(text, match_end)
206
207
208 win = TextViewWindow()
209 win.connect("destroy", Gtk.main_quit)
210 win.show_all()
211 Gtk.main()
```


DIALOGS

Dialog windows are very similar to standard windows, and are used to provide or retrieve information from the user. They are often used to provide a preferences window, for example. The major difference a dialog has is some prepacked widgets which layout the dialog automatically. From there, we can simply add labels, buttons, check buttons, etc. Another big difference is the handling of responses to control how the application should behave after the dialog has been interacted with.

There are several derived Dialog classes which you might find useful. `Gtk.MessageDialog` is used for most simple notifications. But at other times you might need to derive your own dialog class to provide more complex functionality.

18.1 Custom Dialogs

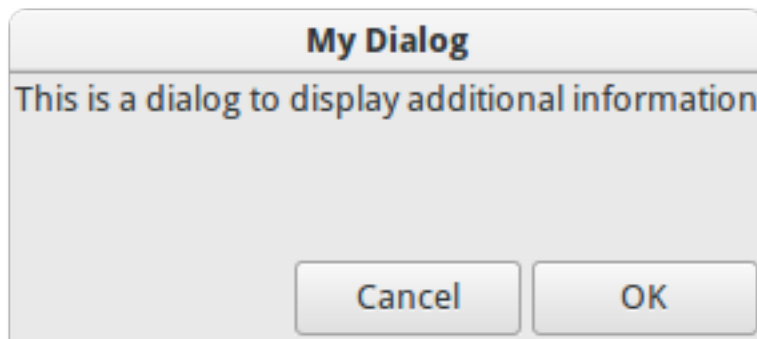
To pack widgets into a custom dialog, you should pack them into the `Gtk.Box`, available via `Gtk.Dialog.get_content_area()`. To just add a `Gtk.Button` to the bottom of the dialog, you could use the `Gtk.Dialog.add_button()` method.

A ‘modal’ dialog (that is, one which freezes the rest of the application from user input), can be created by calling `Gtk.Dialog.set_modal` on the dialog or set the `flags` argument of the `Gtk.Dialog` constructor to include the `Gtk.DialogFlags.MODAL` flag.

Clicking a button will emit a signal called “response”. If you want to block waiting for a dialog to return before returning control flow to your code, you can call `Gtk.Dialog.run()`. This method returns an int which may be a value from the `Gtk.ResponseType` or it could be the custom response value that you specified in the `Gtk.Dialog` constructor or `Gtk.Dialog.add_button()`.

Finally, there are two ways to remove a dialog. The `Gtk.Widget.hide()` method removes the dialog from view, however keeps it stored in memory. This is useful to prevent having to construct the dialog again if it needs to be accessed at a later time. Alternatively, the `Gtk.Widget.destroy()` method can be used to delete the dialog from memory once it is no longer needed. It should be noted that if the dialog needs to be accessed after it has been destroyed, it will need to be constructed again otherwise the dialog window will be empty.

18.1.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class DialogExample(Gtk.Dialog):
8      def __init__(self, parent):
9          super().__init__(title="My Dialog", transient_for=parent, flags=0)
10         self.add_buttons(
11             Gtk.STOCK_CANCEL, Gtk.ResponseType.CANCEL, Gtk.STOCK_OK, Gtk.ResponseType.
↪OK
12         )
13
14         self.set_default_size(150, 100)
15
16         label = Gtk.Label(label="This is a dialog to display additional information")
17
18         box = self.get_content_area()
19         box.add(label)
20         self.show_all()
21
22
23  class DialogWindow(Gtk.Window):
24      def __init__(self):
25          Gtk.Window.__init__(self, title="Dialog Example")
26
27          self.set_border_width(6)
28
29          button = Gtk.Button(label="Open dialog")
30          button.connect("clicked", self.on_button_clicked)
31
32          self.add(button)
33
34      def on_button_clicked(self, widget):
35          dialog = DialogExample(self)
36          response = dialog.run()
37
38          if response == Gtk.ResponseType.OK:
39              print("The OK button was clicked")
40          elif response == Gtk.ResponseType.CANCEL:
41              print("The Cancel button was clicked")

```

(continues on next page)

(continued from previous page)

```

42         dialog.destroy()
43
44
45
46 win = DialogWindow()
47 win.connect("destroy", Gtk.main_quit)
48 win.show_all()
49 Gtk.main()

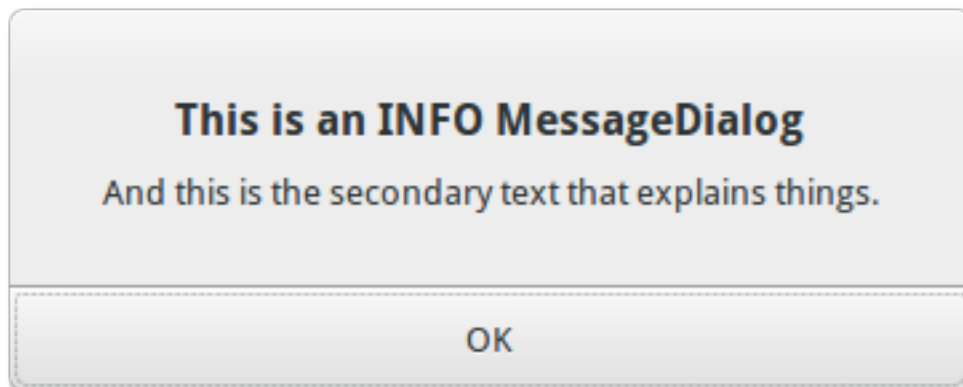
```

18.2 MessageDialog

`Gtk.MessageDialog` is a convenience class, used to create simple, standard message dialogs, with a message, an icon, and buttons for user response. You can specify the type of message and the text in the `Gtk.MessageDialog` constructor, as well as specifying standard buttons.

In some dialogs which require some further explanation of what has happened, a secondary text can be added. In this case, the primary message entered when creating the message dialog is made bigger and set to bold text. The secondary message can be set by calling `Gtk.MessageDialog.format_secondary_text()`.

18.2.1 Example



```

1  import gi
2
3  gi.require_version("Gtk", "3.0")
4  from gi.repository import Gtk
5
6
7  class MessageDialogWindow(Gtk.Window):
8      def __init__(self):
9          super().__init__(title="MessageDialog Example")
10
11         box = Gtk.Box(spacing=6)
12         self.add(box)
13
14         button1 = Gtk.Button(label="Information")
15         button1.connect("clicked", self.on_info_clicked)
16         box.add(button1)

```

(continues on next page)

(continued from previous page)

```
17
18     button2 = Gtk.Button(label="Error")
19     button2.connect("clicked", self.on_error_clicked)
20     box.add(button2)
21
22     button3 = Gtk.Button(label="Warning")
23     button3.connect("clicked", self.on_warn_clicked)
24     box.add(button3)
25
26     button4 = Gtk.Button(label="Question")
27     button4.connect("clicked", self.on_question_clicked)
28     box.add(button4)
29
30     def on_info_clicked(self, widget):
31         dialog = Gtk.MessageDialog(
32             transient_for=self,
33             flags=0,
34             message_type=Gtk.MessageType.INFO,
35             buttons=Gtk.ButtonsType.OK,
36             text="This is an INFO MessageDialog",
37         )
38         dialog.format_secondary_text(
39             "And this is the secondary text that explains things."
40         )
41         dialog.run()
42         print("INFO dialog closed")
43
44         dialog.destroy()
45
46     def on_error_clicked(self, widget):
47         dialog = Gtk.MessageDialog(
48             transient_for=self,
49             flags=0,
50             message_type=Gtk.MessageType.ERROR,
51             buttons=Gtk.ButtonsType.CANCEL,
52             text="This is an ERROR MessageDialog",
53         )
54         dialog.format_secondary_text(
55             "And this is the secondary text that explains things."
56         )
57         dialog.run()
58         print("ERROR dialog closed")
59
60         dialog.destroy()
61
62     def on_warn_clicked(self, widget):
63         dialog = Gtk.MessageDialog(
64             transient_for=self,
65             flags=0,
66             message_type=Gtk.MessageType.WARNING,
67             buttons=Gtk.ButtonsType.OK_CANCEL,
68             text="This is an WARNING MessageDialog",
69         )
70         dialog.format_secondary_text(
71             "And this is the secondary text that explains things."
72         )
73         response = dialog.run()
```

(continues on next page)

(continued from previous page)

```

74     if response == Gtk.ResponseType.OK:
75         print("WARN dialog closed by clicking OK button")
76     elif response == Gtk.ResponseType.CANCEL:
77         print("WARN dialog closed by clicking CANCEL button")
78
79     dialog.destroy()
80
81     def on_question_clicked(self, widget):
82         dialog = Gtk.MessageDialog(
83             transient_for=self,
84             flags=0,
85             message_type=Gtk.MessageType.QUESTION,
86             buttons=Gtk.ButtonsType.YES_NO,
87             text="This is an QUESTION MessageDialog",
88         )
89         dialog.format_secondary_text(
90             "And this is the secondary text that explains things."
91         )
92         response = dialog.run()
93         if response == Gtk.ResponseType.YES:
94             print("QUESTION dialog closed by clicking YES button")
95         elif response == Gtk.ResponseType.NO:
96             print("QUESTION dialog closed by clicking NO button")
97
98         dialog.destroy()
99
100
101 win = MessageDialogWindow()
102 win.connect("destroy", Gtk.main_quit)
103 win.show_all()
104 Gtk.main()

```

18.3 FileChooserDialog

The `Gtk.FileChooserDialog` is suitable for use with “File/Open” or “File/Save” menu items. You can use all of the `Gtk.FileChooser` methods on the file chooser dialog as well as those for `Gtk.Dialog`.

When creating a `Gtk.FileChooserDialog` you have to define the dialog’s purpose:

- To select a file for opening, as for a File/Open command, use `Gtk.FileChooserAction.OPEN`
- To save a file for the first time, as for a File/Save command, use `Gtk.FileChooserAction.SAVE`, and suggest a name such as “Untitled” with `Gtk.FileChooser.set_current_name()`.
- To save a file under a different name, as for a File/Save As command, use `Gtk.FileChooserAction.SAVE`, and set the existing filename with `Gtk.FileChooser.set_filename()`.
- To choose a folder instead of a file, use `Gtk.FileChooserAction.SELECT_FOLDER`.

`Gtk.FileChooserDialog` inherits from `Gtk.Dialog`, so buttons have response IDs such as `Gtk.ResponseType.ACCEPT` and `Gtk.ResponseType.CANCEL` which can be specified in the `Gtk.FileChooserDialog` constructor. In contrast to `Gtk.Dialog`, you can not use custom response codes with `Gtk.FileChooserDialog`. It expects that at least one button will have of the following response IDs:

- `Gtk.ResponseType.ACCEPT`
- `Gtk.ResponseType.OK`

- `Gtk.ResponseType.YES`
- `Gtk.ResponseType.APPLY`

When the user is finished selecting files, your program can get the selected names either as filenames (`Gtk.FileChooser.get_filename()`) or as URIs (`Gtk.FileChooser.get_uri()`).

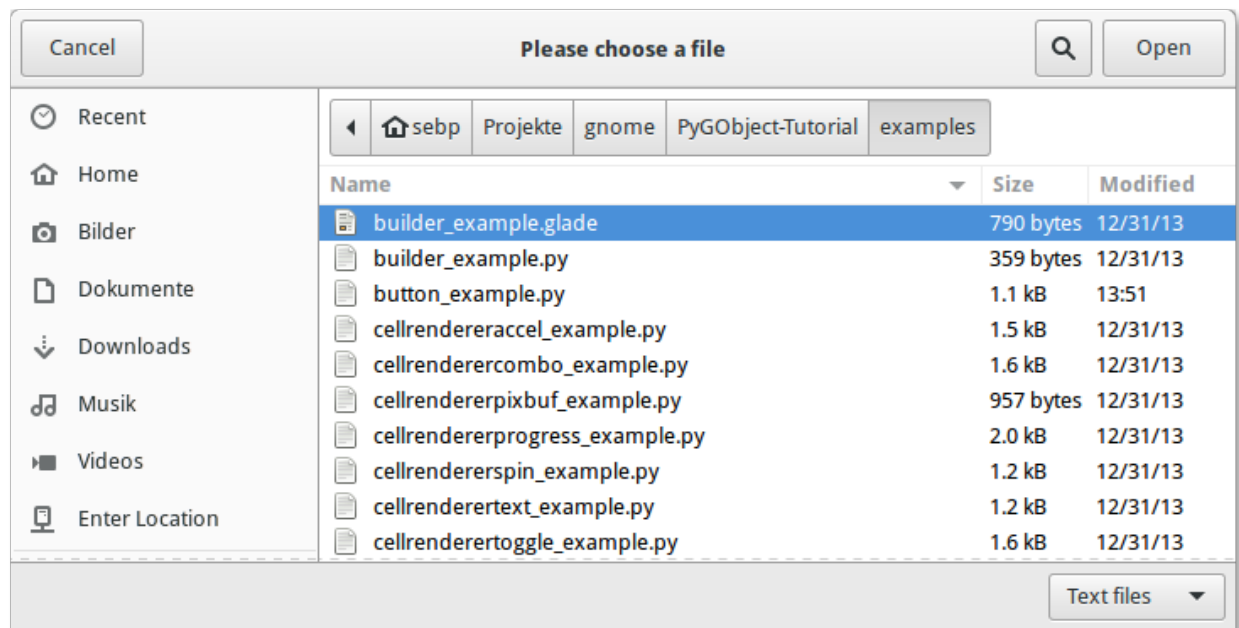
By default, `Gtk.FileChooser` only allows a single file to be selected at a time. To enable multiple files to be selected, use `Gtk.FileChooser.set_select_multiple()`. Retrieving a list of selected files is possible with either `Gtk.FileChooser.get_filenames()` or `Gtk.FileChooser.get_uris()`.

`Gtk.FileChooser` also supports a variety of options which make the files and folders more configurable and accessible.

- `Gtk.FileChooser.set_local_only()`: Only local files can be selected.
- `Gtk.FileChooser.show_hidden()`: Hidden files and folders are displayed.
- `Gtk.FileChooser.set_do_overwrite_confirmation()`: If the file chooser was configured in `Gtk.FileChooserAction.SAVE` mode, it will present a confirmation dialog if the user types a file name that already exists.

Furthermore, you can specify which kind of files are displayed by creating `Gtk.FileFilter` objects and calling `Gtk.FileChooser.add_filter()`. The user can then select one of the added filters from a combo box at the bottom of the file chooser.

18.3.1 Example



```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class FileChooserWindow(Gtk.Window):

```

(continues on next page)

(continued from previous page)

```

8  def __init__(self):
9      super().__init__(title="FileChooser Example")
10
11     box = Gtk.Box(spacing=6)
12     self.add(box)
13
14     button1 = Gtk.Button(label="Choose File")
15     button1.connect("clicked", self.on_file_clicked)
16     box.add(button1)
17
18     button2 = Gtk.Button(label="Choose Folder")
19     button2.connect("clicked", self.on_folder_clicked)
20     box.add(button2)
21
22     def on_file_clicked(self, widget):
23         dialog = Gtk.FileChooserDialog(
24             title="Please choose a file", parent=self, action=Gtk.FileChooserAction.
↪OPEN
25         )
26         dialog.add_buttons(
27             Gtk.STOCK_CANCEL,
28             Gtk.ResponseType.CANCEL,
29             Gtk.STOCK_OPEN,
30             Gtk.ResponseType.OK,
31         )
32
33         self.add_filters(dialog)
34
35         response = dialog.run()
36         if response == Gtk.ResponseType.OK:
37             print("Open clicked")
38             print("File selected: " + dialog.get_filename())
39         elif response == Gtk.ResponseType.CANCEL:
40             print("Cancel clicked")
41
42         dialog.destroy()
43
44     def add_filters(self, dialog):
45         filter_text = Gtk.FileFilter()
46         filter_text.set_name("Text files")
47         filter_text.add_mime_type("text/plain")
48         dialog.add_filter(filter_text)
49
50         filter_py = Gtk.FileFilter()
51         filter_py.set_name("Python files")
52         filter_py.add_mime_type("text/x-python")
53         dialog.add_filter(filter_py)
54
55         filter_any = Gtk.FileFilter()
56         filter_any.set_name("Any files")
57         filter_any.add_pattern("*")
58         dialog.add_filter(filter_any)
59
60     def on_folder_clicked(self, widget):
61         dialog = Gtk.FileChooserDialog(
62             title="Please choose a folder",
63             parent=self,

```

(continues on next page)

(continued from previous page)

```
64         action=Gtk.FileChooserAction.SELECT_FOLDER,
65     )
66     dialog.add_buttons(
67         Gtk.STOCK_CANCEL, Gtk.ResponseType.CANCEL, "Select", Gtk.ResponseType.OK
68     )
69     dialog.set_default_size(800, 400)
70
71     response = dialog.run()
72     if response == Gtk.ResponseType.OK:
73         print("Select clicked")
74         print("Folder selected: " + dialog.get_filename())
75     elif response == Gtk.ResponseType.CANCEL:
76         print("Cancel clicked")
77
78     dialog.destroy()
79
80
81 win = FileChooserWindow()
82 win.connect("destroy", Gtk.main_quit)
83 win.show_all()
84 Gtk.main()
```

POPOVERS

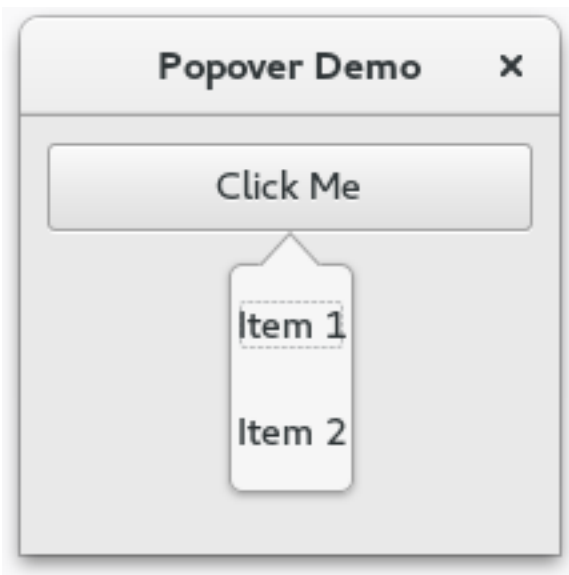
The `Gtk.Popover` is a separate window used for displaying additional information and is often used with button menus and context menus. Popovers are visually connected to a related widget with a small triangle. Their uses are similar to those of dialog windows with the advantage of being less disruptive and having a connection with the widget the popover is pointing to.

A Popover can be created with `Gtk.Popover`; for opening the popover use `Gtk.Popover.popup()`.

19.1 Custom Popover

A widget can be added to a popover using `Gtk.Container.add()`.

19.1.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
```

(continues on next page)

(continued from previous page)

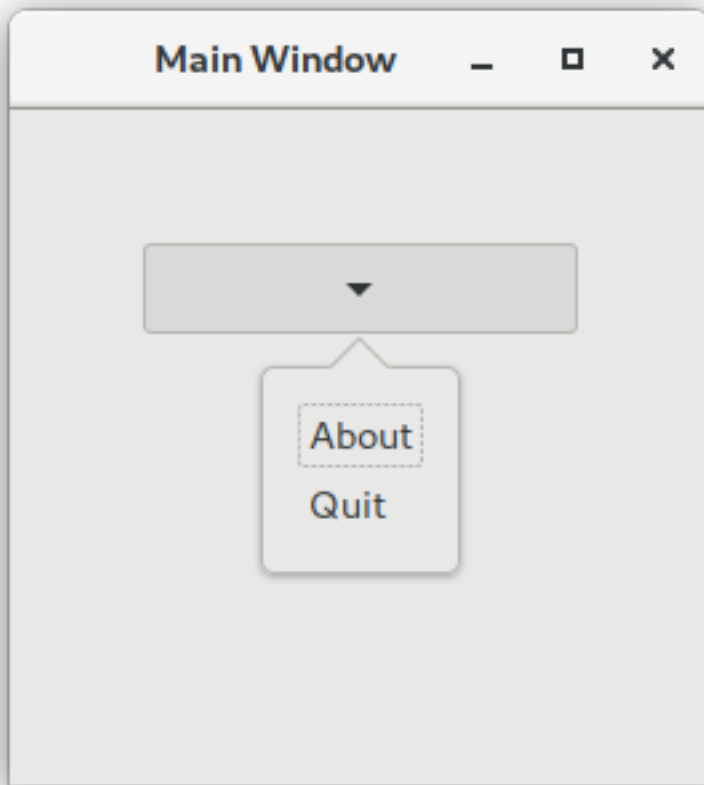
```
7 class PopoverWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="Popover Demo")
10        self.set_border_width(10)
11        self.set_default_size(300, 200)
12
13        outerbox = Gtk.Box(spacing=6, orientation=Gtk.Orientation.VERTICAL)
14        self.add(outerbox)
15
16        self.popover = Gtk.Popover()
17        vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL)
18        vbox.pack_start(Gtk.ModelButton(label="Item 1"), False, True, 10)
19        vbox.pack_start(Gtk.Label(label="Item 2"), False, True, 10)
20        vbox.show_all()
21        self.popover.add(vbox)
22        self.popover.set_position(Gtk.PositionType.BOTTOM)
23
24        button = Gtk.MenuButton(label="Click Me", popover=self.popover)
25        outerbox.pack_start(button, False, True, 0)
26
27
28 win = PopoverWindow()
29 win.connect("destroy", Gtk.main_quit)
30 win.show_all()
31 Gtk.main()
```

19.2 Menu Popover

A popover can be created from a `Gio.MenuModel` using `Gtk.Popover.new_from_model()` and can be changed after creation with `Gtk.Popover.bind_model()`.

Passing a `Gio.MenuModel` as a `menu_model` argument to the `Gtk.MenuButton` constructor implicitly creates a popover.

19.2.1 Example



```
import sys

import gi

gi.require_version("Gtk", "3.0")
from gi.repository import Gio, Gtk

# This would typically be its own file
MENU_XML = """
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <menu id="app-menu">
    <section>
      <item>
        <attribute name="label">About</attribute>
        <attribute name="action">app.about</attribute>
      </item>
      <item>
        <attribute name="label">Quit</attribute>
        <attribute name="action">app.quit</attribute>
      </item>
    </section>
  </menu>
</interface>

```

(continues on next page)

(continued from previous page)

```

"""

class AppWindow(Gtk.ApplicationWindow):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.set_default_size(300, 200)

        outerbox = Gtk.Box(spacing=6, orientation=Gtk.Orientation.VERTICAL)
        self.add(outerbox)
        outerbox.show()

        builder = Gtk.Builder.new_from_string(MENU_XML, -1)
        menu = builder.get_object("app-menu")

        button = Gtk.MenuButton(menu_model=menu)

        outerbox.pack_start(button, False, True, 0)
        button.show()
        self.set_border_width(50)

class Application(Gtk.Application):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, application_id="org.example.myapp", **kwargs)
        self.window = None

    def do_startup(self):
        Gtk.Application.do_startup(self)

        action = Gio.SimpleAction(name="about")
        action.connect("activate", self.on_about)
        self.add_action(action)

        action = Gio.SimpleAction(name="quit")
        action.connect("activate", self.on_quit)
        self.add_action(action)

    def do_activate(self):
        # We only allow a single window and raise any existing ones
        if not self.window:
            # Windows are associated with the application
            # when the last one is closed the application shuts down
            self.window = AppWindow(application=self, title="Main Window")

        self.window.present()

    def on_about(self, action, param):
        about_dialog = Gtk.AboutDialog(transient_for=self.window, modal=True)
        about_dialog.present()

    def on_quit(self, action, param):
        self.quit()

if __name__ == "__main__":
    app = Application()

```

(continues on next page)

(continued from previous page)

```
app.run(sys.argv)
```

19.3 See Also

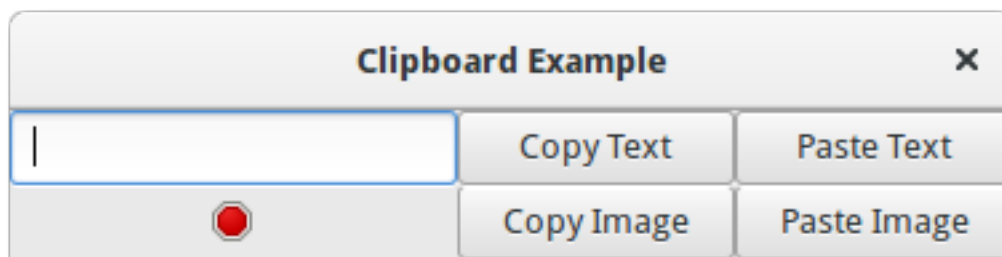
- <https://developer.gnome.org/hig/patterns/containers/popovers.html>

CLIPBOARD

`Gtk.Clipboard` provides a storage area for a variety of data, including text and images. Using a clipboard allows this data to be shared between applications through actions such as copying, cutting, and pasting. These actions are usually done in three ways: using keyboard shortcuts, using a `Gtk.MenuItem`, and connecting the functions to `Gtk.Button` widgets.

There are multiple clipboard selections for different purposes. In most circumstances, the selection named `CLIPBOARD` is used for everyday copying and pasting. `PRIMARY` is another common selection which stores text selected by the user with the cursor.

20.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk, Gdk
5
6
7 class ClipboardWindow(Gtk.Window):
8     def __init__(self):
9         super().__init__(title="Clipboard Example")
10
11         grid = Gtk.Grid()
12
13         self.clipboard = Gtk.Clipboard.get(Gdk.SELECTION_CLIPBOARD)
14         self.entry = Gtk.Entry()
15         self.image = Gtk.Image.new_from_icon_name("process-stop", Gtk.IconSize.MENU)
16
17         button_copy_text = Gtk.Button(label="Copy Text")
18         button_paste_text = Gtk.Button(label="Paste Text")
19         button_copy_image = Gtk.Button(label="Copy Image")
```

(continues on next page)

(continued from previous page)

```

20     button_paste_image = Gtk.Button(label="Paste Image")
21
22     grid.add(self.entry)
23     grid.attach(self.image, 0, 1, 1, 1)
24     grid.attach(button_copy_text, 1, 0, 1, 1)
25     grid.attach(button_paste_text, 2, 0, 1, 1)
26     grid.attach(button_copy_image, 1, 1, 1, 1)
27     grid.attach(button_paste_image, 2, 1, 1, 1)
28
29     button_copy_text.connect("clicked", self.copy_text)
30     button_paste_text.connect("clicked", self.paste_text)
31     button_copy_image.connect("clicked", self.copy_image)
32     button_paste_image.connect("clicked", self.paste_image)
33
34     self.add(grid)
35
36     def copy_text(self, widget):
37         self.clipboard.set_text(self.entry.get_text(), -1)
38
39     def paste_text(self, widget):
40         text = self.clipboard.wait_for_text()
41         if text is not None:
42             self.entry.set_text(text)
43         else:
44             print("No text on the clipboard.")
45
46     def copy_image(self, widget):
47         if self.image.get_storage_type() == Gtk.ImageType.PIXBUF:
48             self.clipboard.set_image(self.image.get_pixbuf())
49         else:
50             print("No image has been pasted yet.")
51
52     def paste_image(self, widget):
53         image = self.clipboard.wait_for_image()
54         if image is not None:
55             self.image.set_from_pixbuf(image)
56
57
58 win = ClipboardWindow()
59 win.connect("destroy", Gtk.main_quit)
60 win.show_all()
61 Gtk.main()

```

DRAG AND DROP

Note: Versions of PyGObject < 3.0.3 contain a bug which does not allow drag and drop to function correctly. Therefore a version of PyGObject >= 3.0.3 is required for the following examples to work.

Setting up drag and drop between widgets consists of selecting a drag source (the widget which the user starts the drag from) with the `Gtk.Widget.drag_source_set()` method, selecting a drag destination (the widget which the user drops onto) with the `Gtk.Widget.drag_dest_set()` method and then handling the relevant signals on both widgets.

Instead of using `Gtk.Widget.drag_source_set()` and `Gtk.Widget.drag_dest_set()` some specialised widgets require the use of specific functions (such as `Gtk.TreeView` and `Gtk.IconView`).

A basic drag and drop only requires the source to connect to the “drag-data-get” signal and the destination to connect to the “drag-data-received” signal. More complex things such as specific drop areas and custom drag icons will require you to connect to *additional signals* and interact with the `Gdk.DragContext` object it supplies.

In order to transfer data between the source and destination, you must interact with the `Gtk.SelectionData` variable supplied in the “drag-data-get” and “drag-data-received” signals using the `Gtk.SelectionData` `get` and `set` methods.

21.1 Target Entries

To allow the drag source and destination to know what data they are receiving and sending, a common list of `Gtk.TargetEntry`'s are required. A `Gtk.TargetEntry` describes a piece of data that will be sent by the drag source and received by the drag destination.

There are two ways of adding `Gtk.TargetEntry`'s to a source and destination. If the drag and drop is simple and each target entry is of a different type, you can use the group of methods [mentioned here](#).

If you require more than one type of data or wish to do more complex things with the data, you will need to create the `Gtk.TargetEntry`'s using the `Gtk.TargetEntry.new()` method.

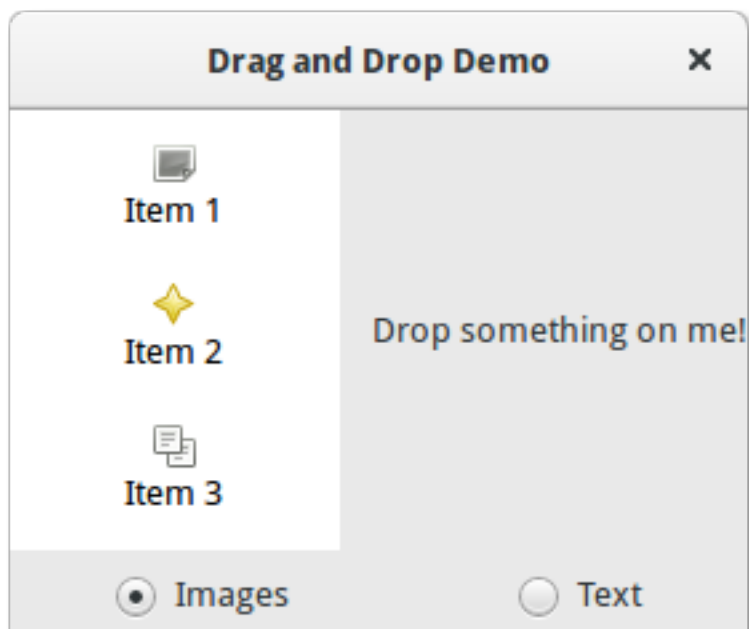
21.2 Drag Source Signals

Name	When it is emitted	Common Purpose
drag-begin	User starts a drag	Set-up drag icon
drag-data-get	When drag data is requested by the destination	Transfer drag data from source to destination
drag-data-delete	When a drag with the action <code>Gdk.DragAction.MOVE</code> is completed	Delete data from the source to complete the 'move'
drag-end	When the drag is complete	Undo anything done in drag-begin

21.3 Drag Destination Signals

Name	When it is emitted	Common Purpose
drag-motion	Drag icon moves over a drop area	Allow only certain areas to be dropped onto
drag-drop	Icon is dropped onto a drag area	Allow only certain areas to be dropped onto
drag-data-received	When drag data is received by the destination	Transfer drag data from source to destination

21.4 Example



```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk, Gdk, GdkPixbuf
5
6 (TARGET_ENTRY_TEXT, TARGET_ENTRY_PIXBUF) = range(2)

```

(continues on next page)

(continued from previous page)

```

7 (COLUMN_TEXT, COLUMN_PIXBUF) = range(2)
8
9 DRAG_ACTION = Gdk.DragAction.COPY
10
11
12 class DragDropWindow(Gtk.Window):
13     def __init__(self):
14         super().__init__(title="Drag and Drop Demo")
15
16         vbox = Gtk.Box(orientation=Gtk.Orientation.VERTICAL, spacing=6)
17         self.add(vbox)
18
19         hbox = Gtk.Box(spacing=12)
20         vbox.pack_start(hbox, True, True, 0)
21
22         self.iconview = DragSourceIconView()
23         self.drop_area = DropArea()
24
25         hbox.pack_start(self.iconview, True, True, 0)
26         hbox.pack_start(self.drop_area, True, True, 0)
27
28         button_box = Gtk.Box(spacing=6)
29         vbox.pack_start(button_box, True, False, 0)
30
31         image_button = Gtk.RadioButton.new_with_label_from_widget(None, "Images")
32         image_button.connect("toggled", self.add_image_targets)
33         button_box.pack_start(image_button, True, False, 0)
34
35         text_button = Gtk.RadioButton.new_with_label_from_widget(image_button, "Text")
36         text_button.connect("toggled", self.add_text_targets)
37         button_box.pack_start(text_button, True, False, 0)
38
39         self.add_image_targets()
40
41     def add_image_targets(self, button=None):
42         targets = Gtk.TargetList.new([])
43         targets.add_image_targets(TARGET_ENTRY_PIXBUF, True)
44
45         self.drop_area.drag_dest_set_target_list(targets)
46         self.iconview.drag_source_set_target_list(targets)
47
48     def add_text_targets(self, button=None):
49         self.drop_area.drag_dest_set_target_list(None)
50         self.iconview.drag_source_set_target_list(None)
51
52         self.drop_area.drag_dest_add_text_targets()
53         self.iconview.drag_source_add_text_targets()
54
55
56 class DragSourceIconView(Gtk.IconView):
57     def __init__(self):
58         Gtk.IconView.__init__(self)
59         self.set_text_column(COLUMN_TEXT)
60         self.set_pixbuf_column(COLUMN_PIXBUF)
61
62         model = Gtk.ListStore(str, GdkPixbuf.Pixbuf)
63         self.set_model(model)

```

(continues on next page)

(continued from previous page)

```

64     self.add_item("Item 1", "image-missing")
65     self.add_item("Item 2", "help-about")
66     self.add_item("Item 3", "edit-copy")
67
68     self.enable_model_drag_source(Gdk.ModifierType.BUTTON1_MASK, [], DRAG_ACTION)
69     self.connect("drag-data-get", self.on_drag_data_get)
70
71     def on_drag_data_get(self, widget, drag_context, data, info, time):
72         selected_path = self.get_selected_items()[0]
73         selected_iter = self.get_model().get_iter(selected_path)
74
75         if info == TARGET_ENTRY_TEXT:
76             text = self.get_model().get_value(selected_iter, COLUMN_TEXT)
77             data.set_text(text, -1)
78         elif info == TARGET_ENTRY_PIXBUF:
79             pixbuf = self.get_model().get_value(selected_iter, COLUMN_PIXBUF)
80             data.set_pixbuf(pixbuf)
81
82     def add_item(self, text, icon_name):
83         pixbuf = Gtk.IconTheme.get_default().load_icon(icon_name, 16, 0)
84         self.get_model().append([text, pixbuf])
85
86
87 class DropArea(Gtk.Label):
88     def __init__(self):
89         Gtk.Label.__init__(self)
90         self.set_label("Drop something on me!")
91         self.drag_dest_set(Gtk.DestDefaults.ALL, [], DRAG_ACTION)
92
93         self.connect("drag-data-received", self.on_drag_data_received)
94
95     def on_drag_data_received(self, widget, drag_context, x, y, data, info, time):
96         if info == TARGET_ENTRY_TEXT:
97             text = data.get_text()
98             print("Received text: %s" % text)
99
100         elif info == TARGET_ENTRY_PIXBUF:
101             pixbuf = data.get_pixbuf()
102             width = pixbuf.get_width()
103             height = pixbuf.get_height()
104
105             print("Received pixbuf with width %spx and height %spx" % (width, height))
106
107
108 win = DragDropWindow()
109 win.connect("destroy", Gtk.main_quit)
110 win.show_all()
111 Gtk.main()

```


GLADE AND GTK.BUILDER

The `Gtk.Builder` class offers you the opportunity to design user interfaces without writing a single line of code. This is achieved by defining the interface in an XML file and then loading that XML UI definition at runtime using the Builder class which creates the objects automatically. To avoid writing the XML manually use the `Glade` application which lets you create the user interface in a WYSIWYG (what you see is what you get) manner

This method has several advantages:

- Less code needs to be written.
- UI changes can be seen more quickly, so UIs are able to improve.
- Designers without programming skills can create and edit UIs.
- The description of the user interface is independent from the programming language being used.

There is still code required for handling interface changes triggered by the user, but `Gtk.Builder` allows you to focus on implementing that functionality.

22.1 Creating and loading the .glade file

First of all you have to download and install Glade. There are [several tutorials](#) about Glade, so this is not explained here in detail. Let's start by creating a window with a button in it and saving it to a file named *example.glade*. The resulting XML file should look like this.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <!-- interface-requires gtk+ 3.0 -->
  <object class="GtkWindow" id="window1">
    <property name="can_focus">False</property>
    <child>
      <object class="GtkButton" id="button1">
        <property name="label" translatable="yes">button</property>
        <property name="use-action-appearance">False</property>
        <property name="visible">True</property>
        <property name="can-focus">True</property>
        <property name="receives-default">True</property>
      </object>
    </child>
  </object>
</interface>
```

To load this file in Python we need a `Gtk.Builder` object.

```
builder = Gtk.Builder()
builder.add_from_file("example.glade")
```

The second line loads all objects defined in *example.glade* into the Builder object.

It is also possible to load only some of the objects. The following line would add only the objects (and their child objects) given in the tuple.

```
# we don't really have two buttons here, this is just an example
builder.add_objects_from_file("example.glade", ("button1", "button2"))
```

These two methods exist also for loading from a string rather than a file. Their corresponding names are `Gtk.Builder.add_from_string()` and `Gtk.Builder.add_objects_from_string()` and they simply take a XML string instead of a file name.

22.2 Accessing widgets

Now that the window and the button are loaded we also want to show them. Therefore the `Gtk.Window.show_all()` method has to be called on the window. But how do we access the associated object?

```
window = builder.get_object("window1")
window.show_all()
```

Every widget can be retrieved from the builder by the `Gtk.Builder.get_object()` method and the widget's *id*. It is really *that* simple.

It is also possible to get a list of all objects with

```
builder.get_objects()
```

22.3 Connecting Signals

Glade also makes it possible to define signals which you can connect to handlers in your code without extracting every object from the builder and connecting to the signals manually. The first thing to do is to declare the signal names in Glade. For this example we will act when the window is closed and when the button was pressed, so we give the name “onDestroy” to the callback handling the “destroy” signal of the window and “onButtonPressed” to the callback handling the “pressed” signal of the button. Now the XML file should look like this.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <!-- interface-requires gtk+ 3.0 -->
  <object class="GtkWindow" id="window1">
    <property name="can-focus">False</property>
    <signal name="destroy" handler="onDestroy" swapped="no"/>
    <child>
      <object class="GtkButton" id="button1">
        <property name="label" translatable="yes">button</property>
        <property name="use-action-appearance">False</property>
        <property name="visible">True</property>
        <property name="can-focus">True</property>
        <property name="receives-default">True</property>
        <property name="use-action-appearance">False</property>
```

(continues on next page)

(continued from previous page)

```

        <signal name="pressed" handler="onButtonPressed" swapped="no"/>
    </object>
</child>
</object>
</interface>

```

Now we have to define the handler functions in our code. The *onDestroy* should simply result in a call to `Gtk.main_quit()`. When the button is pressed we would like to print the string “Hello World!”, so we define the handler as follows

```

def hello(button):
    print("Hello World!")

```

Next, we have to connect the signals and the handler functions. The easiest way to do this is to define a *dict* with a mapping from the names to the handlers and then pass it to the `Gtk.Builder.connect_signals()` method.

```

handlers = {
    "onDestroy": Gtk.main_quit,
    "onButtonPressed": hello
}
builder.connect_signals(handlers)

```

An alternative approach is to create a class which has methods that are called like the signals. In our example the last code snippet could be rewritten as:

```

1 from gi.repository import Gtk
2
3
4 class Handler:
5     def onDestroy(self, *args):
6         Gtk.main_quit()
7
8     def onButtonPressed(self, button):
9         print("Hello World!")
10 builder.connect_signals(Handler())

```

22.4 Example

The final code of the example

```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class Handler:
8     def onDestroy(self, *args):
9         Gtk.main_quit()
10
11     def onButtonPressed(self, button):
12         print("Hello World!")
13
14

```

(continues on next page)

(continued from previous page)

```

15 builder = Gtk.Builder()
16 builder.add_from_file("builder_example.glade")
17 builder.connect_signals(Handler())
18
19 window = builder.get_object("window1")
20 window.show_all()
21
22 Gtk.main()

```

22.5 Gtk.Template

`Gtk.WidgetClass` allows UI definition files to be used to extend a widget, `PyGObject` provides `Gtk.Template` as a way of accessing this from Python.

The UI definition file used in the example needs a small change to include a `<template>` element:

```

<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <!-- interface-requires gtk+ 3.0 -->
  <template class="window1" parent="GtkWindow">
    <signal name="destroy" handler="onDestroy" swapped="no"/>
    <child>
      <object class="GtkButton" id="button1">
        <property name="label" translatable="yes">button</property>
        <property name="use-action-appearance">False</property>
        <property name="visible">True</property>
        <property name="can-focus">True</property>
        <property name="receives-default">True</property>
        <property name="use-action-appearance">False</property>
        <signal name="pressed" handler="onButtonPressed" swapped="no"/>
      </object>
    </child>
  </template>
</interface>

```

Then it can be used to implement the example with a `Gtk.Window` subclass:

```

1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 @Gtk.Template(filename="template_example.ui")
8 class Window1(Gtk.Window):
9     __gtype_name__ = "window1"
10
11     @Gtk.Template.Callback()
12     def onDestroy(self, *args):
13         Gtk.main_quit()
14
15     @Gtk.Template.Callback()
16     def onButtonPressed(self, button):
17         print("Hello World!")

```

(continues on next page)

(continued from previous page)

```
18
19
20 window = Window1()
21 window.show()
22
23 Gtk.main()
```

More information can be found at the [PyGObject](#) website.

OBJECTS

GObject is the fundamental type providing the common attributes and methods for all object types in GTK+, Pango and other libraries based on GObject. The `GObject.GObject` class provides methods for object construction and destruction, property access methods, and signal support.

This section will introduce some important aspects about the GObject implementation in Python.

23.1 Inherit from GObject.GObject

A native GObject is accessible via `GObject.GObject`. It is rarely instantiated directly, we generally use inherited class. A `Gtk.Widget` is an inherited class of a `GObject.GObject`. It may be interesting to make an inherited class to create a new widget, like a settings dialog.

To inherit from `GObject.GObject`, you must call `GObject.GObject.__init__()` in your constructor (if the class inherits from `Gtk.Button`, it must call `Gtk.Button.__init__()` for instance), like in the example below:

```
from gi.repository import GObject

class MyObject(GObject.GObject):

    def __init__(self):
        GObject.GObject.__init__(self)
```

23.2 Signals

Signals connect arbitrary application-specific events with any number of listeners. For example, in GTK+, every user event (keystroke or mouse move) is received from the X server and generates a GTK+ event under the form of a signal emission on a given object instance.

Each signal is registered in the type system together with the type on which it can be emitted: users of the type are said to connect to the signal on a given type instance when they register a function to be invoked upon the signal emission. Users can also emit the signal by themselves or stop the emission of the signal from within one of the functions connected to the signal.

23.2.1 Receive signals

See *Main loop and Signals*

23.2.2 Create new signals

New signals can be created by adding them to `GObject.GObject.__gsignals__`, a dictionary:

When a new signal is created, a method handler can also be defined, it will be called each time the signal is emitted. It is called `do_signal_name`.

```
class MyObject(GObject.GObject):
    __gsignals__ = {
        'my_signal': (GObject.SIGNAL_RUN_FIRST, None,
                      (int,))
    }

    def do_my_signal(self, arg):
        print("method handler for `my_signal' called with argument", arg)
```

`GObject.SIGNAL_RUN_FIRST` indicates that this signal will invoke the object method handler (`do_my_signal()` here) in the first emission stage. Alternatives are `GObject.SIGNAL_RUN_LAST` (the method handler will be invoked in the third emission stage) and `GObject.SIGNAL_RUN_CLEANUP` (invoke the method handler in the last emission stage).

The second part, `None`, indicates the return type of the signal, usually `None`.

`(int,)` indicates the signal arguments, here, the signal will only take one argument, whose type is `int`. Types of arguments required by signal are declared as a sequence, here it is a one-element tuple.

Signals can be emitted using `GObject.GObject.emit()`:

```
my_obj.emit("my_signal", 42) # emit the signal "my_signal", with the
                             # argument 42
```

23.3 Properties

One of `GObject`'s nice features is its generic get/set mechanism for object properties. Each class inherited from `GObject.GObject` can define new properties. Each property has a type which never changes (e.g. `str`, `float`, `int`...). For instance, they are used for `Gtk.Button` where there is a “label” property which contains the text of the button.

23.3.1 Use existing properties

The class `GObject.GObject` provides several useful functions to manage existing properties, `GObject.GObject.get_property()` and `GObject.GObject.set_property()`.

Some properties also have functions dedicated to them, called getter and setter. For the property “label” of a button, there are two functions to get and set them, `Gtk.Button.get_label()` and `Gtk.Button.set_label()`.

23.3.2 Create new properties

A property is defined with a name and a type. Even if Python itself is dynamically typed, you can't change the type of a property once it is defined. A property can be created using `GObject.Property`.

```
from gi.repository import GObject

class MyObject(GObject.GObject):

    foo = GObject.Property(type=str, default='bar')
    property_float = GObject.Property(type=float)
    def __init__(self):
        GObject.GObject.__init__(self)
```

Properties can also be read-only, if you want some properties to be readable but not writable. To do so, you can add some flags to the property definition, to control read/write access. Flags are `GObject.ParamFlags.READABLE` (only read access for external code), `GObject.ParamFlags.WRITABLE` (only write access), `GObject.ParamFlags.READWRITE` (public):

```
foo = GObject.Property(type=str, flags = GObject.ParamFlags.READABLE) # not writable
bar = GObject.Property(type=str, flags = GObject.ParamFlags.WRITABLE) # not readable
```

You can also define new read-only properties with a new method decorated with `GObject.Property`:

```
from gi.repository import GObject

class MyObject(GObject.GObject):

    def __init__(self):
        GObject.GObject.__init__(self)

    @GObject.Property
    def readonly(self):
        return 'This is read-only.'
```

You can get this property using:

```
my_object = MyObject()
print(my_object.readonly)
print(my_object.get_property("readonly"))
```

The API of `GObject.Property` is similar to the builtin `property()`. You can create property setter in a way similar to Python property:

```
class AnotherObject(GObject.Object):
    value = 0

    @GObject.Property
    def prop(self):
        'Read only property.'
        return 1

    @GObject.Property(type=int)
    def propInt(self):
        'Read-write integer property.'
        return self.value
```

(continues on next page)

(continued from previous page)

```
@propInt.setter
def propInt(self, value):
    self.value = value
```

There is also a way to define minimum and maximum values for numbers, using a more verbose form:

```
from gi.repository import GObject

class MyObject(GObject.GObject):

    __gproperties__ = {
        "int-prop": (int, # type
                     "integer prop", # nick
                     "A property that contains an integer", # blurb
                     1, # min
                     5, # max
                     2, # default
                     GObject.ParamFlags.READWRITE # flags
                     ),
    }

    def __init__(self):
        GObject.GObject.__init__(self)
        self.int_prop = 2

    def do_get_property(self, prop):
        if prop.name == 'int-prop':
            return self.int_prop
        else:
            raise AttributeError('unknown property %s' % prop.name)

    def do_set_property(self, prop, value):
        if prop.name == 'int-prop':
            self.int_prop = value
        else:
            raise AttributeError('unknown property %s' % prop.name)
```

Properties must be defined in `GObject.GObject.__gproperties__`, a dictionary, and handled in `do_get_property` and `do_set_property`.

23.3.3 Watch properties

When a property is modified, a signal is emitted, whose name is “notify::property-name”:

```
my_object = MyObject()

def on_notify_foo(obj, gparamstring):
    print("foo changed")

my_object.connect("notify::foo", on_notify_foo)

my_object.set_property("foo", "bar") # on_notify_foo will be called
```

Note that you have to use the canonical property name when connecting to the notify signals, as explained in `GObject.Object.signals.notify()`. For instance, for a Python property `foo_bar_baz` you would connect to the signal `notify::foo-bar-baz` using

```

my_object = MyObject()

def on_notify_foo_bar_baz(obj, gparamstring):
    print("foo_bar_baz changed")

my_object.connect("notify::foo-bar-baz", on_notify_foo_bar_baz)

```

23.4 API

class GObject.GObject

get_property (*property_name*)

Retrieves a property value.

set_property (*property_name*, *value*)

Set property *property_name* to *value*.

emit (*signal_name*, ...)

Emit signal *signal_name*. Signal arguments must follow, e.g. if your signal is of type `(int,)`, it must be emitted with:

```
self.emit(signal_name, 42)
```

freeze_notify ()

This method freezes all the “notify::” signals (which are emitted when any property is changed) until the *thaw_notify()* method is called.

It is recommended to use the *with* statement when calling *freeze_notify()*, that way it is ensured that *thaw_notify()* is called implicitly at the end of the block:

```

with an_object.freeze_notify():
    # Do your work here
    ...

```

thaw_notify ()

Thaw all the “notify::” signals which were frozen by *freeze_notify()*.

It is recommended to not call *thaw_notify()* explicitly but use *freeze_notify()* together with the *with* statement.

handler_block (*handler_id*)

Blocks a handler of an instance so it will not be called during any signal emissions unless *handler_unblock()* is called for that *handler_id*. Thus “blocking” a signal handler means to temporarily deactivate it, a signal handler has to be unblocked exactly the same amount of times it has been blocked before to become active again.

It is recommended to use *handler_block()* in conjunction with the *with* statement which will call *handler_unblock()* implicitly at the end of the block:

```

with an_object.handler_block(handler_id):
    # Do your work here
    ...

```

handler_unblock (*handler_id*)

Undoes the effect of *handler_block()*. A blocked handler is skipped during signal emissions and will not be invoked until it has been unblocked exactly the amount of times it has been blocked before.

It is recommended to not call `handler_unblock()` explicitly but use `handler_block()` together with the `with` statement.

__gsignals__

A dictionary where inherited class can define new signals.

Each element in the dictionary is a new signal. The key is the signal name. The value is a tuple, with the form:

```
(GObject.SIGNAL_RUN_FIRST, None, (int,))
```

`GObject.SIGNAL_RUN_FIRST` can be replaced with `GObject.SIGNAL_RUN_LAST` or `GObject.SIGNAL_RUN_CLEANUP`. `None` is the return type of the signal. `(int,)` is the tuple of the parameters of the signal.

__gproperties__

The `__gproperties__` dictionary is a class property where you define the properties of your object. This is not the recommended way to define new properties, the method written above is much less verbose. The benefits of this method is that a property can be defined with more settings, like the minimum or the maximum for numbers.

The key is the name of the property

The value is a tuple which describe the property. The number of elements of this tuple depends on its first element but the tuple will always contain at least the following items:

- The first element is the property's type (e.g. `int`, `float`...).

- The second element is the property's nick name, which is a string with a short description of the property. This is generally used by programs with strong introspection capabilities, like the graphical user interface builder `Glade`.

- The third one is the property's description or blurb, which is another string with a longer description of the property. Also used by `Glade` and similar programs.

- The last one (which is not necessarily the forth one as we will see later) is the property's flags: `GObject.PARAM_READABLE`, `GObject.PARAM_WRITABLE`, `GObject.PARAM_READWRITE`.

The absolute length of the tuple depends on the property type (the first element of the tuple). Thus we have the following situations:

- If the type is `bool` or `str`, the forth element is the default value of the property.

- If the type is `int` or `float`, the forth element is the minimum accepted value, the fifth element is the maximum accepted value and the sixth element is the default value.

- If the type is not one of these, there is no extra element.

`GObject.SIGNAL_RUN_FIRST`

Invoke the object method handler in the first emission stage.

`GObject.SIGNAL_RUN_LAST`

Invoke the object method handler in the third emission stage.

`GObject.SIGNAL_RUN_CLEANUP`

Invoke the object method handler in the last emission stage.

`GObject.ParamFlags.READABLE`

The property is readable.

`GObject.ParamFlags.WRITABLE`

The property is writable.

`GObject.ParamFlags.READWRITE`
The property is readable and writable.

APPLICATION

`Gtk.Application` encompasses many repetitive tasks that a modern application needs such as handling multiple instances, D-Bus activation, opening files, command line parsing, startup/shutdown, menu management, window management, and more.

24.1 Actions

`Gio.Action` is a way to expose any single task your application or widget does by a name. These actions can be disabled/enabled at runtime and they can either be activated or have a state changed (if they contain state).

The reason to use actions is to separate out the logic from the UI. For example this allows using a menubar on OSX and a gear menu on GNOME both simply referencing the name of an action. The main implementation of this you will be using is `Gio.SimpleAction` which will be demonstrated later.

Many classes such as `Gio.MenuItem` and `Gtk.ModelButton` support properties to set an action name.

These actions can be grouped together into a `Gio.ActionGroup` and when these groups are added to a widget with `Gtk.Widget.insert_action_group()` they will gain a prefix. Such as “win” when added to a `Gtk.ApplicationWindow`. You will use the full action name when referencing it such as “app.about” but when you create the action it will just be “about” until added to the application.

You can also very easily make keybindings for actions by setting the *accel* property in the `Gio.Menu` file or by using `Gtk.Application.set_accels_for_action()`.

24.2 Menus

Your menus should be defined in XML using `Gio.Menu` and would reference the previously mentioned actions you defined. `Gtk.Application` allows you to set a menu either via `Gtk.Application.set_app_menu()` or `Gtk.Application.set_menubar()`. If you make use of `Gio.Resource` this can automatically use the correct menu based on platform, otherwise you can set them manually. A detailed example is shown below.

24.3 Command Line

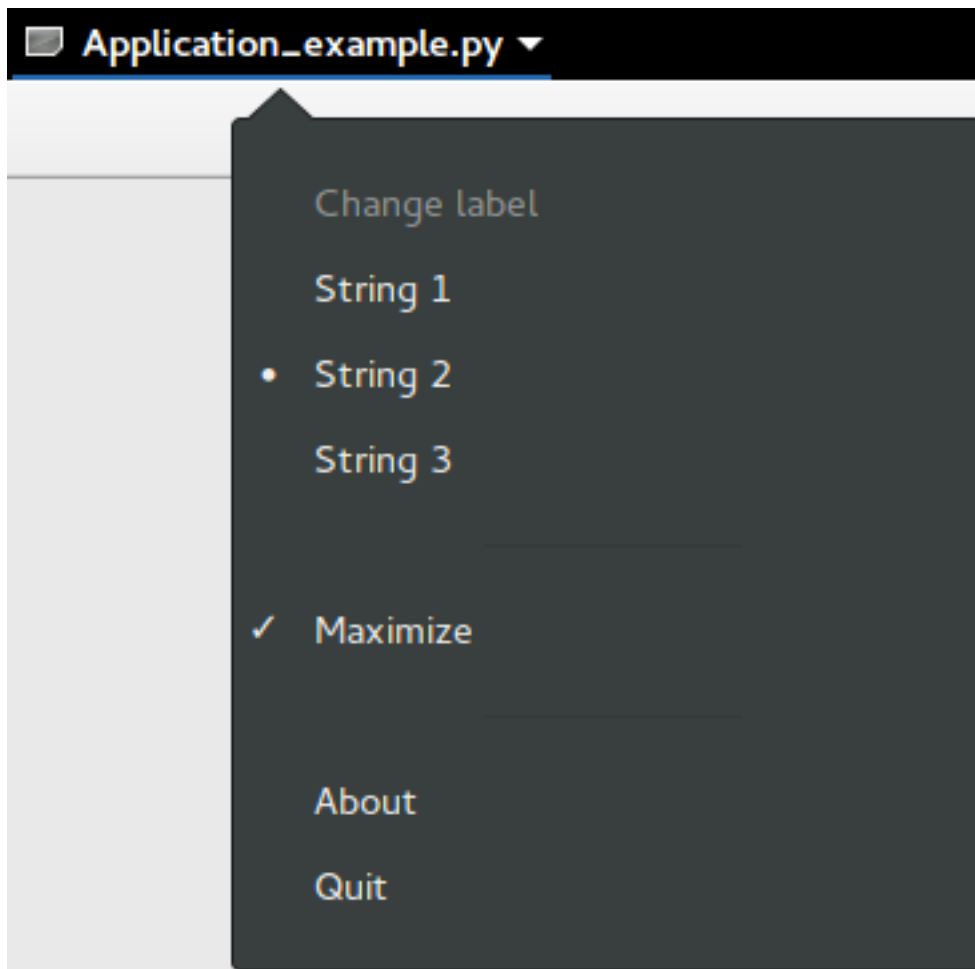
When creating your application it takes a flag property of `Gio.ApplicationFlags`. Using this you can let it handle everything itself or have more custom behavior.

You can use `HANDLES_COMMAND_LINE` to allow custom behavior in `Gio.Application.do_command_line()`. In combination with `Gio.Application.add_main_option()` to add custom options.

Using `HANDLES_OPEN` will do the work of simply taking file arguments for you and let you handle it in `Gio.Application.do_open()`.

If your application is already open these will all be sent to the existing instance unless you use `NON_UNIQUE` to allow multiple instances.

24.4 Example



```
1 import sys
2
3 import gi
4
```

(continues on next page)

(continued from previous page)

```

5 gi.require_version("Gtk", "3.0")
6 from gi.repository import GLib, Gio, Gtk
7
8 # This would typically be its own file
9 MENU_XML = """
10 <?xml version="1.0" encoding="UTF-8"?>
11 <interface>
12     <menu id="app-menu">
13         <section>
14             <attribute name="label" translatable="yes">Change label</attribute>
15             <item>
16                 <attribute name="action">win.change_label</attribute>
17                 <attribute name="target">String 1</attribute>
18                 <attribute name="label" translatable="yes">String 1</attribute>
19             </item>
20             <item>
21                 <attribute name="action">win.change_label</attribute>
22                 <attribute name="target">String 2</attribute>
23                 <attribute name="label" translatable="yes">String 2</attribute>
24             </item>
25             <item>
26                 <attribute name="action">win.change_label</attribute>
27                 <attribute name="target">String 3</attribute>
28                 <attribute name="label" translatable="yes">String 3</attribute>
29             </item>
30         </section>
31         <section>
32             <item>
33                 <attribute name="action">win.maximize</attribute>
34                 <attribute name="label" translatable="yes">Maximize</attribute>
35             </item>
36         </section>
37         <section>
38             <item>
39                 <attribute name="action">app.about</attribute>
40                 <attribute name="label" translatable="yes">_About</attribute>
41             </item>
42             <item>
43                 <attribute name="action">app.quit</attribute>
44                 <attribute name="label" translatable="yes">_Quit</attribute>
45                 <attribute name="accel">&lt;Primary&gt;q</attribute>
46             </item>
47         </section>
48     </menu>
49 </interface>
50 """
51
52
53 class AppWindow(Gtk.ApplicationWindow):
54     def __init__(self, *args, **kwargs):
55         super().__init__(*args, **kwargs)
56
57         # This will be in the windows group and have the "win" prefix
58         max_action = Gio.SimpleAction.new_stateful(
59             "maximize", None, GLib.Variant.new_boolean(False)
60         )
61         max_action.connect("change-state", self.on_maximize_toggle)

```

(continues on next page)

(continued from previous page)

```

62     self.add_action(max_action)
63
64     # Keep it in sync with the actual state
65     self.connect(
66         "notify::is-maximized",
67         lambda obj, pspec: max_action.set_state(
68             GLib.Variant.new_boolean(obj.props.is_maximized)
69         ),
70     )
71
72     lbl_variant = GLib.Variant.new_string("String 1")
73     lbl_action = Gio.SimpleAction.new_stateful(
74         "change_label", lbl_variant.get_type(), lbl_variant
75     )
76     lbl_action.connect("change-state", self.on_change_label_state)
77     self.add_action(lbl_action)
78
79     self.label = Gtk.Label(label=lbl_variant.get_string(), margin=30)
80     self.add(self.label)
81     self.label.show()
82
83     def on_change_label_state(self, action, value):
84         action.set_state(value)
85         self.label.set_text(value.get_string())
86
87     def on_maximize_toggle(self, action, value):
88         action.set_state(value)
89         if value.get_boolean():
90             self.maximize()
91         else:
92             self.unmaximize()
93
94
95 class Application(Gtk.Application):
96     def __init__(self, *args, **kwargs):
97         super().__init__(
98             *args,
99             application_id="org.example.myapp",
100             flags=Gio.ApplicationFlags.HandlesCommandLine,
101             **kwargs
102         )
103         self.window = None
104
105         self.add_main_option(
106             "test",
107             ord("t"),
108             GLib.OptionFlags.NONE,
109             GLib.OptionArg.NONE,
110             "Command line test",
111             None,
112         )
113
114     def do_startup(self):
115         Gtk.Application.do_startup(self)
116
117         action = Gio.SimpleAction.new("about", None)
118         action.connect("activate", self.on_about)

```

(continues on next page)

(continued from previous page)

```

119     self.add_action(action)
120
121     action = Gio.SimpleAction.new("quit", None)
122     action.connect("activate", self.on_quit)
123     self.add_action(action)
124
125     builder = Gtk.Builder.new_from_string(MENU_XML, -1)
126     self.set_app_menu(builder.get_object("app-menu"))
127
128     def do_activate(self):
129         # We only allow a single window and raise any existing ones
130         if not self.window:
131             # Windows are associated with the application
132             # when the last one is closed the application shuts down
133             self.window = AppWindow(application=self, title="Main Window")
134
135         self.window.present()
136
137     def do_command_line(self, command_line):
138         options = command_line.get_options_dict()
139         # convert GVariantDict -> GVariant -> dict
140         options = options.end().unpack()
141
142         if "test" in options:
143             # This is printed on the main instance
144             print("Test argument recieved: %s" % options["test"])
145
146         self.activate()
147         return 0
148
149     def on_about(self, action, param):
150         about_dialog = Gtk.AboutDialog(transient_for=self.window, modal=True)
151         about_dialog.present()
152
153     def on_quit(self, action, param):
154         self.quit()
155
156
157 if __name__ == "__main__":
158     app = Application()
159     app.run(sys.argv)

```

24.5 See Also

- <https://wiki.gnome.org/HowDoI/GtkApplication>
- <https://wiki.gnome.org/HowDoI/GAction>
- <https://wiki.gnome.org/HowDoI/ApplicationMenu>
- <https://wiki.gnome.org/HowDoI/GMenu>

MENUS

Note: `Gtk.UIManager`, `Gtk.Action`, and `Gtk.ActionGroup` have been deprecated since GTK+ version 3.10 and should not be used in newly-written code. Use the *Application* framework instead.

GTK+ comes with two different types of menus, `Gtk.MenuBar` and `Gtk.Toolbar`. `Gtk.MenuBar` is a standard menu bar which contains one or more `Gtk.MenuItem` instances or one of its subclasses. `Gtk.Toolbar` widgets are used for quick accessibility to commonly used functions of an application. Examples include creating a new document, printing a page or undoing an operation. It contains one or more instances of `Gtk.ToolItem` or one of its subclasses.

25.1 Actions

Although, there are specific APIs to create menus and toolbars, you should use `Gtk.UIManager` and create `Gtk.Action` instances. Actions are organised into groups. A `Gtk.ActionGroup` is essentially a map from names to `Gtk.Action` objects. All actions that would make sense to use in a particular context should be in a single group. Multiple action groups may be used for a particular user interface. In fact, it is expected that most non-trivial applications will make use of multiple groups. For example, in an application that can edit multiple documents, one group holding global actions (e.g. quit, about, new), and one group per document holding actions that act on that document (eg. save, cut/copy/paste, etc). Each window's menus would be constructed from a combination of two action groups.

Different classes representing different types of actions exist:

- `Gtk.Action`: An action which can be triggered by a menu or toolbar item
- `Gtk.ToggleAction`: An action which can be toggled between two states
- `Gtk.RadioAction`: An action of which only one in a group can be active
- `Gtk.RecentAction`: An action of which represents a list of recently used files

Actions represent operations that the user can perform, along with some information how it should be presented in the interface, including its name (not for display), its label (for display), an accelerator, whether a label indicates a tooltip as well as the callback that is called when the action gets activated.

You can create actions by either calling one of the constructors directly and adding them to a `Gtk.ActionGroup` by calling `Gtk.ActionGroup.add_action()` or `Gtk.ActionGroup.add_action_with_accel()`, or by calling one of the convenience functions:

- `Gtk.ActionGroup.add_actions()`,
- `Gtk.ActionGroup.add_toggle_actions()`
- `Gtk.ActionGroup.add_radio_actions()`.

Note that you must specify actions for sub menus as well as menu items.

25.2 UI Manager

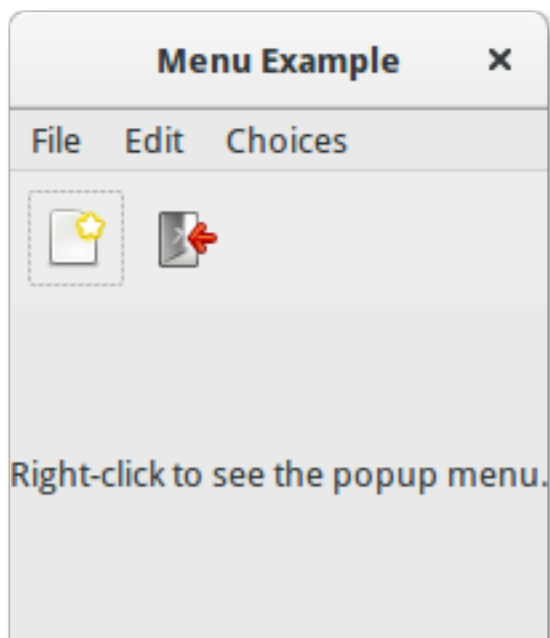
`Gtk.UIManager` provides an easy way of creating menus and toolbars using an XML-like description.

First of all, you should add the `Gtk.ActionGroup` to the UI Manager with `Gtk.UIManager.insert_action_group()`. At this point is also a good idea to tell the parent window to respond to the specified keyboard shortcuts, by using `Gtk.UIManager.get_accel_group()` and `Gtk.Window.add_accel_group()`.

Then, you can define the actual visible layout of the menus and toolbars, and add the UI layout. This “ui string” uses an XML format, in which you should mention the names of the actions that you have already created. Remember that these names are just the identifiers that we used when creating the actions. They are not the text that the user will see in the menus and toolbars. We provided those human-readable names when we created the actions.

Finally, you retrieve the root widget with `Gtk.UIManager.get_widget()` and add the widget to a container such as `Gtk.Box`.

25.3 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk, Gdk
5
6 UI_INFO = """
7 <ui>
8   <menubar name='MenuBar'>
9     <menu action='FileMenu'>
```

(continues on next page)

(continued from previous page)

```

10     <menu action='FileNew'>
11         <menuitem action='FileNewStandard' />
12         <menuitem action='FileNewFoo' />
13         <menuitem action='FileNewGoo' />
14     </menu>
15     <separator />
16     <menuitem action='FileQuit' />
17 </menu>
18 <menu action='EditMenu'>
19     <menuitem action='EditCopy' />
20     <menuitem action='EditPaste' />
21     <menuitem action='EditSomething' />
22 </menu>
23 <menu action='ChoicesMenu'>
24     <menuitem action='ChoiceOne' />
25     <menuitem action='ChoiceTwo' />
26     <separator />
27     <menuitem action='ChoiceThree' />
28 </menu>
29 </menubar>
30 <toolbar name='ToolBar'>
31     <toolitem action='FileNewStandard' />
32     <toolitem action='FileQuit' />
33 </toolbar>
34 <popup name='PopupMenu'>
35     <menuitem action='EditCopy' />
36     <menuitem action='EditPaste' />
37     <menuitem action='EditSomething' />
38 </popup>
39 </ui>
40 """
41
42
43 class MenuExampleWindow(Gtk.Window):
44     def __init__(self):
45         super().__init__(title="Menu Example")
46
47         self.set_default_size(200, 200)
48
49         action_group = Gtk.ActionGroup(name="my_actions")
50
51         self.add_file_menu_actions(action_group)
52         self.add_edit_menu_actions(action_group)
53         self.add_choices_menu_actions(action_group)
54
55         uimanager = self.create_ui_manager()
56         uimanager.insert_action_group(action_group)
57
58         menubar = uimanager.get_widget("/MenuBar")
59
60         box = Gtk.Box(orientation=Gtk.Orientation.VERTICAL)
61         box.pack_start(menubar, False, False, 0)
62
63         toolbar = uimanager.get_widget("/ToolBar")
64         box.pack_start(toolbar, False, False, 0)
65
66         eventbox = Gtk.EventBox()

```

(continues on next page)

(continued from previous page)

```

67     eventbox.connect("button-press-event", self.on_button_press_event)
68     box.pack_start(eventbox, True, True, 0)
69
70     label = Gtk.Label(label="Right-click to see the popup menu.")
71     eventbox.add(label)
72
73     self.popup = uimanager.get_widget("/PopupMenu")
74
75     self.add(box)
76
77     def add_file_menu_actions(self, action_group):
78         action_filemenu = Gtk.Action(name="FileMenu", label="File")
79         action_group.add_action(action_filemenu)
80
81         action_filenewmenu = Gtk.Action(name="FileNew", stock_id=Gtk.STOCK_NEW)
82         action_group.add_action(action_filenewmenu)
83
84         action_new = Gtk.Action(
85             name="FileNewStandard",
86             label="_New",
87             tooltip="Create a new file",
88             stock_id=Gtk.STOCK_NEW,
89         )
90         action_new.connect("activate", self.on_menu_file_new_generic)
91         action_group.add_action_with_accel(action_new, None)
92
93         action_group.add_actions(
94             [
95                 (
96                     "FileNewFoo",
97                     None,
98                     "New Foo",
99                     None,
100                    "Create new foo",
101                    self.on_menu_file_new_generic,
102                ),
103                 (
104                     "FileNewGoo",
105                     None,
106                     "_New Goo",
107                     None,
108                     "Create new goo",
109                     self.on_menu_file_new_generic,
110                ),
111             ]
112         )
113
114         action_filequit = Gtk.Action(name="FileQuit", stock_id=Gtk.STOCK_QUIT)
115         action_filequit.connect("activate", self.on_menu_file_quit)
116         action_group.add_action(action_filequit)
117
118     def add_edit_menu_actions(self, action_group):
119         action_group.add_actions(
120             [
121                 ("EditMenu", None, "Edit"),
122                 ("EditCopy", Gtk.STOCK_COPY, None, None, None, self.on_menu_others),
123                 ("EditPaste", Gtk.STOCK_PASTE, None, None, None, self.on_menu_others),

```

(continues on next page)

(continued from previous page)

```

124         (
125             "EditSomething",
126             None,
127             "Something",
128             "<control><alt>S",
129             None,
130             self.on_menu_others,
131         ),
132     ]
133 )
134
135 def add_choices_menu_actions(self, action_group):
136     action_group.add_action(Gtk.Action(name="ChoicesMenu", label="Choices"))
137
138     action_group.add_radio_actions(
139         [
140             ("ChoiceOne", None, "One", None, None, 1),
141             ("ChoiceTwo", None, "Two", None, None, 2),
142         ],
143         1,
144         self.on_menu_choices_changed,
145     )
146
147     three = Gtk.ToggleAction(name="ChoiceThree", label="Three")
148     three.connect("toggled", self.on_menu_choices_toggled)
149     action_group.add_action(three)
150
151 def create_ui_manager(self):
152     uimanager = Gtk.UIManager()
153
154     # Throws exception if something went wrong
155     uimanager.add_ui_from_string(UI_INFO)
156
157     # Add the accelerator group to the toplevel window
158     accelgroup = uimanager.get_accel_group()
159     self.add_accel_group(accelgroup)
160     return uimanager
161
162 def on_menu_file_new_generic(self, widget):
163     print("A File|New menu item was selected.")
164
165 def on_menu_file_quit(self, widget):
166     Gtk.main_quit()
167
168 def on_menu_others(self, widget):
169     print("Menu item " + widget.get_name() + " was selected")
170
171 def on_menu_choices_changed(self, widget, current):
172     print(current.get_name() + " was selected.")
173
174 def on_menu_choices_toggled(self, widget):
175     if widget.get_active():
176         print(widget.get_name() + " activated")
177     else:
178         print(widget.get_name() + " deactivated")
179
180 def on_button_press_event(self, widget, event):

```

(continues on next page)

(continued from previous page)

```
181     # Check if right mouse button was preseed
182     if event.type == Gdk.EventType.BUTTON_PRESS and event.button == 3:
183         self.popup.popup(None, None, None, None, event.button, event.time)
184         return True # event has been handled
185
186
187 window = MenuExampleWindow()
188 window.connect("destroy", Gtk.main_quit)
189 window.show_all()
190 Gtk.main()
```

TABLE

Note: `Gtk.Table` has been deprecated since GTK+ version 3.4 and should not be used in newly-written code. Use the *Grid* class instead.

Tables allows us to place widgets in a grid similar to `Gtk.Grid`.

The grid's dimensions need to be specified in the `Gtk.Table` constructor. To place a widget into a box, use `Gtk.Table.attach()`.

`Gtk.Table.set_row_spacing()` and `Gtk.Table.set_col_spacing()` set the spacing between the rows at the specified row or column. Note that for columns, the space goes to the right of the column, and for rows, the space goes below the row.

You can also set a consistent spacing for all rows and/or columns with `Gtk.Table.set_row_spacings()` and `Gtk.Table.set_col_spacings()`. Note that with these calls, the last row and last column do not get any spacing.

Deprecated since version 3.4: It is recommended that you use the `Gtk.Grid` for new code.

26.1 Example



```
1 import gi
2
3 gi.require_version("Gtk", "3.0")
4 from gi.repository import Gtk
5
6
7 class TableWindow(Gtk.Window):
```

(continues on next page)

(continued from previous page)

```
8  def __init__(self):
9      super().__init__(title="Table Example")
10
11     table = Gtk.Table(n_rows=3, n_columns=3, homogeneous=True)
12     self.add(table)
13
14     button1 = Gtk.Button(label="Button 1")
15     button2 = Gtk.Button(label="Button 2")
16     button3 = Gtk.Button(label="Button 3")
17     button4 = Gtk.Button(label="Button 4")
18     button5 = Gtk.Button(label="Button 5")
19     button6 = Gtk.Button(label="Button 6")
20
21     table.attach(button1, 0, 1, 0, 1)
22     table.attach(button2, 1, 3, 0, 1)
23     table.attach(button3, 0, 1, 1, 3)
24     table.attach(button4, 1, 3, 1, 2)
25     table.attach(button5, 1, 2, 2, 3)
26     table.attach(button6, 2, 3, 2, 3)
27
28
29 win = TableWindow()
30 win.connect("destroy", Gtk.main_quit)
31 win.show_all()
32 Gtk.main()
```

INDICES AND TABLES

- search

Symbols

`__gproperties__` (*GObject.GObject attribute*), 142
`__gsignals__` (*GObject.GObject attribute*), 142

E

`emit()` (*GObject.GObject method*), 141

F

`freeze_notify()` (*GObject.GObject method*), 141

G

`get_property()` (*GObject.GObject method*), 141
`GObject.GObject` (*built-in class*), 141

H

`handler_block()` (*GObject.GObject method*), 141
`handler_unblock()` (*GObject.GObject method*),
 141

R

`READABLE` (*GObject.ParamFlags attribute*), 142
`READWRITE` (*GObject.ParamFlags attribute*), 142

S

`set_property()` (*GObject.GObject method*), 141
`SIGNAL_RUN_CLEANUP` (*GObject attribute*), 142
`SIGNAL_RUN_FIRST` (*GObject attribute*), 142
`SIGNAL_RUN_LAST` (*GObject attribute*), 142

T

`thaw_notify()` (*GObject.GObject method*), 141

W

`WRITABLE` (*GObject.ParamFlags attribute*), 142